МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение высшего образования
«Новосибирский национальный исследовательский государственный университет»
(Новосибирский государственный университет, НГУ)
Структурное подразделение Новосибирского государственного университета – Высший колледж информатики Университета (ВКИ НГУ)
КАФЕДРА ИНФОРМАТИКИ

Разработка программного комплекса «MealSync» для автоматизации бизнес-процессов организации общественного питания ТОО «СУПЕРВКУС»

Квалификация программист

Руководитель Литвинова О.В.
Инженер ООО «Кожура»

«___» _____2026 г.

Студент 4 курса Карпова М.Е.
гр. 2207д1 «____» _____2026 г.

Новосибирск

2026

СОДЕРЖАНИЕ

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, УСЛОВНЫХ ОБОЗНАЧЕНИЙ И ТЕРМИНОВ 5
ВВЕДЕНИЕ 6
1 ПОСТАНОВКА ЗАДАЧИ ВКР 8
1.1 Бизнес-требования 8
1.2 Системные требования 9
1.3 Пользовательские требования 11
1.4 Требования к пользовательскому интерфейсу 12
1.4.1 Общие требования к интерфейсу 12
1.4.2 Требования к десктопному интерфейсу 13
1.4.3 Требования к веб-интерфейсу 14
1.5 План-график выполнения ВКР 14
2 АНАЛИЗ ТРЕБОВАНИЙ И ОПРЕДЕЛЕНИЕ СПЕЦИФИКАЦИЙ 16
2.1 Описание предметной области задачи ВКР 16
2.1.1 Характеристика деятельности ТОО «СуперВкус» 16
2.1.2 Информационные объекты предметной области и взаимосвязи между ними 19
2.1.3 Информационные потребности пользователей 20
2.1.4 Методы работы с информационными объектами предметной области 22
2.1.4.1 Способы хранения информации об объектах предметной области 22
2.1.4.2 Математические модели, используемые для обработки информации 24
2.1.4.3 Способы интерпретации и визуального представления информации 29
2.1.4.4 Технологии получения и передачи информации 31
2.1.5 Обзор существующих программных реализаций решения задачи 32
2.2 Классы и характеристики пользователей 37
2.3 Функциональные требования 38
2.3.1 Определение функциональных возможностей 38
2.3.2 Описание прецедентов 42
2.4 Нефункциональные требования 45
3 ВЫБОР ПРОГРАММНЫХ СРЕД И СРЕДСТВ РАЗРАБОТКИ 47
3.1 Выбор языка программирования 47
3.2 Выбор инструментария для десктоп-интерфейса 47
3.3 Выбор фреймворка для серверной части 48
3.4 Выбор базы данных 48
3.5 Выбор инструментов для доступа к данным и миграций 49
3.6 Выбор технологии для веб-интерфейса 49

3.7 Выбор вспомогательных инструментов....50
3.8 Обобщение результатов выбора....50
4 АЛГОРИТМЫ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ....52
4.1 Этапы реализации....52
4.2 Пользовательский интерфейс....53
4.2.1 Описание взаимодействия всех пользователей с системой....53
4.2.1.1 Описание взаимодействия технолога с системой....55
4.2.1.2 Описание взаимодействия кладовщика с системой....56
4.2.1.3 Описание взаимодействия повара с системой....57
4.2.1.4 Описание взаимодействия руководителя с системой....57
4.2.1.5 Описание взаимодействия гостя с системой....58
4.2.2 Определение операций пользователей и составление функциональных блоков....59
4.2.3 Проектирование структуры экранов и схемы навигации....62
4.2.4 Разработка дизайна интерфейса....65
4.3 Входные, выходные и промежуточные данные....68
4.4 Реализация используемых методов хранения, обработки и передачи информации об объектах предметной области....72
4.4.1 Методы хранения данных....72
4.4.2 Алгоритмы реализации используемых математических моделей....81
4.4.3 Алгоритмы применения методов графического анализа данных....86
4.4.4 Алгоритмы использования технологий передачи данных....90
4.5 Описание архитектурного решения....92
4.5.1 Структурная организация программной системы....92
4.5.2 Архитектура программного кода....96
5 ТЕСТИРОВАНИЕ И ОПТИМИЗАЦИЯ....100
5.1 План тестирования....100
5.2 Функциональное тестирование....101
5.3 Тестирование пользовательского интерфейса....102
5.4 Нагрузочное тестирование....103
5.5 Тестирование безопасности....103
5.6 Общие выводы....104
6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ....105
6.1 Вход на сайт....105
6.2 Просмотр публичного меню....107
6.3 Установка настольного приложения....108
6.4 Работа технолога....109

6.4.1 Работа технолога в веб-интерфейсе 110
6.4.2 Работа технолога в настольном приложении 112
6.5 Работа кладовщика 116
6.5.1 Работа кладовщика в веб-интерфейсе 116
6.5.2 Работа кладовщика в настольном приложении 117
6.6 Работа повара 119
6.6.1 Работа повара в веб-интерфейсе 119
6.6.2 Работа повара в настольном приложении 120
6.7 Работа руководителя 121
6.8 Завершение работы 123
6.9 Действия при возникновении ошибок 123
ЗАКЛЮЧЕНИЕ 124
СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ 127
ПРИЛОЖЕНИЯ 128
Приложение А 128
Приложение Б 133
Приложение В 134
Приложение Г 135

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ, УСЛОВНЫХ ОБОЗНАЧЕНИЙ И ТЕРМИНОВ

Калорийность – количественная характеристика энергетической ценности пищевого продукта или готового блюда, выражаемая в килокалориях на 100 г массы или на порцию. Используется для оценки энергетического баланса и контроля соответствия нормативам.

Белки, жиры, углеводы (КБЖУ) – группа ключевых нутриентов, определяющих биологическую и энергетическую ценность пищи. Параметры КБЖУ применяются при расчётах пищевой ценности технологических карт.

Масса брутто – масса сырьевого продукта в исходном состоянии до выполнения технологических операций. Включает несъедобные и отходные фракции. Используется при нормировании и составлении рецептур.

Масса нетто – масса сырья после технологической обработки, являющаяся фактическим количеством, участвующим в приготовлении блюда. Используется в расчётах выхода продукции и себестоимости.

Процент потерь – относительная величина, отражающая уменьшение массы продукта вследствие механической или тепловой обработки. Применяется для корректировки массы ингредиентов и экономических расчётов.

Технологическая карта– нормативный документ, содержащий структурированное описание состава блюда, норм закладки сырья, этапов технологической обработки, выхода готовой продукции, а также расчётов пищевой ценности и себестоимости. Используется в предприятиях общественного питания для стандартизации производственных процессов.

Норма закладки – установленное количество продукта, требуемое для приготовления блюда заданного выхода. Регламентируется методическими указаниями и отраслевыми нормами.

Себестоимость блюда – интегральный экономический показатель, отражающий стоимость ингредиентов с учётом их потерь, фактической массы и текущих цен. Используется в расчётах экономической эффективности.

ВВЕДЕНИЕ

Глобальная индустрия общественного питания в последние годы демонстрирует устойчивый переход к использованию цифровых технологий. Международные предприятия активно внедряют программные решения для автоматизации расчётов пищевой ценности, себестоимости и параметров технологических карт [1], [2]. Такие системы позволяют снижать вероятность ошибок, поддерживать стабильное качество блюд и повышать эффективность производственных процессов в условиях роста объёмов выпуска.

Несмотря на общий тренд, для малых и средних столовых переход к цифровым инструментам остаётся сложной задачей. Значительная часть таких предприятий продолжает использовать несвязанные электронные таблицы, отдельные файлы и бумажные формы. Это приводит к фрагментарности данных, отсутствию единого хранилища рецептур и невозможности оперативно обновлять документацию. Усиление требований к качеству, безопасности питания и контролю производственных операций делает необходимым внедрение доступных программных инструментов, которые обеспечат структурированное хранение и обработку технологической информации.

Особенно остро данная проблема стоит в столовой «СуперВкус» — заказчике данной разработки. Ручное ведение технологических карт, расчёт пищевой ценности и себестоимости блюд не только отнимают значительную часть рабочего времени технолога, но и создают риск выпуска блюд с неподтверждёнными показателями. Отсутствие единой цифровой системы замедляет реакцию на изменения цен поставщиков, усложняет контроль за остатками на складе и ведёт к прямым финансовым потерям из-за ошибок в списании продуктов. Ограниченность кадровых и временных ресурсов диктует необходимость в компактном и надёжном программном решении, способном автоматизировать основные производственные операции и обеспечить единый порядок работы с данными.

В этих условиях разработка специализированной информационной системы становится актуальной и практически значимой задачей.

Цель работы состоит в разработке информационной системы «MealSync», включающей десктопный и веб-модули, предназначенной для автоматизации технологических и складских процессов предприятия общественного питания. Система должна обеспечивать ведение технологических карт, расчёт пищевой ценности и себестоимости блюд, формирование и публикацию меню, обработку заявок между производством и складом, а также структурированное хранение нормативной и производственной информации.

Задачи выпускной квалификационной работы:

1. Изучить современные подходы к автоматизации предприятий общественного питания и определить актуальные проблемы малых и средних столовых.
2. Выполнить анализ предметной области и описать процессы технологического и складского документооборота.
3. Оценить существующие программные решения и определить ограничения их применения для целевого формата предприятия.
4. Сформировать функциональные требования к системе с учётом ролевой модели доступа (технолог, повар, кладовщик, руководитель, публичный пользователь).
5. Спроектировать архитектуру решения «десктоп + веб + серверный API» и модель хранения данных с разделением по филиалам.
6. Реализовать десктопный модуль ведения технологических карт, расчётов КБЖУ/себестоимости, формирования меню и складских документов.
7. Реализовать веб-модули: публичный просмотр меню и рабочие кабинеты сотрудников с доступом по ролям.
8. Разработать механизмы синхронизации данных, экспорта и печати документов, а также формирования управленческой отчётности.
9. Выполнить тестирование системы и оценить соответствие полученного результата установленным требованиям.

Технологическая основа разработки включает язык Python, серверный фреймворк FastAPI, ORM-уровень SQLAlchemy и механизм миграций Alembic. Десктопный интерфейс реализуется на PySide6, веб-интерфейс – на HTML, CSS и JavaScript. Хранение данных организовано в SQL-совместимом хранилище с общей нормативной частью и раздельным операционным учётом по филиалам.

1 ПОСТАНОВКА ЗАДАЧИ ВКР

1.1 Бизнес-требования

Деятельность организации ТОО «Супервкус» связана с ежедневным приготовлением широкого ассортимента блюд, ведением технологической документации, учётом движения продуктов на складе и формированием отчётности для руководителя. В настоящее время перечисленные процессы выполняются с использованием разрозненных электронных таблиц и бумажных носителей, что приводит к ошибкам в расчётах, дублированию информации и замедлению принятия управленческих решений. Для устранения этих недостатков требуется внедрение единой автоматизированной системы, которая объединит технологические, складские и управленческие функции в рамках всего предприятия.

Разрабатываемый программный комплекс «MealSync» предназначен для автоматизации ключевых бизнес-процессов столовой «Супервкус».

Основные бизнес-требования, которым должна удовлетворять система:

– Сократить трудозатраты на ведение технологической документации и выполнение расчётов. Автоматизация расчётов себестоимости, пищевой ценности и норм закладки продуктов позволит технологу тратить меньше времени на рутинные операции [1].

– Обеспечить достоверность и актуальность нормативно-справочной информации для всех филиалов. Единое централизованное хранилище продуктов, технологических карт и меню исключает ситуацию, когда в разных филиалах используются разные версии одних и тех же документов. Любое изменение, внесённое уполномоченным сотрудником, становится доступным всем подразделениям мгновенно.

– Повысить эффективность складского учёта и снизить потери от неконтролируемого движения продуктов. Автоматическая обработка заявок между производством и складом, контроль движения продуктов и своевременное информирование о критических остатках или дефиците позволит кладовщику своевременно выявлять дефицит продуктов и предотвращать простои производства из-за отсутствия сырья.

– Ускорить подготовку управленческой отчётности и обеспечить прозрачность производственных процессов для руководителя. Формирование сводных отчётов по филиалам в разрезе остатков, расхода сырья, себестоимости выпуска и статусов заявок даёт руководителю возможность оперативно выявлять отклонения, сравнивать эффективность разных столовых и принимать обоснованные управленческие решения без задержек, связанных с ручным сбором данных.

– Обеспечить непрерывность ключевых производственных операций при временных сбоях интернет-соединения. Сохранение работоспособности функций приёмки продуктов, формирования заявок и списания при отсутствии сети гарантирует, что производственный процесс не остановится из-за технических сбоев. После восстановления соединения накопленные изменения автоматически передаются на сервер, что обеспечивает целостность учёта.

– Повысить удобство для конечных потребителей и снизить нагрузку на сотрудников. Публикация актуального меню с возможностью выбора филиала, просмотра состава блюд, калорийности, цены и фотографий через веб-интерфейс позволяет посетителям самостоятельно получать интересующую информацию, разгружая сотрудников от телефонных консультаций и улучшая общее качество обслуживания.

1.2 Системные требования

Программный комплекс «MealSync» представляет собой распределённую информационную систему, состоящую из нескольких взаимодействующих компонентов. Целесообразно рассматривать состав системы с двух позиций: архитектурные компоненты (физические или логические части системы, обеспечивающие её развёртывание и взаимодействие) и функциональные модули (наборы функций, реализующие конкретные бизнес-задачи). Ниже приведено описание состава программного комплекса в соответствии с указанным разделением.

Архитектурные компоненты:

– Серверная часть (бэкенд). Обеспечивает централизованную обработку запросов от всех клиентских компонентов, управление бизнес-логикой (расчёты себестоимости, КБЖУ, формирование отчётов, обработка заявок), а также взаимодействие с базой данных. Серверная часть реализует единый API, через который десктопное и веб-приложения обмениваются данными.

– База данных. Представляет собой единое хранилище всей информации системы. Содержит нормативно-справочную часть (продукты, единицы измерения, категории блюд, технологические карты) и операционные данные (меню по филиалам, заявки, движения продуктов, результаты инвентаризаций). Обеспечивает целостность, согласованность и надёжное хранение данных.

– Десктопное приложение. Локальный клиент, устанавливаемый на рабочие места сотрудников (технолог, кладовщик, руководитель). Работает с локальной копией базы данных, поддерживает автономную работу при отсутствии связи с сервером

с последующей синхронизацией. Обеспечивает ведение технологических карт, расчёты себестоимости и КБЖУ, формирование меню, учёт складских операций (приход, расход, инвентаризация), подготовку печатных форм документов.

– Веб-приложение. Доступно в двух вариантах: публичная часть для посетителей (гостей) без авторизации, позволяющая просматривать актуальное меню с выбором филиала, отображением состава блюда, пищевой ценности, выхода порции, цены и фотографий; а также веб-кабинеты для авторизованных сотрудников с ролями «повар» (и опционально «кладовщик» / «руководитель») для выполнения ограниченного круга задач без установки десктопного приложения.

Функциональные модули:

– Модуль управления нормативно-справочной информацией. Обеспечивает ведение справочников: продукты, единицы измерения, категории блюд, цеха, филиалы. Поддерживает централизованное хранение с автоматическим распространением изменений на все филиалы.
– Модуль технологических карт. Обеспечивает создание, редактирование и хранение технологических карт. Включает расчёт брутто/нетто, КБЖУ, себестоимости блюда, норм закладки и выхода готового изделия.
– Модуль меню. Обеспечивает формирование и публикацию меню по филиалам с привязкой к датам. Поддерживает выборку блюд из технологических карт, установку цен и сортировку.
– Модуль складского учёта. Обеспечивает учёт движения продуктов: приход, списание, инвентаризация. Ведёт учёт остатков в разрезе филиалов и складов.
– Модуль заявок. Обеспечивает создание, согласование и обработку заявок между производством и складом. Поддерживает обработку заявок, контроль доступных остатков и учёт фактической выдачи продуктов.
– Модуль отчётности и аналитики. Обеспечивает формирование управленческих отчётов: остатки, обороты, расход сырья, себестоимость выпуска, статусы заявок. Поддерживает сводку по филиалам и экспорт данных.
– Модуль экспорта и печати. Обеспечивает экспорт технологических карт, меню и данных инвентаризации в форматы PDF и Excel.
– Модуль синхронизации данных. Обеспечивает обмен информацией между десктопными клиентами и сервером. Поддерживает автономную работу с локальной копией и последующую синхронизацию при восстановлении сети.

– Модуль аутентификации и ролевого доступа обеспечивает аутентификацию пользователей и разграничение прав доступа на основе ролей (технолог, повар, кладовщик, руководитель, гость) [3].

1.3 Пользовательские требования

Программный комплекс «MealSync» должен предоставлять широкий и разнообразный набор возможностей, ориентированный на потребности различных категорий пользователей. К ним относятся как незарегистрированные посетители (гости), так и зарегистрированные сотрудники предприятия, включая технолога, повара, кладовщика и руководителя. Для каждой из этих групп предусмотрен свой набор инструментов, направленный на упрощение выполнения повседневных производственных задач, автоматизацию расчётов, контроль движения продуктов и эффективную организацию технологических и складских процессов.

Пользовательские требования были сформированы на основе технического задания, представленного в приложении А, согласно которому были выделены следующие требования:

1. Ролевое разграничение доступа. Система должна предоставлять различные наборы функций в зависимости от роли пользователя: технолог, повар, кладовщик, руководитель, гость (публичный пользователь). Пользователь должен видеть только те функции, которые соответствуют его должностным обязанностям, и не иметь доступа к избыточным возможностям.
2. Работа технолога с нормативно-справочной информацией. Технолог должен иметь возможность создавать и редактировать технологические карты, выполнять автоматизированный расчёт КБЖУ и себестоимости блюда, формировать меню для филиала, а также создавать производственные заявки на склад.
3. Работа кладовщика со складским учётом. Кладовщик должен получать доступ к поступившим заявкам, сверять их с фактическими остатками, оформлять приход и расход продукции, а также проводить инвентаризацию складских запасов.
4. Работа повара с производственной информацией. Повар должен иметь возможность просматривать утверждённые технологические карты и опубликованное меню, используя их как основу для приготовления блюд. Редактирование технологических карт и выполнение складских операций повару не требуется.
5. Доступ руководителя к аналитике и отчётности. Руководитель должен получать доступ к сводным аналитическим и отчетным данным по филиалу (остатки,

расход сырья, себестоимость выпуска, статусы заявок) для оценки состояния производства и принятия управленческих решений.

6. Публичный доступ для гостей. Гость (внешний пользователь) должен иметь возможность просматривать опубликованное меню с выбором филиала, составом блюд, пищевой ценностью, выходом порции, ценой и фотографиями без необходимости авторизации в системе.
7. Удобство повседневного использования. Пользователи должны иметь возможность быстро находить нужные данные с помощью средств поиска, фильтрации и сортировки, а также выбирать филиал для просмотра информации. Интерфейс должен быть интуитивно понятным, а функции — сгруппированы в соответствии с ролью пользователя.
8. Поддержка печатных форм документов. Система должна обеспечивать формирование печатных форм (пригодных для вывода на бумагу) для технологических карт, меню и складских документов, поскольку указанные документы могут использоваться в бумажном виде в производственной деятельности.
9. Надёжность сохранения данных. Пользователь должен быть уверен, что введённые данные не будут потеряны, а сохранённые документы останутся доступными для дальнейшего использования. Система должна гарантировать целостность и сохранность информации.
10. Локальная работа с синхронизацией. Для сотрудников, работающих в десктопном модуле, должна быть предусмотрена возможность локальной работы при временном отсутствии интернет-соединения с последующей синхронизацией изменений после восстановления связи.

1.4 Требования к пользовательскому интерфейсу

Пользовательский интерфейс программного комплекса «MealSync» должен обеспечивать оперативный доступ к ключевым операциям технологического, складского и управленческого контуров: ведению технологических карт, редактированию состава блюд, работе со справочниками, формированию меню, обработке заявок и просмотру отчётности.

1.4.1 Общие требования к интерфейсу

К общим требованиям к интерфейсу, применимым как к десктопному, так и к веб-приложению, относятся следующие:

– Ролевая доступность разделов. Основные разделы должны быть доступны сразу после входа и формироваться в зависимости от роли пользователя: для технолога – технологические карты, продукты, меню, заявки; для кладовщика – остатки, приход, расход, заявки, инвентаризация; для повара – просмотр утверждённых технологических карт и меню; для руководителя – аналитика и отчётность.

– Единообразие поиска и фильтрации. Поиск и фильтрация должны быть реализованы единообразно во всех модулях: по наименованию блюда, продукта, категории, дате и статусу документа, с мгновенным открытием выбранной записи и возвратом к списку без потери контекста.

– Группировка элементов управления. Элементы управления должны быть визуально разделены по назначению (действия с данными, действия с документом, действия экспорта), чтобы снизить вероятность ошибочных операций и ускорить выполнение типовых сценариев работы.

– Автоматическое обновление данных. Обновление данных на экране должно происходить автоматически после ввода или сохранения, обеспечивая прозрачность расчётов и актуальность отображаемой информации.

– Единый принцип экспорта. Система должна поддерживать экспорт отдельных документов и табличных данных в предусмотренных модулях.

1.4.2 Требования к десктопному интерфейсу

Для десктопного приложения, используемого технологом, кладовщиком и руководителем, предъявляются следующие требования:

– Главная форма приложения. Главная форма десктопного приложения должна содержать постоянную навигационную панель и единую рабочую область, чтобы пользователь выполнял основные действия без перехода через цепочку отдельных окон. Элементы интерфейса требуется группировать по бизнес-логике: отдельные блоки для данных блюда, состава ингредиентов, расчётных показателей, складских документов и итоговой аналитики. Расположение компонентов должно позволять выполнять полный цикл операций в пределах одного экрана модуля с минимальным количеством переходов.

– Интерфейс модуля технологических карт. Формы создания и редактирования технологических карт должны включать чётко структурированные поля: категория блюда, цех, выход, состав ингредиентов, брутто, потери, нетто, КБЖУ, себестоимость, технологическое описание и фото блюда. Логика ввода должна исключать ручное форматирование документов. Рабочая область модуля должна одновременно отображать список блюд, карточку

выбранного блюда, состав и блок итоговых расчётов, чтобы пользователь видел все ключевые показатели без открытия дополнительных окон.

– Интерфейс модуля меню. Модуль меню должен поддерживать выбор даты и филиала, добавление блюд по приёмам пищи, утверждение и публикацию, а также быстрый переход к формированию заявки на склад по данным меню.

– Интерфейс складского контура. Интерфейс складского контура должен обеспечивать последовательную обработку документов движения: создание заявок, изменение статусов, подтверждение частичной или полной выдачи, просмотр остатков и фиксацию инвентаризационных расхождений по каждому филиалу.

1.4.3 Требования к веб-интерфейсу

Для веб-интерфейса, включающего публичную часть для посетителей и кабинет сотрудников, предъявляются следующие требования:

– Публичная страница меню. Веб-интерфейс должен содержать публичную страницу меню для посетителей, которая отображает состав блюда, КБЖУ, выход порции, цену и фотографию. Авторизация для просмотра публичной страницы не требуется.

– Кабинет сотрудников. Веб-интерфейс должен содержать отдельный кабинет сотрудников с ролевым доступом (повар, опционально кладовщик и руководитель), обеспечивающий выполнение ограниченного круга задач без установки десктопного приложения.

– Переключение языка интерфейса. Веб-интерфейс должен поддерживать переключение языка для удобства пользователей.

1.5 План-график выполнения ВКР

Представление календарного плана в табличной форме позволяет структурировать процесс разработки, выделить основные стадии проектирования и реализации программного средства, а также установить временные границы для выполнения каждой из них. Такой подход обеспечивает наглядное отображение последовательности действий, упрощает контроль за ходом выполнения проекта и позволяет своевременно соотносить достигнутые результаты с запланированными сроками. Все этапы выполнения выпускной квалификационной работы, а также соответствующие им сроки реализации сведены в таблицу 1.5.1.

Таблица 1.5.1 – График работ

№	Наименование этапа	Количество недель	Даты выполнения
1.	Анализ предметной области	2	12.01.2026 – 26.01.2026
2.	Формирование требований	2	27.01.2026 – 07.02.2026
3.	Проектирование архитектуры	2	08.02.2026 – 20.02.2026

№	Наименование этапа	Количество недель	Даты выполнения
4.	Проектирование интерфейса	2	21.02.2026 – 09.03.2026
5.	Проектирование модели данных	2	10.03.2026 – 20.03.2026
6.	Реализация модулей	2	21.03.2026 – 20.04.2026
7.	Интеграция компонентов	2	21.04.2026 – 03.05.2026
8.	Тестирование ПС	2	04.05.2026 – 15.05.2026
9.	Оптимизация ПС	2	16.05.2026 – 23.05.2026
10.	Подготовка документации	2	24.05.2026 – 31.05.2026

2 АНАЛИЗ ТРЕБОВАНИЙ И ОПРЕДЕЛЕНИЕ СПЕЦИФИКАЦИЙ

2.1 Описание предметной области задачи ВКР

2.1.1 Характеристика деятельности ТОО «СуперВкус»

ТОО «СуперВкус» является предприятием общественного питания, включающее три филиала различной специализации:

- столовая, расположенная на конечной автобусной остановке — обслуживает рабочих, водителей и кондукторов, приоритетами являются скорость получения пищи, насыщенность блюд и доступная цена;
- столовая-буфет при больнице — ориентирована на посетителей с особыми требованиями к пищевой ценности и составу блюд, приоритетами выступают диетические характеристики, калорийность и ограничения по продуктам;
- столовая в административном корпусе металлургического завода — обслуживает офисных сотрудников, приоритетами являются разнообразие меню, стабильное качество и возможность предварительного ознакомления с ассортиментом.

Несмотря на различия в профилях, все три филиала работают под единым управлением и используют общую нормативно-справочную базу, включающую технологические карты и справочники продуктов.

В настоящее время технологические и складские процессы в ТОО «СуперВкус» выполняются преимущественно вручную с использованием бумажных документов и электронных таблиц.

Технолог ведёт технологические карты в Excel. При изменении рецептуры или цен на продукты себестоимость и КБЖУ пересчитываются вручную, после чего обновлённые файлы рассылаются или копируются по трём филиалам. Меню формируется отдельно для каждого филиала на основе техкарт ручным отбором блюд. На основе утверждённого меню составляется заявка на склад, расчёт объёмов которой выполняется вручную или в Excel, что при ассортименте из десятков наименований создаёт высокую вероятность ошибок.

Складской учёт ведётся кладовщиком в Excel-файлах индивидуально по каждому филиалу: оформляются приход и расход продуктов, при обработке заявок запрошенное количество вручную сверяется с фактическими остатками.

Повар использует бумажные или файловые версии технологических карт, при этом разные филиалы могут работать с разными версиями одного документа, что приводит к расхождениям в составе и себестоимости блюд.

Отчётность для руководителя формируется путём ручного сбора данных по остаткам, расходу сырья и себестоимости из разрозненных Excel-файлов трёх филиалов, что занимает значительное время и не гарантирует достоверности.

Анализ текущего состояния деятельности ТОО «СуперВкус» позволяет выделить следующие системные проблемы, препятствующие эффективной работе организации:

- разрозненность данных — информация о продуктах, технологических картах, остатках и заявках хранится в разных файлах и на разных компьютерах, отсутствует единое хранилище;
- высокая вероятность ошибок — ручной расчёт себестоимости, КБЖУ и объёмов заявок неизбежно приводит к арифметическим и логическим ошибкам;
- задержки при передаче информации — при изменении технологической карты технолог должен вручную уведомить все заинтересованные стороны, что приводит к использованию разных версий документов в разных филиалах;
- трудозатраты на формирование отчётности — руководитель тратит несколько часов в неделю на ручной сбор данных из трёх филиалов;
- отсутствие контроля остатков в реальном времени — кладовщик узнаёт о дефиците продукта только в момент обработки заявки, что приводит к простоям производства и экстренным закупкам;
- риски для качества блюд — использование устаревших версий технологических карт может приводить к нарушению рецептуры, что особенно критично для больничной столовой.

На рисунке 2.1.1.1 схематично представлен существующий документооборот между участниками процессов. Схема наглядно иллюстрирует перечисленные проблемы: разрозненность носителей, отсутствие централизованного хранилища и многократное дублирование данных.

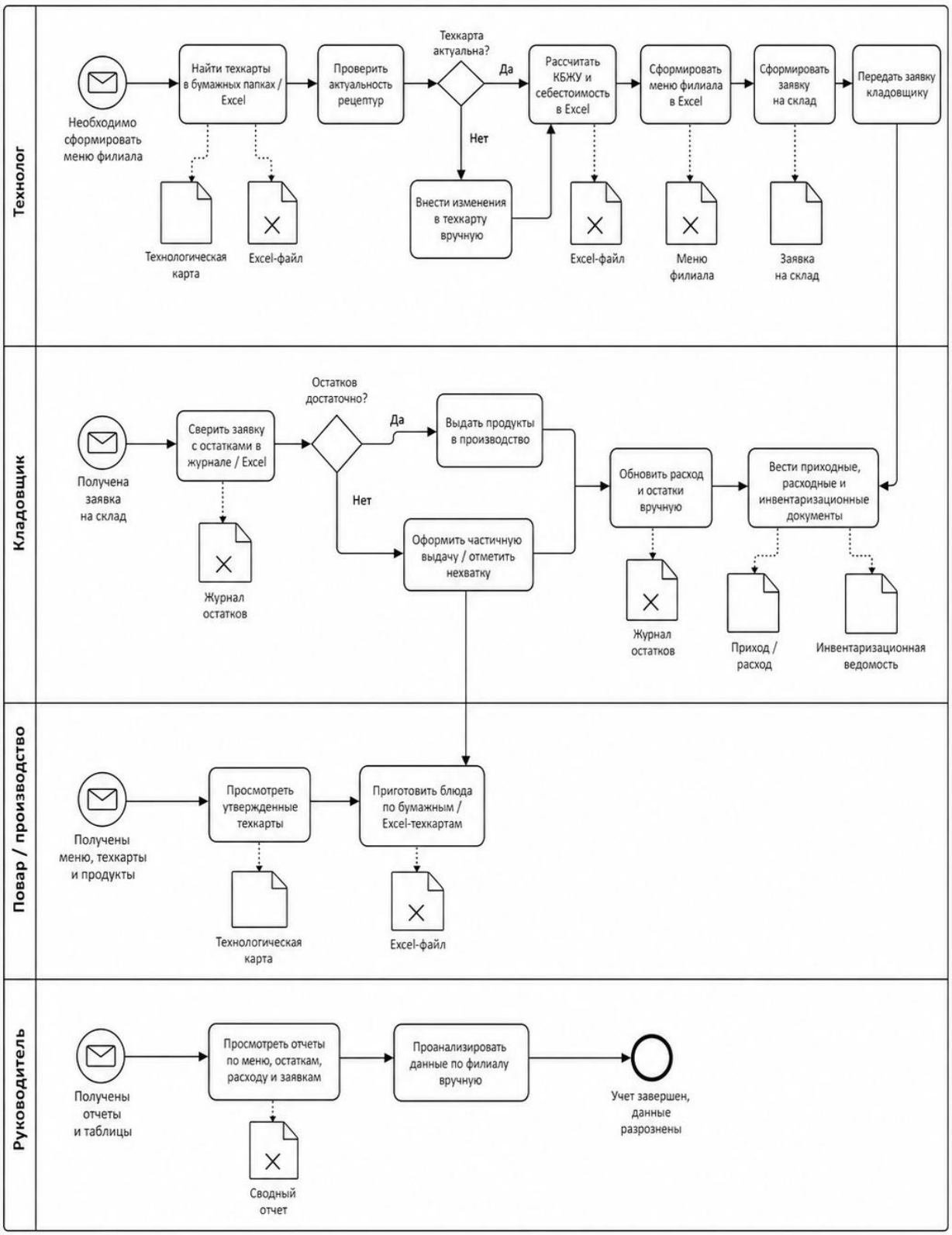


Рисунок 2.1.1.1 – Схема документооборота ТОО «СуперВкус» (существующий порядок)

Как показано на рисунке 2.1.1.1, ключевыми участниками процесса являются технолог, кладовщик, повар и руководитель. Основные документы (технологические карты, меню, заявки на склад, журналы остатков, приходно-расходные документы, отчёты) передаются между участниками в виде бумажных или файловых копий, что приводит к

многократному дублированию одних и тех же данных, задержкам при передаче обновлённых версий и высокой вероятности ошибок при ручном пересчёте.

Сложившийся способ ведения технологической и складской документации в ТОО «СуперВкус» достиг предела своей эффективности. Ручные методы не позволяют обеспечить требуемую скорость, точность и согласованность данных при работе трёх филиалов. В этих условиях единственным способом устранения перечисленных недостатков является внедрение специализированного программного средства, которое обеспечит:

- единое централизованное хранение нормативно-справочной информации;
- автоматизированные расчёты себестоимости, КБЖУ и объёмов заявок;
- ролевой доступ и разграничение функций между сотрудниками;
- раздельный операционный учёт по филиалам при единой нормативной базе;
- формирование сводной отчётности в автоматическом режиме;
- поддержку непрерывной работы при временных сбоях связи.

2.1.2 Информационные объекты предметной области и взаимосвязи между ними

Деятельность ТОО «СуперВкус» оперирует совокупностью информационных объектов, которые создаются, изменяются и передаются в ходе технологических, складских и управленческих процессов. Выделение и классификация этих объектов необходима для последующего проектирования структуры хранения данных в автоматизированной системе.

Все информационные объекты, используемые в деятельности организации, делятся на четыре группы: нормативно-справочные, плановые, учётные и аналитические. Их состав и назначение приведены в таблице 2.1.2.1.

Таблица 2.1.2.1 – Информационные объекты предметной области

Группа	Объекты	Назначение
Нормативно-справочные	Справочник продуктов, категории блюд, цехов; технологическая карта	Содержат условно-постоянную информацию, используемую во всех филиалах
Плановые	Меню филиала, заявка на склад	Формируются на определённый период, определяют план производства и обеспечения
Учётные	Журнал остатков, приходные и расходные документы, акт инвентаризации	Фиксируют фактические события движения продуктов
Аналитические	Сводный отчёт	Формируются на основе учётных данных для управления и контроля

Технологическая карта является основным нормативным документом, содержащим рецептуру блюда, нормы закладки, процент потерь, выход, а также расчётные показатели (пищевую ценность, калорийность, себестоимость). Она служит источником данных для

большинства других объектов: изменение состава или норм закладки в технологической карте влечёт пересчёт показателей во всех связанных документах.

На рисунке 2.1.2.1 представлена схема взаимосвязей информационных объектов. Стрелками показаны основные информационные потоки.

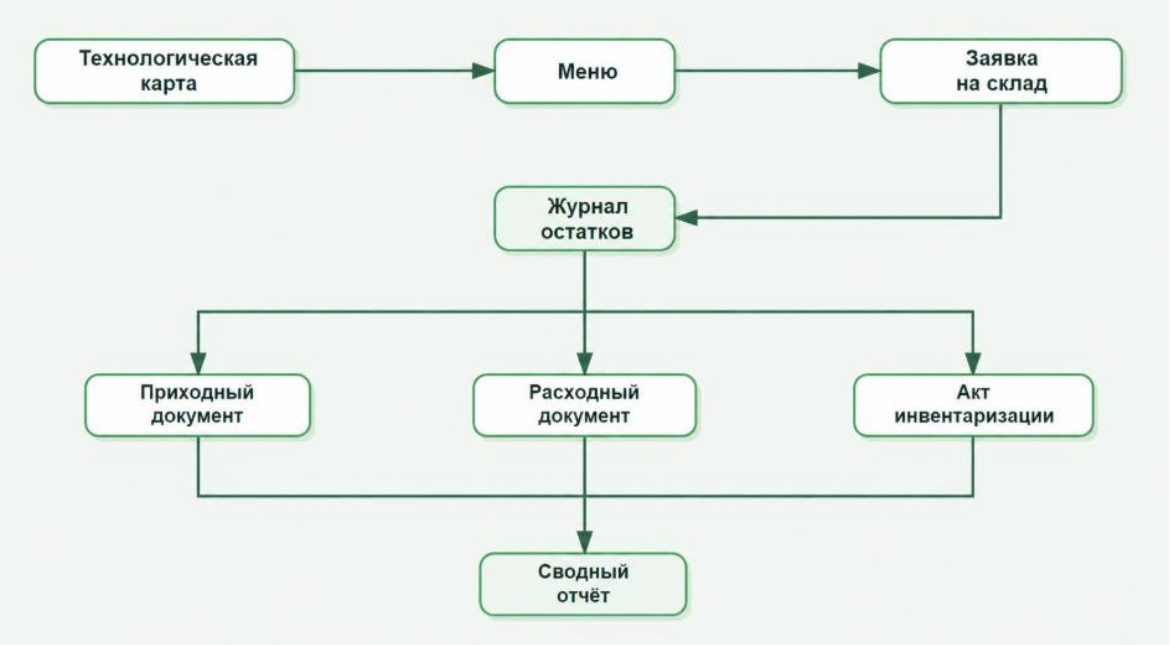


Рисунок 2.1.2.1 – Взаимосвязи информационных объектов предметной области

Как показано на рисунке 2.1.2.1, информационные потоки имеют следующую направленность. Технологическая карта является исходным объектом для формирования меню. Меню служит основанием для создания заявки на склад. Заявка сверяется с журналом остатков. Выдача продуктов оформляется расходным документом, поступление — приходным документом, которые изменяют остатки. Данные из учётных документов используются при инвентаризации и формировании сводной отчётности.

Таким образом, информационные объекты образуют сквозной поток данных: технологическая карта → меню → заявка на склад → учёт движения → отчётность. Нарушение целостности или актуальности любого из объектов ведёт к ошибкам во всех последующих. Выделенные объекты и их взаимосвязи являются основой для проектирования базы данных программного комплекса «MealSync».

2.1.3 Информационные потребности пользователей

Для эффективного выполнения должностных обязанностей каждый участник производственного процесса ТОО «СуперВкус» нуждается в определённом составе информации, её своевременности, достоверности и удобной форме представления. Выявленные информационные потребности являются основой для определения функциональных требований

к будущей системе. Ниже перечислены потребности каждой категории пользователей в соответствии с их ролью в технологическом процессе.

Технолог отвечает за нормативно-справочную базу и планирование производства. В своей деятельности он нуждается в следующей информации:

- полный и актуальный перечень технологических карт с возможностью поиска по наименованию, категории, составу;
- автоматический расчёт пищевой ценности (КБЖУ) и себестоимости блюда при изменении состава или цен на продукты;
- возможность копирования и адаптации технологических карт для разных филиалов с учётом их специализации;
- формирование меню на указанную дату с возможностью выбора филиала, приёма пищи и отбора блюд;
- расчёт потребности в продуктах на основе утверждённого меню для формирования заявки на склад;
- доступ к истории изменений технологических карт и меню.

Кладовщик осуществляет учёт движения продуктов и контроль складских запасов. Его информационные потребности включают:

- актуальные остатки продуктов по каждому филиалу в реальном времени;
- поступление заявок с производства с возможностью просмотра, фильтрации по статусу и дате;
- автоматическая сверка запрошенного количества продуктов с фактическими остатками;
- оформление прихода, расхода и списания продуктов с минимальным ручным вводом данных;
- проведение инвентаризации с автоматическим расчётом расхождений между учётными и фактическими остатками;
- формирование отчётов по движению продуктов за период.

Повар непосредственно готовит блюда и нуждается в оперативном доступе к нормативной информации:

- актуальная версия технологической карты на каждое блюдо с указанием состава, норм закладки, выхода порции и технологического описания;
- ежедневное меню филиала с перечнем блюд по приёмам пищи;
- быстрый поиск блюда по наименованию;
- просмотр фотографии готового блюда для контроля внешнего вида.

Руководитель осуществляет управление тремя филиалами и контроль их деятельности. Его информационные потребности носят аналитический характер:

- сводные данные по всем филиалам: остатки продуктов, расход сырья, себестоимость выпущенных блюд;
- статусы заявок на склад (создана, в работе, выполнена, частично выполнена);
- результаты инвентаризаций с выявленными расхождениями;
- сравнение показателей между филиалами для выявления эффективных и проблемных точек;
- формирование отчётов за произвольный период с возможностью экспорта.

Гость (посетитель) не участвует в производственных процессах, но имеет информационные потребности как конечный потребитель:

- актуальное меню филиала с указанием состава блюд, пищевой ценности, выхода порции, цены и фотографии;
- возможность выбора филиала для просмотра;
- наглядное и удобное представление информации без необходимости регистрации.

2.1.4 Методы работы с информационными объектами предметной области

2.1.4.1 Способы хранения информации об объектах предметной области

Для рассматриваемой предметной области, связанной с деятельностью предприятия общественного питания, хранение информации должно обеспечивать фиксацию, поиск, изменение и последующее использование данных в производственном, складском и управленческом процессах. Поскольку значительная часть объектов имеет документный характер, система хранения должна поддерживать как структурированные данные, так и документы, пригодные для просмотра, печати и архивирования.

Вся информация, используемая в деятельности ТОО «СуперВкус», может быть разделена на четыре группы:

– нормативно-справочная информация — сведения о продуктах, единицах измерения, категориях продуктов и блюд, производственных участках, а также о пользователях и их ролях;

– производственные документы — технологические карты и составы блюд (наименование, состав ингредиентов, нормы закладки, потери, выход, описание приготовления, КБЖУ, себестоимость, фото);

– плановые и учётные документы — меню филиалов (дата, филиал, перечень блюд, количество порций, статус публикации), заявки на склад (дата, состав продуктов, объём, статус обработки, автор), приходные и расходные документы, остатки, инвентаризационные ведомости;

– история изменений — версии документов, статусы заявок, результаты инвентаризаций.

Хранение перечисленных объектов должно удовлетворять следующим требованиям:

– централизованность — единое хранилище для всех филиалов исключает разрозненность данных;
– целостность — изменение одного объекта не должно нарушать согласованность связанных с ним данных;
– поиск и фильтрация — возможность быстрого нахождения информации по различным критериям (наименование, дата, филиал, статус);
– контроль доступа — разграничение прав на чтение, создание и изменение в зависимости от роли пользователя;
– сохранение истории — фиксация изменений документов для контроля и аудита.

С учётом перечисленных требований наиболее предпочтительным для данной предметной области является использование централизованной реляционной базы данных. Это обусловлено следующими причинами:

– предметная область содержит множество взаимосвязанных объектов (технологические карты, меню, заявки, складские документы);
– реляционная модель позволяет хранить объекты в единой структуре, уменьшает дублирование данных и поддерживает ссылочную целостность;
– база данных обеспечивает поиск, сортировку, фильтрацию и формирование отчётов по различным разрезам (филиалы, даты, типы операций).

Для документов и медиафайлов (фотографии блюд) дополнительно используется файловое хранилище, связанное с соответствующими записями в базе данных.

Для десктопной части системы предусматривается локальное хранилище, предназначенное для работы в офлайн-режиме. В нём временно сохраняются копии справочников, технологических карт, меню и части складских документов до момента синхронизации с центральной системой. Локальное хранилище не является основным, а выполняет вспомогательную функцию — обеспечивает непрерывность работы сотрудников при временном отсутствии связи.

Таким образом, для предметной области ТОО «СуперВкус» наиболее целесообразным является комбинированный способ хранения информации: основная структурированная информация хранится в централизованной реляционной базе данных, документы и медиафайлы — в файловом хранилище, локальная база используется как временный офлайн-слой.

2.1.4.2 Математические модели, используемые для обработки информации

В системе автоматизируются процессы, связанные с формированием технологических карт, расчетом пищевой ценности и себестоимости блюд, составлением меню, подготовкой заявок на склад, учетом движения продуктов и анализом результатов работы филиала. Для решения указанных задач применяются преимущественно детерминированные расчетные методы, основанные на нормативных показателях сырья, рецептурных зависимостях и учетных правилах складского движения. Это объясняется тем, что в рассматриваемой предметной области основная часть информации задается в виде формализованных норм: массы ингредиентов, процентов потерь, стоимости сырья, количества порций, объемов выдачи и фактических остатков. Следовательно, математическая модель системы должна описывать правила преобразования исходных данных в производственные, учетные и аналитические показатели.

Одной из базовых задач является определение массы нетто ингредиента на основе массы брутто и процента механических потерь. Такой расчет используется при формировании технологической карты и дальнейших расчетах состава блюда.

$$m_{ni} = m_{bi}\left(1 - \frac{p_{mi}}{100}\right) \quad (1)$$

где

m_{ni}– масса нетто i-го ингредиента, г;

m_{bi} – масса брутто i-го ингредиента, г;

p_{mi}– процент механических потерь i-го ингредиента, % [1], [2].

Абсолютная величина потерь массы при первичной обработке определяется как разность между массой брутто и нетто.

$$\Delta m_{mi} = m_{bi} - m_{ni} \quad (2)$$

где

Δm_{mi}– потери массы при механической обработке, г [2].

После определения массы нетто выполняется расчет нормативного выхода блюда с учетом тепловых потерь. Данный показатель необходим для контроля соответствия рецептуры производственным требованиям.

$$m_{out} = \sum_{i=1}^{k} m_{ni} \left(1 - \frac{p_{ti}}{100}\right) \quad (3)$$

где

m_{out}– нормативный выход готового блюда, г;

`p_{ti} – процент тепловых потерь i-го ингредиента, %;

`k` – количество ингредиентов в рецептуре [2], [3].

Следующим этапом является расчет пищевой ценности блюда. Для этого используются справочные показатели калорийности, белков, жиров и углеводов на 100 г продукта. Содержание белков, жиров и углеводов по каждому ингредиенту определяется пропорционально его массе нетто.

$$Prot_i = \frac{m_{ni}}{100} \cdot prot_i^{100} \quad (4)$$

$$Fat_i = \frac{m_{ni}}{100} \cdot fat_i^{100} \quad (5)$$

$$Carb_i = \frac{m_{ni}}{100} \cdot carb_i^{100} \quad (6)$$

где

$Prot_i$,Fat_i,$Carb_i$– количество соответственно белков, жиров и углеводов, приходящееся на i-й ингредиент, г;

$prot_i^{100}$, fat_i^{100},$carb_i^{100}$– содержание белков, жиров и углеводов в 100 г продукта [4].

Суммарное содержание макронутриентов в готовом блюде определяется суммированием по всем ингредиентам.

$$Prot = \sum_{i=1}^{k} Prot_i\,, \quad Fat = \sum_{i=1}^{k} Fat_i\,, \quad Carb = \sum_{i=1}^{k} Carb_i \quad (7)$$

где

$Prot$– общее количество белков в блюде, г;

Fat– общее количество жиров, г;

$Carb$ – общее количество углеводов, г.

Энергетическая ценность блюда рассчитывается по общеупотребимой формуле, основанной на энергетических коэффициентах для основных пищевых веществ.

$$En = 4 \cdot Prot + 9 \cdot Fat + 4 \cdot Carb \quad (8)$$

где

En– энергетическая ценность блюда, ккал [5].

Для автоматизации работы технолога важен также расчет себестоимости блюда. Стоимость каждого ингредиента определяется исходя из массы нетто и цены продукта за 1 кг.

$$Cost_i = \frac{m_{ni}}{1000} \cdot P_i \quad (9)$$

где

$Cost_i$– стоимость i-го ингредиента;

P_i– цена 1 кг i-го продукта, тенге/кг.

Полная себестоимость блюда рассчитывается как сумма стоимостей всех ингредиентов.

$$Cost_{dish} = \sum_{i=1}^{k} Cos\, t_i \quad (10)$$

где

$Cost_{dish}$– себестоимость блюда.

При формировании меню на день или неделю система должна учитывать количество порций каждого блюда. Поэтому суммарная стоимость меню определяется с учетом числа порций.

$$Cost_{menu} = \sum_{j=1}^{n} q_j \cdot Cost_{dish,j} \quad (11)$$

где $Cost_{menu}$– общая стоимость меню;

q_j– количество порций j-го блюда;

$Cost_{dish,j}$– себестоимость j-го блюда;

n– число блюд в меню.

На основе меню автоматически рассчитывается потребность в сырье для формирования заявки на склад. Требуемое количество i-го продукта определяется как сумма его норм расхода во всех блюдах меню с учетом числа порций.

$$Need_i = \sum_{j=1}^{n} q_j \cdot r_{ij} \tag{12}$$

где

$Need_i$– общая потребность в i-м продукте, г;

r_{ij}– норма расхода i-го продукта на одно j-е блюдо, г;

q_j– количество порций j-го блюда.

В складском контуре применяется балансовая модель движения запасов. Остаток продукта на конец периода определяется через начальный остаток, приход, расход и корректировки.

$$Q_{end} = Q_{beg} + Q_{in} - Q_{out} \pm Q_{adj} \tag{13}$$

где

Q_{beg}– начальный остаток;

Q_{in}– объем прихода;

Q_{out}– объем расхода;

Q_{adj}– корректировка по результатам инвентаризации;

Q_{end}– конечный остаток.

При проведении инвентаризации система должна сравнивать учетные и фактические данные. Абсолютное отклонение определяется следующим выражением.

$$\Delta Q = Q_{fact} - Q_{acc} \tag{14}$$

где

Q_{fact}– фактическое количество продукта;

Q_{acc}– учетное количество продукта;

ΔQ– отклонение между фактическим и учетным остатком.

Для оценки масштаба расхождения дополнительно может использоваться относительное отклонение.

$$\delta = \frac{\Delta Q}{Q_{acc}} \cdot 100\% \quad (15)$$

где

δ– процент отклонения фактического остатка от учетного.

При обработке заявок со склада для руководителя и кладовщика важен показатель исполнения заявки. Он показывает, какая доля запрошенного объема была фактически выдана.

$$K_{sup} = \frac{Q_{issued}}{Q_{req}} \cdot 100\% \quad (16)$$

где

K_{sup}– коэффициент обеспеченности заявки, %;

Q_{req}– запрошенное количество продукта;

Q_{issued}– фактически выданное количество продукта.

Для управленческой отчетности могут применяться и простые статистические методы, в частности расчет средних значений. Например, средняя калорийность блюд опубликованного меню определяется по следующей формуле.

$$\bar{En} = \frac{1}{n}\sum_{j=1}^{n} E\, n_j \quad (17)$$

где

$\bar{En}$– средняя энергетическая ценность одного блюда, ккал;

En_j– энергетическая ценность j-го блюда;

n– число блюд.

Представленная математическая модель охватывает несколько взаимосвязанных групп расчетов: расчет массы ингредиентов, расчет выхода блюда, расчет КБЖУ, расчет себестоимости, определение потребности в сырье, учет движения складских запасов и анализ отклонений. Использование этих соотношений позволяет формализовать основные процессы предприятия общественного питания и обеспечить автоматическое получение значимых для пользователя результатов. В результате система сокращает объем ручных

вычислений, повышает точность расчетов и обеспечивает согласованность данных на этапах производства, складского учета и управленческого анализа.

2.1.4.3 Способы интерпретации и визуального представления информации

В предметной области общественного питания данные используются различными категориями пользователей и должны быть представлены в форме, соответствующей их задачам. В таблице 2.1.4.3.1 приведены предпочтительные способы визуализации для каждой роли.

Таблица 2.1.4.3.1 – Предпочтительные способы визуализации по ролям пользователей

Роль	Предпочтительные способы представления
Технолог	Таблицы, карточки объектов, печатные формы
Кладовщик	Таблицы, печатные формы
Повар	Карточки объектов, экранные формы
Руководитель	Таблицы, графики, диаграммы, сводные отчёты
Гость (посетитель)	Публичное меню

Система должна поддерживать следующие основные способы представления информации: табличный, документный (печатные формы), карточный, графический и экранный. Рассмотрим подробнее каждый из способов:

1. Табличный способ. Таблицы являются наиболее распространённым способом для отображения перечней продуктов, состава технологических карт, меню, заявок, приходных и расходных документов, журналов остатков и результатов инвентаризации. Таблица позволяет представить большое количество однотипных реквизитов в компактном и сопоставимом виде. Пример состава блюда в табличной форме представлен на рисунке 2.1.4.3.1.

Продукт	*Брутто, г*	*Потери, %*	*Нетто, г*	*Стоимость, тг*	*КБЖУ*
...	...	...	...	...	...

Рисунок 2.1.4.3.1 – Пример табличного отображения данных

2. Документный способ (печатные формы). Печатные формы необходимы для технологических карт, меню, заявок, приходных и расходных документов, инвентаризационных ведомостей и отчётов. Причины использования печатных форм:

– часть информации по-прежнему используется в бумажном виде на производстве или складе;

– печатные документы удобны для согласования, утверждения, передачи между подразделениями и архивирования;
– унифицированная печатная форма повышает наглядность и завершённость учётного процесса.

3. Карточный способ. Карточная форма удобна для отображения отдельных сущностей со сложным составом реквизитов. В системе «MealSync» к таким объектам относятся:

– технологическая карта блюда;
– карточка заявки;
– карточка документа прихода или расхода;
– карточка инвентаризации.

Карточка позволяет пользователю сосредоточиться на одном объекте и отобразить его характеристики подробно и логически сгруппированно. Например, в карточке технологической карты объединяются: общие сведения о блюде, состав ингредиентов, технология приготовления, КБЖУ, себестоимость и фотография.

4. Графический способ (для руководителя). Графики и диаграммы применяются для обобщения учётных данных и выявления тенденций (таблица 2.1.4.3.2).

Таблица 2.1.4.3.2 – Типы графиков и диаграмм

Тип диаграммы	Назначение	Пример применения
Столбчатая	Сравнение величин	Объём заявок по филиалам, расход продуктов по категориям
Линейный график	Анализ изменения во времени	Динамика расхода, изменение остатков, колебания количества заявок
Круговая диаграмма	Отображение структуры	Распределение расходов по категориям продуктов, доля типов блюд в меню

В операционной деятельности графики играют вспомогательную роль. На уровне аналитики и управленческой отчётности они позволяют быстро оценить общую картину, сравнить показатели или выявить отклонения.

5. Экранные формы. Экранные формы являются основным средством взаимодействия пользователя с информацией в автоматизированной системе. Они обеспечивают ввод, редактирование, просмотр, поиск и фильтрацию данных. В зависимости от роли пользователя экранные формы адаптируются:

– для технолога — формы работы с технологическими картами, меню и заявками;
– для кладовщика — формы заявок, прихода, расхода, остатков и инвентаризации;
– для повара — экранные представления утверждённых технологических карт и меню;

– для руководителя — аналитические окна и отчётные формы;
– для гостя — веб-страницы публичного меню.

2.1.4.4 Технологии получения и передачи информации

В рамках автоматизируемого процесса получение и передача информации организуются по клиент-серверному принципу. Такой подход предполагает разделение системы на клиентскую и серверную части, каждая из которых выполняет собственные функции в общем цикле обработки данных. Клиентская часть обеспечивает взаимодействие пользователя с системой, ввод, просмотр и изменение информации, тогда как серверная часть отвечает за хранение данных, обработку запросов, выполнение расчетов и предоставление результатов пользователю.

Клиентская часть представлена десктоп-приложением и веб-интерфейсом. Десктоп-приложение используется для выполнения более насыщенных операционных действий, связанных с работой технолога и кладовщика, например для ведения технологических карт, учета складских операций и инвентаризации. Веб-интерфейс используется для просмотра опубликованного меню, работы сотрудников в браузере и получения аналитической информации. Независимо от формы клиентского приложения обмен данными осуществляется с серверной частью по единому принципу через сетевые запросы.

Серверная часть реализуется в виде backend API, которое выступает посредником между клиентом и хранилищем данных. При обращении пользователя клиент формирует запрос к серверу, передавая необходимые параметры, например идентификатор филиала, дату меню, сведения о заявке или данные авторизации. Сервер принимает запрос, выполняет проверку прав доступа, обрабатывает полученные данные, при необходимости обращается к базе данных, после чего формирует и возвращает ответ в структурированном виде. На стороне клиента этот ответ интерпретируется и отображается пользователю в форме таблицы, карточки, меню, документа или отчета.

Основной технологией передачи информации между клиентом и сервером является взаимодействие по API. Использование API позволяет обеспечить единый способ доступа к данным для разных клиентских приложений. Алгоритм такого взаимодействия включает несколько этапов: формирование клиентского запроса, передачу его на сервер, обработку сервером, обращение к базе данных или внутренним сервисам, формирование ответа и возврат результата клиенту. Например, при входе в систему клиент передает учетные данные пользователя на сервер, сервер выполняет проверку, после чего возвращает данные о роли пользователя и правах доступа. Аналогичным образом выполняются запросы на получение

меню, загрузку технологических карт, формирование заявок, обработку складских документов и получение аналитических сведений.

Для обеспечения целостности работы системы особенно важно, что все ключевые действия пользователей выполняются через серверную часть, а не напрямую через базу данных. Это позволяет централизованно контролировать корректность операций, разграничивать доступ, исключать несанкционированное изменение данных и поддерживать единые правила обработки информации для всех ролей. Кроме того, такой способ взаимодействия упрощает масштабирование системы и позволяет использовать одни и те же данные как в десктоп-приложении, так и в веб-интерфейсе.

2.1.5 Обзор существующих программных реализаций решения задачи

Для определения функциональных требований к разрабатываемому приложению был проведён анализ программных решений, частично выполняющих задачи, связанные с ведением рецептурной информации, расчётами производственных показателей и организацией технологических процессов. В качестве аналогов рассмотрены три широко распространённые системы: StoreHouse (R-Keeper), 1С Общепитит iChef.

1. StoreHouse представляет собой крупную и функционально насыщенную систему складского и калькуляционного учёта, входящую в программный комплекс R-Keeper. Приложение изначально разрабатывалось для ресторанов, кафе и сетевых предприятий питания, где требуется строгий контроль за движением сырья, точность калькуляций и формирование отчётности в соответствии с нормативами. На рисунке 2.1.5.1 показан интерфейс программы, отражающий её профессиональную направленность: множество справочников, детализированные таблицы, калькуляционные карты и средства формирования отчётов. StoreHouse позволяет одновременно вести технологические карты, контролировать остатки на складе, отслеживать списания и автоматически пересчитывать себестоимость блюд при изменении цен или сырьевых норм. Система интегрируется с POS-оборудованием, что делает её универсальным инструментом для предприятий среднего и крупного масштаба. Однако высокая сложность и обширность функционала создают существенные требования к подготовке персонала и сопровождению системы.

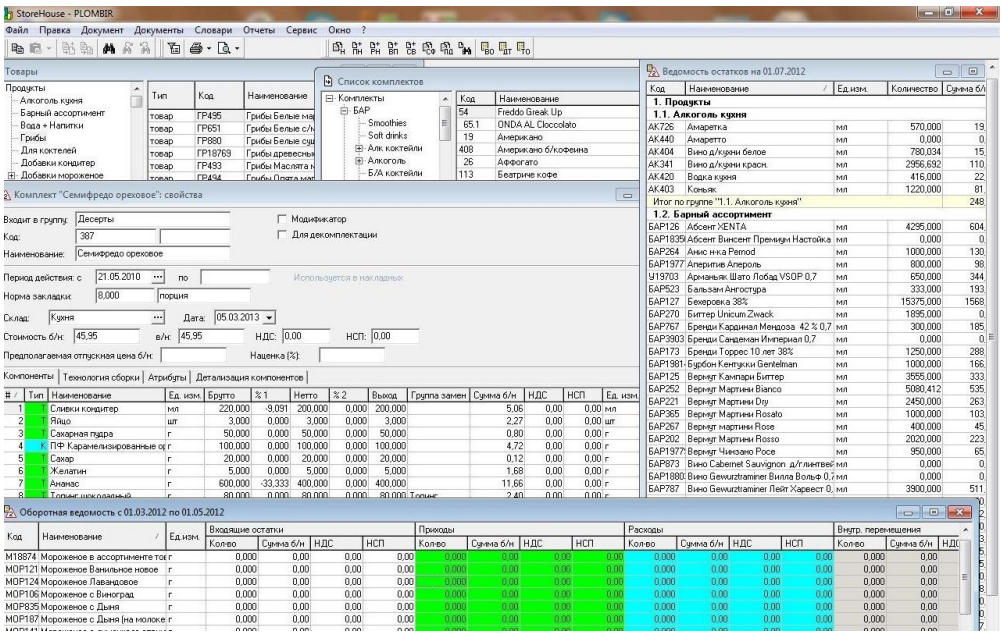


Рисунок 2.1.5.1 – Интерфейс системы StoreHouse

Достоинства:

- высокая точность расчётов;
- наличие автоматизированного складского учёта;
- поддержка технологических карт;
- развитые отчётные формы.

Недостатки:

- высокая стоимость внедрения;
- сложность освоения для неподготовленного персонала;
- избыточность функционала для небольших столовых;
- необходимость технического сопровождения.

2. 1С Общепит – это приложение для создания, хранения и структурирования рецептов, ориентированное преимущественно на бытовое использование. Изначально программа разрабатывалась как удобный инструмент для домашних пользователей и небольших кулинарных проектов, не связанных с профессиональным производством. Интерфейс системы, представленный на рисунке 2.1.5.2, отличается простотой и доступностью: рецепты хранятся в виде карточек, содержащих список ингредиентов, пошаговое описание приготовления и фотографии. Несмотря на удобство и лёгкость редактирования, 1С Общепит не поддерживает расчёт пищевой ценности, анализ потерь сырья, учёт себестоимости или формирование структурированной технологической карты. Приложение подходит для хранения и быстрого поиска рецептов, но не предназначено для

выполнения производственных расчётов, необходимых в столовых, ресторанах или других предприятиях общепита. Это делает его полезным бытовым инструментом, но недостаточным для профессиональных задач.

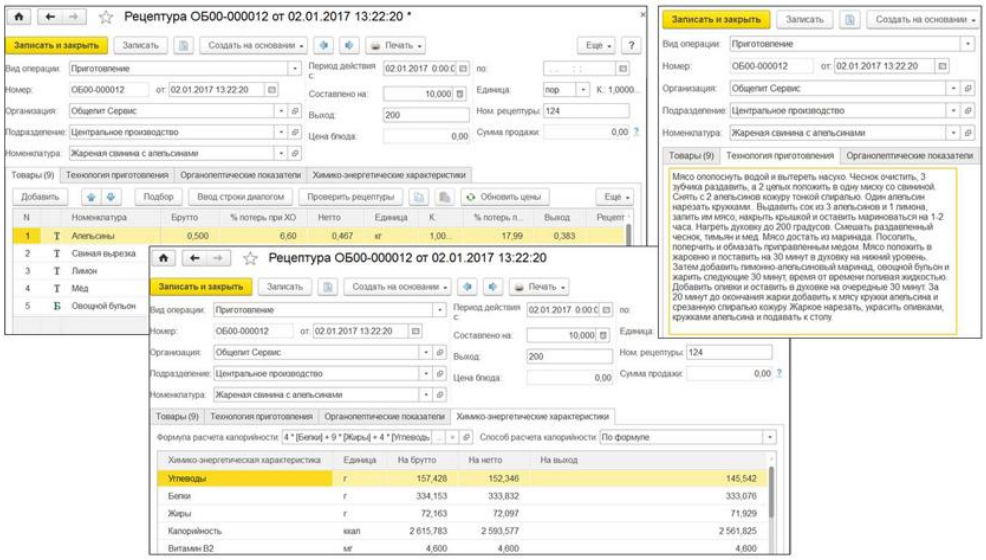


Рисунок 2.1.5.2 – Интерфейс приложения 1С Общепит

Достоинства:

- простой и интуитивный интерфейс;
- удобное хранение и редактирование рецептов;
- доступная печать рецептурных карточек.

Недостатки:

- отсутствие расчётов КБЖУ;
- отсутствие расчётов себестоимости;
- отсутствие структурированной технологической карты;
- отсутствие функционала, ориентированного на общепит.

3. iChef – это прикладная система автоматизации процессов кухни, разработанная для оперативного управления заказами в ресторанах и кафе. Главная цель программы – упрощение взаимодействия между залом и кухней, повышение скорости выполнения заказов и снижение ошибок при обслуживании гостей. Интерфейс, изображённый на рисунке 2.1.5.3, ориентирован на отображение активных заказов, статусов блюд и очередности их приготовления. Приложение позволяет персоналу быстро ориентироваться в текущей загрузке кухни, распределять задачи между поварами и контролировать выполнение заказов. Однако iChef не содержит инструментов для формирования технологических карт, расчёта себестоимости, ведения рецептурной базы или анализа производственных

параметров. Таким образом, система решает задачи оперативного управления, но не охватывает область технологической документации, что ограничивает её применимость в процессах, связанных с оформлением и стандартизацией блюд.

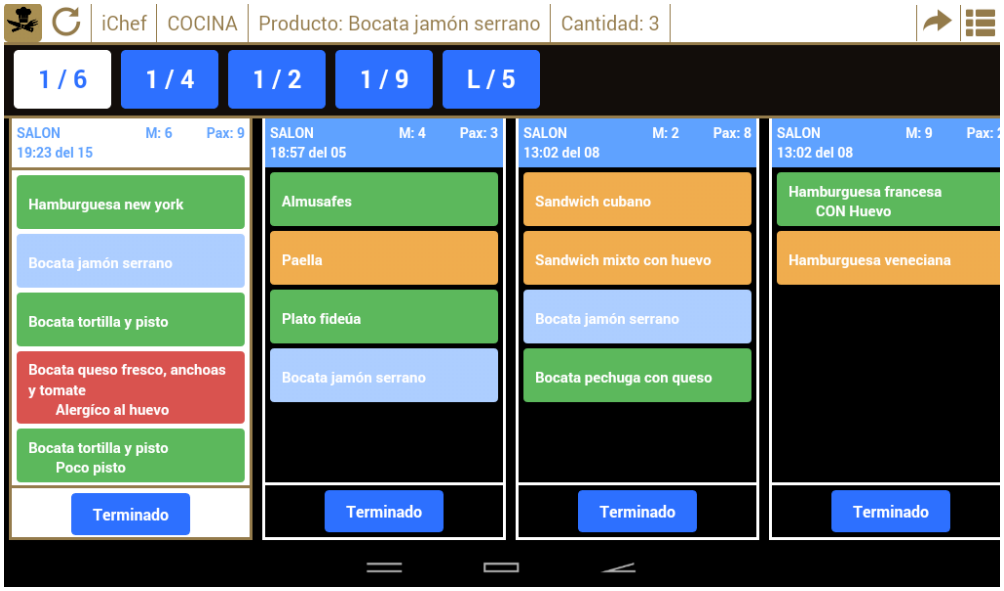


Рисунок 2.1.5.3 – Интерфейс системы iChef

Достоинства:

- высокая скорость работы с заказами;
- наглядное отображение процессов приготовления;
- удобство для оперативных задач кухни.

Недостатки:

- отсутствие расчётов себестоимости;
- отсутствие расчётов КБЖУ;
- отсутствие технологических карт;
- ограниченная применимость в производственных процессах столовой.

После рассмотрения программ-аналогов необходимо систематизировать их свойства для объективного сравнения.

Критерии сравнения выбраны на основе ключевых задач автоматизации предприятия общественного питания и включают: поддержку расчёта КБЖУ и себестоимости блюд, работу с технологическими картами и рецептами, складской учёт, ориентацию на профессиональный общепит, сложность интерфейса, стоимость использования, формирование печатных документов и гибкость настройки.

Для оценки используется трёхуровневая шкала: «Полная» — функция реализована полностью; «Частичная» — реализована с ограничениями; «Отсутствует» — не

реализована. Для критериев «Сложность интерфейса», «Стоимость использования» и «Гибкость настройки» применяется шкала «Высокая/Средняя/Низкая».

Сравнение программ по перечисленным параметрам приведено в таблице 2.1.5.1.

Таблица 2.1.5.1 – Сравнительная таблица программ-аналогов

Характеристика	StoreHouse	1С Общепит	iChef
Поддержка расчёта КБЖУ	Полная	Отсутствует	Отсутствует
Расчёт себестоимости блюд	Полная	Отсутствует	Отсутствует
Работа с технологическими картами	Полная	Отсутствует	Отсутствует
Работа с рецептами	Частичная	Полная	Частичная
Складской учёт	Полная	Отсутствует	Отсутствует
Ориентация на профессиональный общепит	Полная	Отсутствует	Частичная
Сложность интерфейса	Высокая	Низкая	Средняя
Стоимость использования	Высокая	Низкая	Средняя
Формирование печатных документов	Полная	Полная	Частичная
Гибкость настройки	Средняя	Низкая	Средняя

Проведённый анализ показал, что существующие решения либо ориентированы на крупные предприятия общественного питания (StoreHouse), либо предназначены для бытового хранения рецептов (1С Общепит), либо оптимизируют взаимодействие кухни и зала без полноценной работы с технологическими данными (iChef). Ни один из рассмотренных аналогов не обеспечивает комплексной автоматизации технологической документации, расчёта КБЖУ и себестоимости, складского учёта и ролевого доступа в единой системе, адаптированной для небольшого предприятия с несколькими филиалами.

Именно эти недостатки характерны и для текущей организации работы в ТОО «СуперВкус»: технологические карты, меню, заявки и складские документы не образуют единого информационного пространства. Данные дублируются, передаются в несогласованных формах и существуют в нескольких версиях, что приводит к ошибкам в расчётах, задержкам и усложнению контроля. Поскольку готовое программное решение, учитывающее все выявленные потребности, на рынке отсутствует, обоснованной является разработка собственного программного комплекса «MealSync», направленного на создание единой цифровой среды, объединяющей производственный, складской и аналитический контуры, с поддержкой ролевого доступа, автоматическими расчётами и возможностью локальной работы с последующей синхронизацией.

2.2 Классы и характеристики пользователей

Организация работы предприятия общественного питания предполагает участие нескольких категорий сотрудников, выполняющих различные функции в процессе подготовки, расчёта и оформления технологической документации. Эти сотрудники взаимодействуют с рецептурной информацией на разных этапах производственного цикла, начиная от подбора ингредиентов и определения технологических параметров и заканчивая проверкой расчётов и утверждением итоговых документов. Различия в уровнях квалификации, профессиональных обязанностях и степени вовлечённости в технологический процесс формируют ряд специфических требований к структуре и функциональности программных инструментов, используемых в работе. Учет особенностей каждой группы пользователей позволяет оптимизировать рабочие действия. Основные категории пользователей, участвующих в процессе оформления производственной документации, представлены в таблице 2.2.1.

Таблица 2.2.1 – Описание пользовательских ролей

Пользователь	Гость	Повар	Кладовщик	Технолог	Руководитель
Социальные характеристики	Посетители столовой, не являющиеся сотрудниками предприятия. Возраст и уровень подготовки не регламентируются.	Сотрудники производства, занятые приготовлением блюд. Обычно имеют профессиональную подготовку в сфере общественного питания.	Сотрудник, ответственные за складской учет и движение сырья. Имеют практический опыт работы с товарно-материальными ценностями.	Специалисты, отвечающие за технологические карты, меню и расчетные показатели. Как правило, имеют профильное среднее специальное или высшее образование.	Лица, выполняющие управленческие функции, контролирующие производственные и складские процессы филиала.
Мотивационно-целевая среда	Получение доступной информации о меню, составе блюд, КБЖУ и цене.	Обеспечение правильного и своевременного приготовления блюд в соответствии с утвержденными техкартами и меню.	Контроль обеспеченности производства сырьем, учет прихода, расхода и остатков продукции.	Формирование нормативной и расчетной базы производства, контроль актуальности рецептур и планирование меню.	Контроль деятельности филиала, анализ состояния запасов, заявок, расхода и результатов инвентаризации.
Навыки и умения	Базовые навыки работы с веб-интерфейсом.	Умение работать с технологическими картами, рецептами и производственной документацией.	Навыки ведения учета, сверки данных, оформления складских документов и работы с таблицами.	Уверенное владение ПК, навыки расчета КБЖУ и себестоимости, работа с нормативной и технологической документацией.	Навыки анализа данных, работы с отчетностью и использования цифровых средств контроля.
Требования к ПО	Простой и понятный интерфейс, быстрый	Удобный доступ к утвержденным	Быстрый доступ к заявкам, остаткам,	Поддержка работы с техкартами,	Получение сводной информации и

Пользователь	Гость	Повар	Кладовщик	Технолог	Руководитель
	доступ к меню по филиалу, наглядное отображение блюд и характеристик.	техкартам и опубликованному меню, четкое представление состава блюда, выхода и КБЖУ.	приходным и расходным операциям, понятные формы инвентаризации.	автоматизация расчетов, формирование меню и заявок, хранение нормативных данных.	отчетов в удобной для анализа форме, доступ к данным по филиалам.
Задачи пользователя	Просмотр публичного меню, выбор филиала, ознакомление с составом блюда, КБЖУ, ценой и меню на день или неделю.	Просмотр утвержденных техкарт, состава блюд, КБЖУ, выхода и опубликованного меню.	Обработка заявок, оформление прихода и расхода, контроль остатков, проведение инвентаризации.	Создание и редактирование техкарт, расчет КБЖУ и себестоимости, формирование меню, публикация меню, создание заявки на склад.	Просмотр аналитики, отчетов по заявкам, остаткам, расходу и инвентаризации, контроль работы филиала.
Рабочая среда	Веб-интерфейс, доступный с пользовательского устройства через браузер.	Веб-интерфейс и производственная среда кухни.	Десктоп-приложение и веб-интерфейс, рабочее место склада.	Десктоп-приложение и веб-интерфейс, рабочее место технолога.	Веб-интерфейс, рабочее место руководителя или удаленный доступ через браузер.

2.3 Функциональные требования

2.3.1 Определение функциональных возможностей

Определение функциональных возможностей разрабатываемой системы выполняется на основе выделения пользовательских ролей и анализа набора действий, которые каждая из них может выполнять в рамках автоматизируемого процесса. Для формализации таких действий применяются диаграммы вариантов использования, позволяющие отразить взаимодействие пользователя с программной системой, состав доступных ему функций, а также логические связи между отдельными сценариями работы. Использование данного подхода дает возможность установить границы ответственности каждой роли, определить состав предоставляемых системой сервисов и показать, каким образом программное решение поддерживает реальный производственный и учетный цикл предприятия общественного питания.

Функциональные возможности технолога представлены на диаграмме вариантов использования, приведенной на рисунке 2.3.1.1. Для данной роли предусмотрены функции авторизации, выбора филиала, просмотра технологических карт, создания и редактирования техкарт, утверждения техкарт, управления справочником продуктов, формирования меню и создания заявки на склад. Центральной расчетной функцией является расчет КБЖУ и себестоимости, который включается в сценарии создания и редактирования

технологической карты. Это означает, что при изменении рецептуры система должна автоматически обеспечивать пересчет основных показателей блюда. После подготовки технологической базы пользователь получает возможность сформировать меню филиала, а затем опубликовать его для дальнейшего использования другими ролями. Кроме того, с процессом формирования меню связан сценарий создания складской заявки, что отражает зависимость производственного планирования от обеспеченности сырьем. Таким образом, для технолога система охватывает как нормативный контур, связанный с рецептурами, так и плановый контур, связанный с меню и подготовкой заявок.

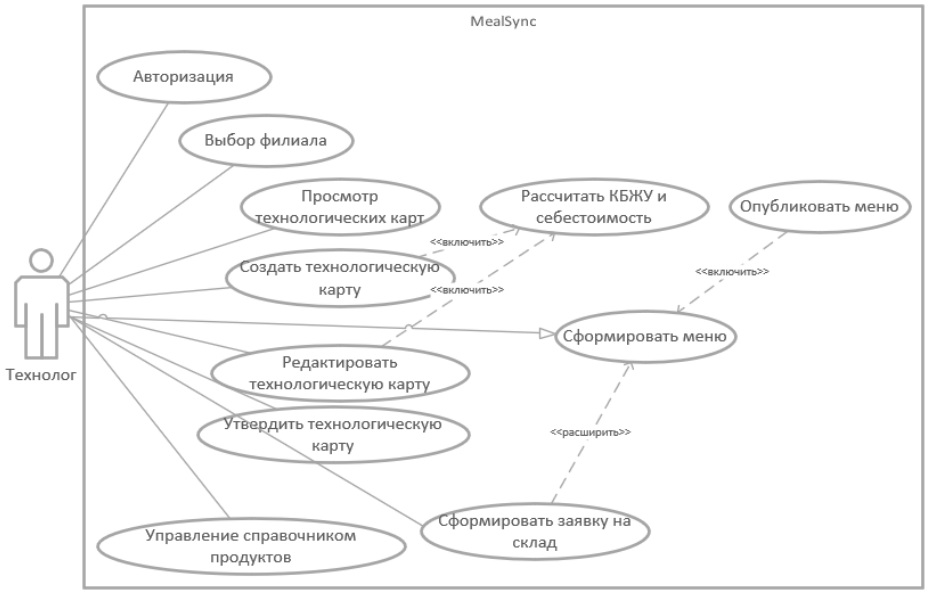


Рисунок 2.3.1.1 – Диаграмма вариантов технолога

Функциональные возможности кладовщика представлены на рисунке 2.3.1.2. Для данной роли система поддерживает авторизацию, просмотр заявок, обработку заявок, оформление прихода, просмотр остатков, проведение инвентаризации и просмотр складской отчетности. Основным сценарием является обработка заявки, от которого как расширения отходят действия по подтверждению заявки, частичной выдаче продукции, отклонению заявки и оформлению расхода. Такая структура отражает специфику складской деятельности: заявка может быть исполнена полностью, частично либо отклонена в зависимости от состояния запасов. При этом оформление расхода связано с фактической выдачей продукции и логически продолжает процесс обработки заявки. Помимо работы с заявками система предоставляет кладовщику средства оперативного учета, позволяющие фиксировать поступление сырья, контролировать остатки и проводить инвентаризацию. Следовательно, функциональные возможности данной роли ориентированы на обеспечение движения товарно-материальных ценностей и поддержание актуальности складских данных.

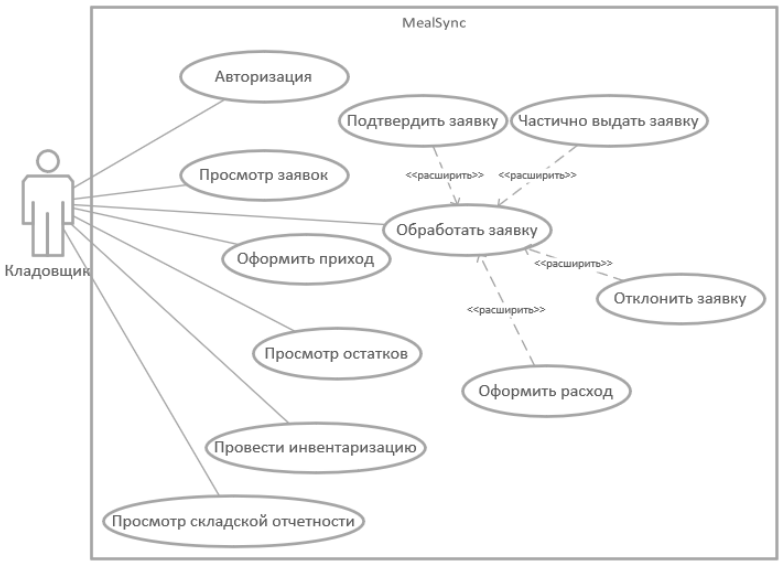


Рисунок 2.3.1.2 – Диаграмма вариантов кладовщика

Функциональные возможности повара отражены на рисунке 2.3.1.3. Для этой роли предусмотрены функции авторизации, просмотра утвержденных технологических карт и просмотра опубликованного меню. Сценарий просмотра утвержденных техкарт расширяется возможностями просмотра состава блюда, КБЖУ и выхода блюда, что позволяет использовать систему как источник нормативной производственной информации. Сценарий просмотра опубликованного меню, в свою очередь, расширяется функциями просмотра меню на сегодня и меню на неделю. Такое построение диаграммы подчеркивает, что повар не занимается изменением или расчетом данных, а использует уже подготовленную и утвержденную информацию в процессе приготовления продукции. Следовательно, функциональные возможности этой роли ограничены доступом к нормативным и плановым сведениям, необходимым для выполнения производственных операций.



Рисунок 2.3.1.3 – Диаграмма вариантов повара

Функциональные возможности руководителя представлены на рисунке 2.3.1.4. Для данной роли система предусматривает авторизацию, выбор филиала, просмотр аналитики и просмотр опубликованного меню. Базовый сценарий просмотра аналитики включает в себя просмотр отчетов по заявкам, остаткам, расходу и инвентаризации. Это означает, что аналитическая функция в системе рассматривается как интегрированный сценарий, объединяющий несколько направлений отчетности. За счет выбора филиала руководитель получает возможность анализировать показатели как по отдельному подразделению, так и в нужном организационном разрезе. Дополнительный доступ к опубликованному меню позволяет сопоставлять плановые данные с фактическими и управленческими показателями. Таким образом, функциональные возможности руководителя сосредоточены на контроле, анализе и принятии решений, а не на выполнении первичных операций учета или производства.



Рисунок 2.3.1.4 – Диаграмма вариантов руководителя

Функциональные возможности гостя отражены на рисунке 2.3.1.5. Для этой категории пользователей система предоставляет доступ к публичному меню без необходимости прохождения авторизации. Базовым сценарием является открытие публичного меню, которое связано с предварительным выбором филиала. Данный сценарий расширяется возможностями просмотра меню на сегодня, просмотра меню на неделю, просмотра состава блюда, КБЖУ, цены и печати меню. Такое содержание диаграммы показывает, что для гостя система ориентирована на информационное обслуживание и представление итоговых данных, сформированных внутренними пользователями. В отличие от сотрудников предприятия, посетитель не изменяет и не создает данные, а только получает доступ к опубликованной информации в наглядной и удобной форме.



Рисунок 2.3.1.5 – Диаграмма вариантов гостя

Таким образом, анализ диаграмм вариантов использования показывает, что функциональные возможности системы распределены по ролям в соответствии с их местом в автоматизируемом процессе. Общая диаграмма вариантов представлена в приложении Б. Технолог выполняет нормативно-расчетные и плановые функции, кладовщик – учетно-складские операции, повар – использует утвержденную производственную информацию, руководитель – получает аналитические и отчетные сведения, а гость – доступ к публичному меню. Такое разграничение обеспечивает согласованность работы пользователей, упорядочивает доступ к данным и позволяет реализовать в программной системе все ключевые функции, необходимые для поддержки деятельности предприятия общественного питания.

2.3.2 Описание прецедентов

Для детализации функциональных возможностей разрабатываемого программного средства MealSync используется описание прецедентов, отражающее типичные действия пользователей и реакцию системы на их выполнение. Прецеденты позволяют формализовать сценарии взаимодействия различных категорий пользователей с системой, определить последовательность операций и уточнить ожидаемый результат выполнения каждой функции.

Ниже представлены ключевые прецеденты системы:

- авторизация сотрудника;
- формирование и публикация меню;
- создание заявки на склад;

– обработка складской заявки;
– просмотр утвержденных технологических карт и меню;
– просмотр аналитики и отчетности;
– просмотр публичного меню.

В таблицах 2.3.2.1 - 2.3.2.7 представлено описание прецедентов.

Таблица 2.3.2.1 – Описание прецедента «Авторизация сотрудника»

Параметр	Значение
Наименование прецедента	Авторизация сотрудника
Цель	Получение доступа к функциям системы в соответствии с ролью пользователя
Исполнитель	Технолог, кладовщик, повар, руководитель
Предусловие	Пользователь имеет действующую учетную запись
Основной сценарий	1. Пользователь открывает форму входа. 2. Вводит логин и пароль. 3. Система проверяет учетные данные. 4. При успешной проверке определяет роль пользователя. 5. Пользователь получает доступ к функциям системы
Результат	Пользователь входит в систему и получает доступ к функциям, соответствующим его роли

Таблица 2.3.2.2 – Описание прецедента «Формирование и публикация меню»

Параметр	Значение
Наименование прецедента	Формирование и публикация меню
Цель	Создание меню филиала на день или неделю и его публикация для дальнейшего использования
Исполнитель	Технолог
Предусловие	Пользователь авторизован, выбран филиал, технологические карты и справочные данные доступны
Основной сценарий	1. Пользователь открывает раздел меню. 2. Выбирает дату и филиал. 3. Добавляет блюда из технологических карт. 4. Указывает тип приема пищи и количество порций. 5. Сохраняет меню. 6. Публикует меню для сотрудников и гостей
Результат	Меню филиала сформировано, сохранено и опубликовано

Таблица 2.3.2.3 – Описание прецедента «Создание заявки на склад»

Параметр	Значение
Наименование прецедента	Создание заявки на склад
Цель	Формирование заявки на выдачу продуктов для обеспечения меню филиала
Исполнитель	Технолог
Предусловие	Пользователь авторизован, меню сформировано, выбран филиал
Основной сценарий	1. Пользователь открывает раздел заявок. 2. Выбирает продукты или формирует заявку на основе меню. 3. Указывает требуемое количество сырья. 4. Сохраняет заявку. 5. Система фиксирует заявку и передает ее в складской контур
Результат	Заявка на склад создана и доступна кладовщику для обработки

Таблица 2.3.2.4 – Описание прецедента «Обработка складской заявки»

Параметр	Значение
Наименование прецедента	Обработка складской заявки

Параметр	Значение
Цель	Проверка и исполнение заявки на выдачу продуктов
Исполнитель	Кладовщик
Предусловие	Пользователь авторизован, в системе имеется поступившая заявка
Основной сценарий	1. Пользователь открывает список заявок. 2. Выбирает нужную заявку. 3. Проверяет наличие продуктов на складе. 4. Подтверждает заявку, частично выдает продукцию или отклоняет заявку. 5. При необходимости оформляет расход. 6. Система обновляет статус заявки и складские данные
Результат	Заявка обработана, ее статус обновлен, движение продуктов отражено в системе

Таблица 2.3.2.5 – Описание прецедента «Просмотр утвержденных технологических карт и меню»

Параметр	Значение
Наименование прецедента	Просмотр утвержденных технологических карт и меню
Цель	Получение производственной информации для приготовления блюд
Исполнитель	Повар
Предусловие	Пользователь авторизован, меню опубликовано, технологические карты утверждены
Основной сценарий	1. Пользователь открывает раздел утвержденных техкарт. 2. Выбирает нужное блюдо. 3. Просматривает состав, КБЖУ и выход блюда. 4. Открывает опубликованное меню на сегодня или неделю
Результат	Пользователь получает доступ к актуальным техкартам и меню для работы в производстве

Таблица 2.3.2.6 – Описание прецедента «Просмотр аналитики и отчетности»

Параметр	Значение
Наименование прецедента	Просмотр аналитики и отчетности
Цель	Получение сводной информации для контроля и принятия управленческих решений
Исполнитель	Руководитель
Предусловие	Пользователь авторизован и выбрал филиал
Основной сценарий	1. Пользователь открывает раздел аналитики. 2. Выбирает филиал. 3. Просматривает отчеты по заявкам, остаткам, расходу и инвентаризации. 4. При необходимости сопоставляет отчетные данные с опубликованным меню
Результат	Пользователь получает аналитическую и отчетную информацию по филиалу

Таблица 2.3.2.7 – Описание прецедента «Просмотр публичного меню»

Параметр	Значение
Наименование прецедента	Просмотр публичного меню
Цель	Получение информации о доступных блюдах без авторизации
Исполнитель	Гость
Предусловие	Система доступна через веб-интерфейс, меню опубликовано
Основной сценарий	1. Пользователь открывает публичную страницу. 2. Выбирает филиал. 3. Просматривает меню на сегодня или неделю. 4. При необходимости открывает состав блюда, КБЖУ и цену. 5. Может распечатать меню
Результат	Пользователь получает доступ к опубликованному меню и характеристикам блюд

2.4 Нефункциональные требования

Нефункциональные требования определяют условия функционирования разрабатываемого программного средства и качественные характеристики его работы: производительность, безопасность, надёжность, удобство использования, совместимость и сопровождение.

Производительность:

– Авторизация, открытие списков меню, заявок, технологических карт и остатков должна выполняться за время не более 2 секунд при стандартной нагрузке (до 10 одновременных пользователей).

– Сохранение документов, публикация меню, обработка заявок и формирование отчётных данных должны выполняться за время не более 3 секунд.

Надёжность и устойчивость:

– Система должна обеспечивать устойчивую работу при длительном использовании (непрерывно в течение рабочего дня). Сбой в работе одного клиентского приложения не должен приводить к нарушению функционирования серверной части и других клиентов.

– При сохранении меню, заявок, складских документов и результатов инвентаризации данные должны фиксироваться полностью, без частичной потери.

– При временной недоступности сервера или сбоях связи десктопный модуль должен поддерживать локальную работу с последующей синхронизацией данных. Потеря уже сохранённой информации не допускается.

– Для центральной базы данных должно быть предусмотрено резервное копирование, обеспечивающее восстановление информации при технических сбоях.

Безопасность и разграничение доступа:

– Доступ к внутренним функциям системы должен предоставляться только после успешной авторизации. Гость имеет доступ только к публичной части (просмотр меню) и не получает доступа к технологическим, складским и аналитическим данным.

– Технолог, кладовщик, повар и руководитель должны видеть только те разделы и данные, которые соответствуют их должностным обязанностям (ролевая модель доступа).

– Учётные данные пользователей должны храниться в защищённом виде (хеширование). Передача информации между клиентской и серверной частью должна выполняться по защищённому каналу (HTTPS).

Целостность и согласованность данных:

– Система должна обеспечивать целостность и согласованность взаимосвязанных данных. Изменения в технологических картах, меню, заявках и складских документах не должны приводить к появлению противоречивых записей.

– Данные по филиалам должны храниться раздельно. Действия пользователя должны выполняться в контексте выбранного филиала.

Удобство использования (Usability):

– Интерфейс должен быть понятным для сотрудников предприятия и не требовать длительного обучения. Формы ввода, таблицы, карточки объектов и документы должны быть логично организованы и соответствовать реальным рабочим сценариям.

– Для пользователей должны быть доступны средства поиска, фильтрации и сортировки данных.

– Система должна поддерживать формирование и печать документов (технологические карты, меню, заявки, складские документы).

3 ВЫБОР ПРОГРАММНЫХ СРЕД И СРЕДСТВ РАЗРАБОТКИ

Разработка программного средства для автоматизации работы предприятия общественного питания требует выбора технологий, позволяющих реализовать все ключевые функции системы: ведение технологических карт, формирование меню, обработку складских заявок, учёт остатков, проведение инвентаризации, публикацию меню для гостей и получение аналитической отчётности. Решение имеет смешанную архитектуру и включает десктоп-часть, веб-интерфейс, серверную логику и централизованное хранилище данных.

3.1 Выбор языка программирования

Для реализации всех компонентов системы необходим язык, который обеспечивает высокую скорость разработки, наличие библиотек для работы с базами данных, веб-технологиями и графическими интерфейсами, а также удобство реализации расчётных алгоритмов. В таблице 3.1.1 приведено сравнение рассматриваемых языков.

Таблица 3.1.1 – Сравнение языков программирования

Критерий	Python	Java	C#
Скорость разработки	Высокая	Средняя	Средняя
Наличие библиотек для GUI	Да (PySide6, Tkinter)	Да (Swing, JavaFX)	Да (WPF, Windows Forms)
Поддержка веб-разработки	Да (FastAPI, Django)	Да (Spring)	Да (ASP.NET)
Удобство расчётных алгоритмов	Высокое	Среднее	Среднее
Порог вхождения	Низкий	Высокий	Средний

На основе сравнения в качестве базового языка разработки выбран Python. Его преимуществами являются высокая скорость разработки, большое количество библиотек и удобство реализации расчётных алгоритмов (КБЖУ, себестоимость). Недостаток Python — более низкая производительность по сравнению с компилируемыми языками — не является критическим, поскольку основная нагрузка связана с обработкой бизнес-данных и пользовательских запросов, а не с высокопроизводительными вычислениями.

3.2 Выбор инструментария для десктоп-интерфейса

Для разработки десктоп-части системы необходимо средство, позволяющее создавать современный, удобный и функционально насыщенный интерфейс с поддержкой таблиц, форм, диалогов и печатных представлений. В таблице 3.2.1 приведено сравнение рассматриваемых вариантов.

Таблица 3.2.1 – Сравнение инструментов для десктоп-интерфейса

Критерий	Tkinter	Веб-оболочка (Electron)	PySide6
Простота освоения	Высокая	Средняя	Средняя
Набор компонентов	Ограниченный	Богатый (HTML/CSS)	Развитый
Поддержка сложных форм	Низкая	Высокая	Высокая
Интеграция с Python	Встроенная	Через мосты	Отличная
Ресурсные требования	Низкие	Высокие	Средние

Наиболее рациональным выбором является PySide6. Данная библиотека предоставляет развитый набор компонентов интерфейса, поддерживает создание сложных оконных форм, таблиц, диалогов и печатных представлений, а также хорошо интегрируется с Python. Недостатком является более высокий порог вхождения по сравнению с Tkinter, однако это компенсируется функциональными возможностями.

3.3 Выбор фреймворка для серверной части

Серверная часть должна реализовывать API для взаимодействия с десктоп- и веб-клиентами, обрабатывать бизнес-логику и обеспечивать асинхронную обработку запросов. В таблице 3.3.1 приведено сравнение рассматриваемых фреймворков.

Таблица 3.3.1 – Сравнение фреймворков для серверной части

Критерий	Django	Flask	FastAPI
Полнота функционала	Высокая (избыточная)	Низкая (требует ручной настройки)	Средняя
Поддержка асинхронности	Частичная	Нет	Да
Автоматическая документация API	Нет	Нет	Да
Удобство работы со схемами данных	Среднее	Низкое	Высокое
Скорость разработки	Высокая	Средняя	Высокая

Наиболее подходящим выбором является FastAPI. Фреймворк хорошо подходит для построения API-ориентированных приложений, поддерживает асинхронную обработку запросов, автоматически формирует документацию и обеспечивает удобную работу со схемами данных. Django избыточен для компактного backend API, Flask требует более ручной настройки многих компонентов.

3.4 Выбор базы данных

Для централизованного хранения данных необходима реляционная СУБД, обеспечивающая надёжность, целостность данных и поддержку многопользовательского доступа. Кроме того, для локальной работы в офлайн-режиме требуется легковесное встроенное хранилище. В таблице 3.4.1 приведено сравнение рассматриваемых СУБД.

Таблица 3.4.1 – Сравнение реляционных баз данных

Критерий	SQLite	MySQL	PostgreSQL
Простота развёртывания	Высокая (без сервера)	Средняя	Средняя
Поддержка многопользовательского доступа	Низкая	Высокая	Высокая
Поддержка сложных запросов	Низкая	Средняя	Высокая
Обеспечение целостности данных	Ограниченное	Среднее	Высокое
Поддержка транзакций	Да	Да	Да

В качестве центральной базы данных выбран PostgreSQL. Его преимуществами являются надёжность, поддержка сложных запросов, обеспечение целостности данных и хорошая совместимость с Python-инструментами. Для локального офлайн-режима десктоп-приложения используется SQLite как встроенное легковесное хранилище, применяемое для временной работы и последующей синхронизации с сервером.

3.5 Выбор инструментов для доступа к данным и миграций

Для взаимодействия с базой данных и управления изменениями её структуры необходимы инструменты, обеспечивающие удобную объектно-реляционную модель и отслеживание миграций схемы. В таблице 3.5.1 приведено описание выбранных инструментов.

Таблица 3.5.1 – Инструменты для доступа к данным и миграций

Инструмент	Назначение	Преимущества
SQLAlchemy	ORM (объектно-реляционное отображение)	Упрощает работу с таблицами, повышает поддерживаемость кода
Alembic	Управление миграциями схемы БД	Отслеживает и применяет изменения структуры базы данных

Выбранные инструменты хорошо интегрируются с Python и PostgreSQL, обеспечивая надёжную работу с данными и удобство сопровождения проекта при изменениях структуры хранения.

3.6 Выбор технологии для веб-интерфейса

Веб-интерфейс системы включает публичную страницу меню для гостей и кабинет сотрудников. Необходимо выбрать подход, обеспечивающий достаточную функциональность без избыточной сложности. В таблице 3.6.1 приведено сравнение рассматриваемых подходов.

Таблица 3.6.1 – Сравнение подходов к разработке веб-интерфейса

Критерий	Frontend-фреймворк (React/Vue)	Стандартные HTML, CSS, JS
Сложность реализации	Высокая	Низкая
Гибкость интерфейса	Высокая	Средняя
Требования к инфраструктуре	Высокие (Node.js, сборка)	Низкие (статические файлы)

Критерий	Frontend-фреймворк (React/Vue)	Стандартные HTML, CSS, JS
Скорость разработки	Средняя	Высокая
Избыточность для данной задачи	Высокая	Низкая

Для рассматриваемого проекта выбран второй подход — HTML, CSS, JavaScript. Интерфейс ориентирован не на сложное одностороннее приложение, а на понятные страницы меню, кабинета сотрудников и аналитики.

3.7 Выбор вспомогательных инструментов

Для сопровождения разработки необходимы среда разработки, система контроля версий и инструмент для контейнеризации. Выбранные средства приведены в таблице 3.7.1.

Таблица 3.7.1 – Вспомогательные инструменты разработки

Инструмент	Назначение	Обоснование выбора
Visual Studio Code	Среда разработки	Поддержка Python, веб-технологий и Git, удобная навигация и отладка
Git	Система контроля версий	Отслеживание истории изменений, надёжность сопровождения
Docker	Контейнеризация	Упрощение развёртывания и тестирования серверной части, переносимость между средами

3.8 Обобщение результатов выбора

На основе проведённого анализа в таблице 3.8.1 представлен итоговый набор программных средств, выбранных для реализации проекта.

Таблица 3.8.1 – Итоговый набор программных средств

Компонент	Выбранное средство	Ключевое обоснование
Язык программирования	Python	Высокая скорость разработки, удобство расчётов
Десктоп-интерфейс	PySide6	Развитый набор компонентов, интеграция с Python
Серверная часть	FastAPI	Асинхронность, автоматическая документация API
ORM	SQLAlchemy	Удобство работы с таблицами, поддерживаемость кода
Миграции	Alembic	Отслеживание изменений схемы БД
Центральная БД	PostgreSQL	Надёжность, целостность данных, сложные запросы
Локальная БД (offline)	SQLite	Встроенное легковесное хранилище
Веб-интерфейс	HTML, CSS, JavaScript	Простота реализации, отсутствие избыточности
Среда разработки	Visual Studio Code	Поддержка необходимых технологий
Система контроля версий	Git	История изменений, надёжность
Контейнеризация	Docker	Переносимость, упрощение тестирования

Выбранный инструментарий соответствует архитектуре разрабатываемой системы, обеспечивает баланс между гибкостью, функциональностью и удобством разработки. Python как базовый язык позволяет быстро реализовать расчётные алгоритмы и интеграцию с базами данных. PySide6 даёт возможность создать современный десктоп-интерфейс,

адаптированный для работы технолога, кладовщика и руководителя. FastAPI обеспечивает высокую производительность серверной части и удобство разработки API. PostgreSQL гарантирует надёжное централизованное хранение данных, а SQLite — автономную работу при сбоях связи. Веб-интерфейс на HTML, CSS и JavaScript реализуется без избыточной сложности. Таким образом, предложенный набор средств является наиболее рациональным для решения поставленной задачи.

4 АЛГОРИТМЫ РЕШЕНИЯ ПОСТАВЛЕННОЙ ЗАДАЧИ

4.1 Этапы реализации

Реализация программного средства выполнялась последовательно, что позволило последовательно перейти от проектных решений к созданию программных модулей и их объединению в единую систему.

Основные этапы реализации можно представить в виде следующей последовательности:

– На первом этапе была определена общая структура программного средства, включающая десктоп-часть, веб-интерфейс, серверную часть и централизованное хранилище данных. На этом же этапе была выбрана клиент-серверная архитектура и установлен принцип разделения ролей пользователей.

– На втором этапе была разработана структура хранения данных. Были определены основные сущности, необходимые для работы со справочниками, технологическими картами, меню, складскими заявками, приходными и расходными документами, остатками и инвентаризацией. После этого была создана схема базы данных и подготовлены модели хранения информации.

– На третьем этапе была реализована серверная часть программного средства. Были созданы модули для авторизации пользователей, работы со справочниками, продуктами, технологическими картами, меню, складскими заявками, остатками, инвентаризацией, публичным меню и аналитическими данными. На данном этапе была обеспечена единая логика обработки запросов и взаимодействия с хранилищем данных.

– На четвертом этапе была реализована десктоп-часть системы. Были разработаны пользовательские формы и окна, обеспечивающие работу технолога и кладовщика. В рамках этого этапа были созданы интерфейсы для управления технологическими картами, меню, заявками, приходными и расходными документами, остатками и инвентаризацией.

– На пятом этапе была реализована веб-часть программного средства. Были разработаны страницы публичного меню, кабинет сотрудника и интерфейсы для ролей повара, кладовщика, технолога и руководителя. Также были подготовлены средства отображения меню, заявок и отчетных данных через браузер.

– На шестом этапе был реализован механизм клиент-серверного взаимодействия. Были настроены запросы между десктоп и веб-клиентами и серверной частью, обеспечена передача данных через API и организована единая логика обмена информацией между компонентами системы.

– На седьмом этапе была реализована работа с локальным хранилищем данных в -контуре. Были подготовлены средства локальной записи данных и механизмы их использования в офлайн-режиме, что позволило поддерживать непрерывность работы при временном отсутствии соединения с сервером.

– На восьмом этапе был реализован механизм синхронизации локальных и серверных данных. В рамках этого этапа была обеспечена передача изменений между локальным и центральным хранилищем, что позволило объединить офлайн- и онлайн-режимы работы в рамках одной системы.

– На девятом этапе была выполнена интеграция всех реализованных модулей в единое программное средство. Были согласованы взаимодействие интерфейсов, серверных компонентов, моделей данных и средств синхронизации, что обеспечило целостность работы всей системы.

Таким образом, реализация программного средства выполнялась как последовательное создание и объединение его основных компонентов: структуры хранения данных, серверной части, десктоп-приложения, веб-интерфейса, механизмов обмена данными и синхронизации. Такой подход позволил сформировать целостное программное решение, соответствующее архитектуре и задачам проекта.

4.2 Пользовательский интерфейс

4.2.1 Описание взаимодействия всех пользователей с системой

В рамках проектирования пользовательского интерфейса разрабатывается логика взаимодействия пользователей с программным средством. Для этого определяются основные пользовательские сценарии, последовательность выполнения действий и реакция системы на запросы пользователей.

Общая схема взаимодействия пользователей с программным средством представлена на рисунке в приложении В. Диаграмма активности отражает укрупненную логику работы системы и показывает, каким образом дальнейшее развитие сценария зависит от типа пользователя и его роли.

Начало процесса располагается в дорожке системы, где выполняется определение типа пользователя. На этом этапе различаются два основных варианта работы: использование публичной части без авторизации и работа сотрудника во внутреннем контуре. Если пользователь является гостем, взаимодействие сразу переводится в соответствующую ролевую область. Если пользователь относится к сотрудникам предприятия, система

инициирует процедуру авторизации и после проверки учетных данных определяет роль пользователя.

В дорожке гостя отражен наиболее простой вариант взаимодействия. Пользователь открывает публичное меню, выбирает филиал, просматривает доступные блюда, после чего может ознакомиться с составом, КБЖУ и ценой. Такой сценарий показывает, что внешний пользователь не участвует во внутренних производственных и учетных процессах, а использует только опубликованную информацию, предназначенную для посетителей.

В дорожке технолога показан сценарий, связанный с нормативно-плановой деятельностью. После выполнения входа и выбора филиала пользователь переходит к работе с технологическими картами. Далее выполняется формирование и публикация меню, после чего создается заявка на склад. Такое представление подчеркивает, что деятельность технолога связана с последовательным преобразованием нормативной информации в плановые документы, используемые другими участниками процесса.

В дорожке кладовщика отражен учетно-складской сценарий. После входа в систему пользователь открывает и обрабатывает заявки, после чего принимает решение по заявке: подтвердить ее, частично выдать продукцию или отклонить. Затем выполняются операции прихода и расхода, просмотр остатков и проведение инвентаризации. Эта часть диаграммы показывает, что складской контур является наиболее разветвленным, так как связан с обработкой различных состояний заявки и последующим изменением учетных данных.

В дорожке повара зафиксирован производственный сценарий. После входа в систему пользователь просматривает утвержденные технологические карты, затем получает сведения о составе, КБЖУ и выходе блюда, после чего обращается к опубликованному меню. В данном случае система выступает источником производственной информации, используемой в процессе приготовления блюд. Из диаграммы видно, что действия повара не связаны с редактированием данных, а ограничиваются использованием утвержденных и опубликованных сведений.

В дорожке руководителя отражен сценарий управленческого взаимодействия. После входа в систему пользователь выбирает филиал, просматривает аналитику и отчетность, а затем при необходимости обращается к опубликованному меню. Такой порядок действий показывает, что руководитель использует систему как средство контроля и анализа, а не как инструмент ввода или изменения первичных данных.

Таким образом, диаграмма активности, представленная, позволяет в обобщенном виде отразить структуру взаимодействия различных категорий пользователей с системой, показать роль серверной части в маршрутизации сценариев и продемонстрировать различие между контурами работы.

4.2.1.1 Описание взаимодействия технолога с системой

Алгоритм взаимодействия технолога с программным средством представлен на рисунке 4.2.1.1. В рамках данного сценария пользователь выполняет вход в систему через веб-интерфейс, после чего получает доступ к функциям, соответствующим его роли. Для работы с технологическими картами используется десктоп-контур, через который выполняется запрос на получение перечня технологических карт и сохранение изменений в них. Далее через веб-интерфейс технолог работает с меню: получает данные о доступных днях меню, сохраняет сформированное меню и публикует его. После этого пользователь открывает раздел заявок и создает заявку на склад. Таким образом, диаграмма отражает совмещенное использование десктоп- и веб-компонентов при выполнении технологом нормативно-расчетных и плановых функций.

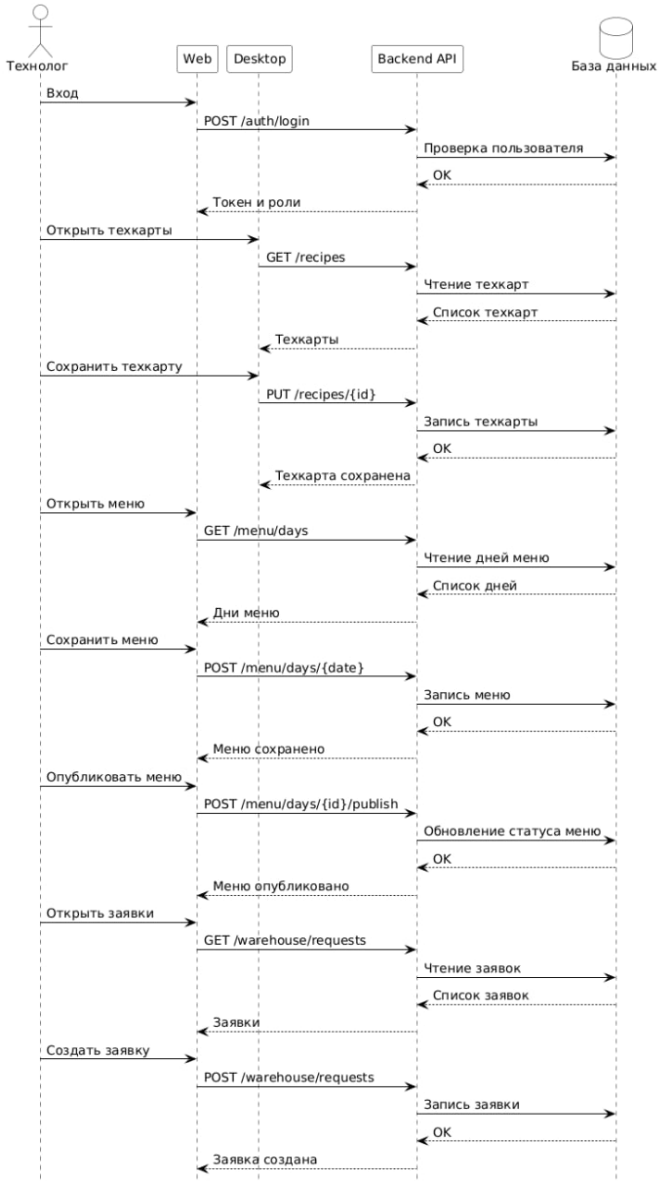


Рисунок 4.2.1.1 – Диаграмма последовательности взаимодействия технолога с программным средством

4.2.1.2 Описание взаимодействия кладовщика с системой

Алгоритм взаимодействия кладовщика с программным средством представлен на рисунке 4.2.1.2. Пользователь начинает работу с авторизации через веб-интерфейс, после чего получает доступ к заявкам. Затем кладовщик открывает список заявок, выбирает нужную заявку и получает ее детальное описание. После этого выполняются действия по подтверждению заявки или частичной выдаче продукции. Операции прихода, расхода, просмотра остатков и проведения инвентаризации выполняются через десктоп-контур, который обеспечивает поддержку складских операций. Серверная часть обрабатывает соответствующие запросы, вносит изменения в базу данных и возвращает пользователю обновленные результаты. Диаграмма показывает, что для данной роли веб-часть используется для обработки заявок, а десктоп-часть — для выполнения основных операций складского учета.

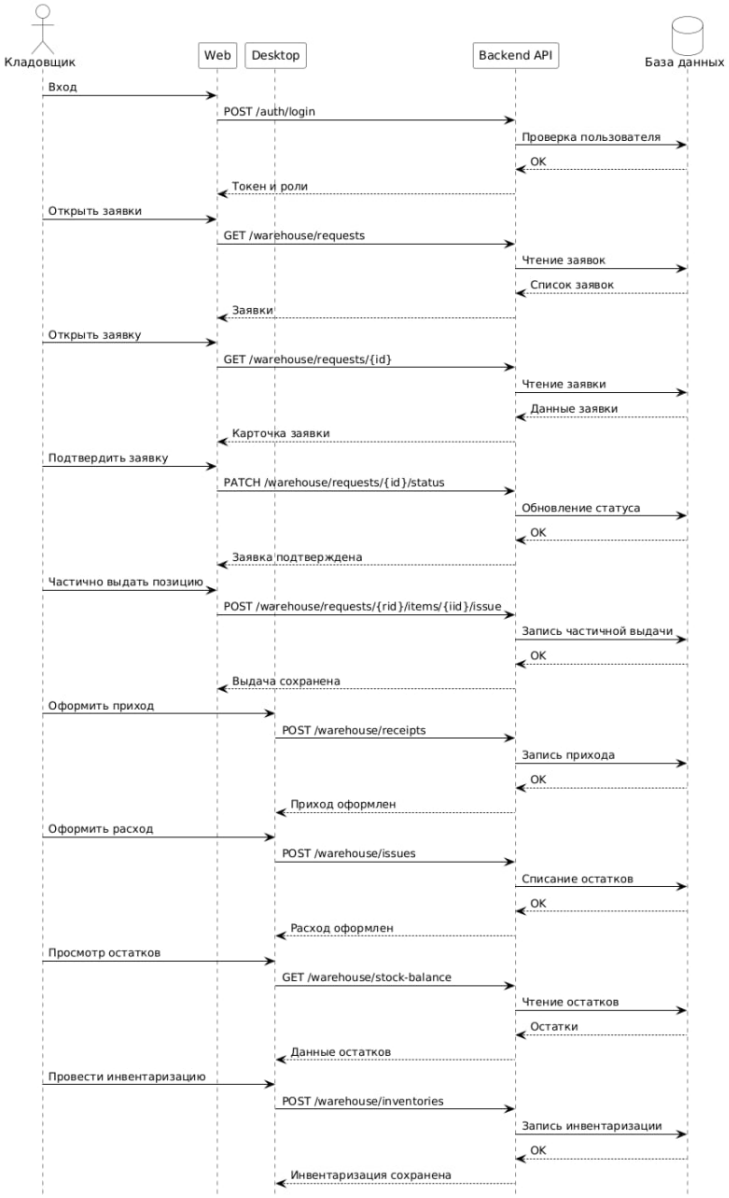


Рисунок 4.2.1.2 – Диаграмма последовательности взаимодействия кладовщика с программным средством

4.2.1.3 Описание взаимодействия повара с системой

Алгоритм взаимодействия повара с программным средством представлен на рисунке 4.2.1.3. После входа в систему пользователь через веб-интерфейс получает доступ к утвержденным технологическим картам. При выборе конкретной техкарты система запрашивает сведения о составе блюда, КБЖУ и выходе, после чего возвращает их пользователю. Далее повар может открыть опубликованное меню на текущий день или запросить меню на неделю. Серверная часть получает соответствующие запросы, обращается к базе данных и возвращает готовые данные в веб-интерфейс. Данный сценарий отражает производственный характер работы повара, для которого система выступает источником нормативной и плановой информации.

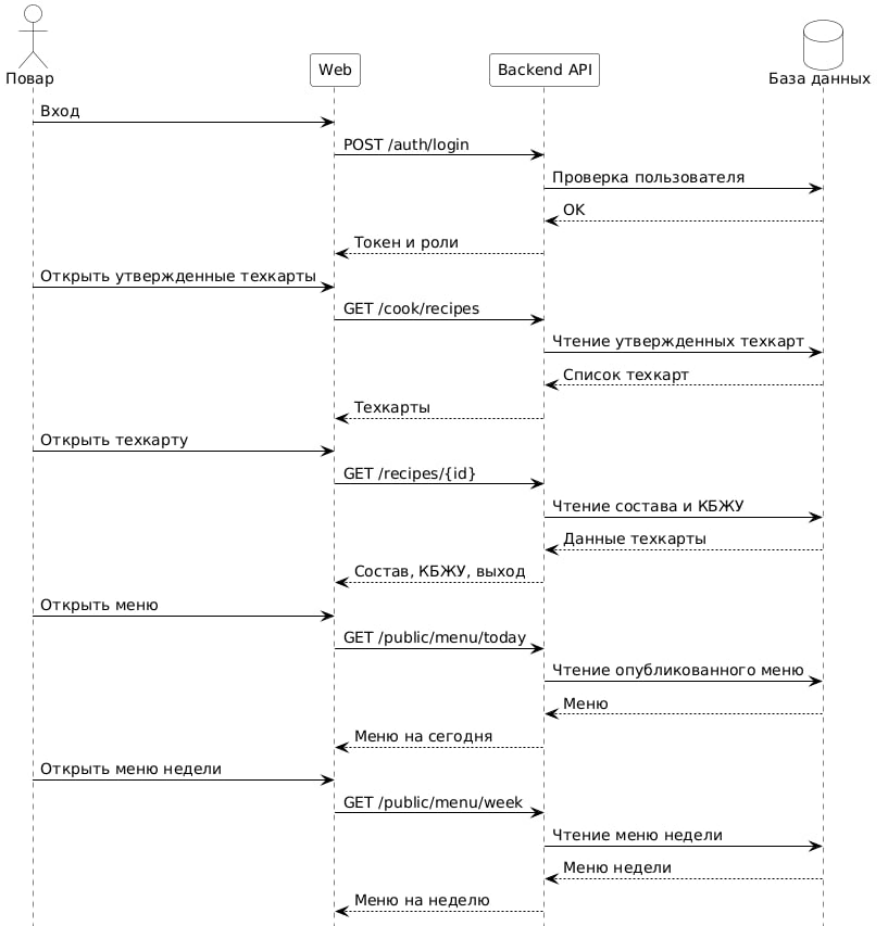


Рисунок 4.2.1.3 – Диаграмма последовательности взаимодействия повара с программным средством

4.2.1.4 Описание взаимодействия руководителя с системой

Алгоритм взаимодействия руководителя с программным средством представлен на рисунке 4.2.1.4. После авторизации пользователь выбирает филиал и получает доступ к данным по соответствующему подразделению. Далее через веб-интерфейс открывается

раздел аналитики, в котором серверная часть формирует агрегированные показатели на основе информации, содержащейся в базе данных. Кроме того, руководитель может открыть отчетный раздел и просмотреть сводные данные по заявкам, остаткам, расходу и инвентаризации. При необходимости ему также доступен просмотр опубликованного меню. Таким образом, данный сценарий показывает, что для руководителя система выполняет прежде всего информационно-аналитическую и контрольную функцию.

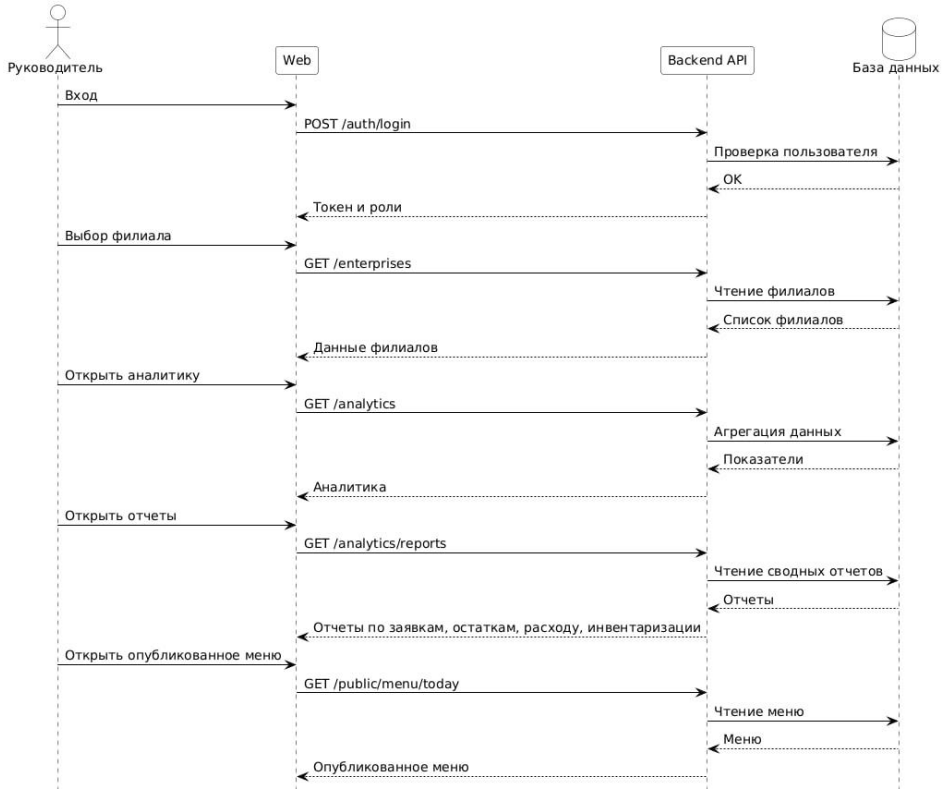


Рисунок 4.2.1.4 – Диаграмма последовательности взаимодействия руководителя с программным средством

4.2.1.5 Описание взаимодействия гостя с системой

Алгоритм взаимодействия гостя с программным средством представлен на рисунке 4.2.1.5. В данном случае пользователь не проходит авторизацию, а сразу открывает публичную страницу. После этого выбирается филиал, и через веб-интерфейс отправляется запрос на получение данных о доступных подразделениях. Затем пользователь запрашивает меню на сегодня, при необходимости открывает состав и КБЖУ конкретного блюда, а также может получить меню на неделю. Дополнительно предусмотрена возможность формирования печатной формы меню. Диаграмма показывает, что для гостя взаимодействие ограничивается публичным информационным контуром и не затрагивает внутренние функции системы.

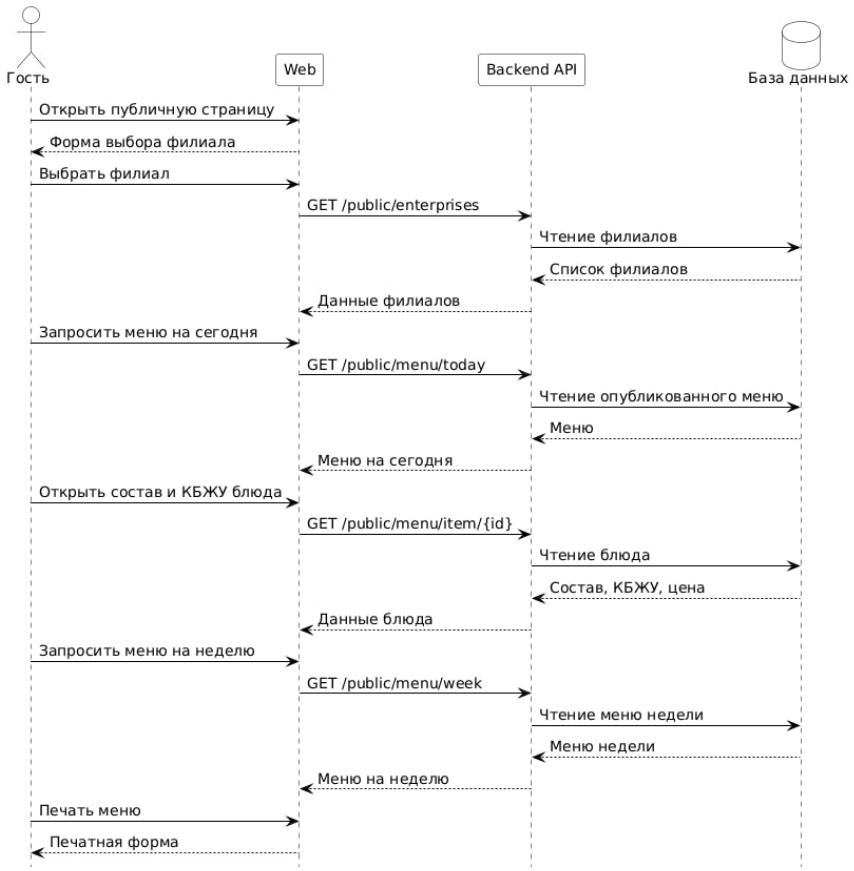


Рисунок 4.2.1.5 – Диаграмма последовательности взаимодействия гостя с программным средством

Разработанные диаграммы последовательности и диаграмма активности позволяют структурировать основные сценарии взаимодействия пользователей с программным средством и показать распределение функций между ролями. Диаграммы последовательности отражают порядок обмена данными между пользователем, клиентской частью, сервером и базой данных в рамках отдельных прецедентов. Диаграмма активности дает обобщенное представление о логике прохождения пользовательских сценариев в зависимости от роли и типа доступа. Совместное использование этих способов описания помогает уточнить структуру пользовательского диалога, выделить ключевые точки перехода между этапами работы и сформировать целостное представление о взаимодействии пользователей с программным средством. Это создает основу для дальнейшей реализации интерфейсов, серверной логики и механизмов обработки данных в рамках разрабатываемой системы.

4.2.2 Определение операций пользователей и составление функциональных блоков

В рамках проектирования интерфейса необходимо определить перечень операций, которые пользователи смогут выполнять при работе с программным средством. Под

операцией пользователя понимается отдельное завершенное действие над элементами интерфейса: ввод данных, выбор записи, нажатие кнопки, переход между разделами, сохранение документа или запуск печатной формы. Каждая операция вызывает определенную реакцию приложения: открытие окна, загрузку данных, обновление таблицы, сохранение изменений, пересчет показателей или формирование выходного документа.

Список операций пользователей:

1. Запуск десктоп-приложения или открытие веб-интерфейса.
2. Ввод логина и пароля.
3. Выбор филиала при входе.
4. Повторный ввод данных при ошибке авторизации.
5. Переход к рабочему окну после успешного входа.
6. Переход между разделами через элементы навигации.
7. Выход из учетной записи.
8. Открытие списка технологических карт.
9. Поиск техкарты по названию.
10. Выбор технологической карты из списка.
11. Просмотр реквизитов, состава и расчетных показателей техкарты.
12. Создание новой технологической карты.
13. Редактирование реквизитов существующей техкарты.
14. Добавление продукта в состав блюда.
15. Редактирование или удаление продукта из состава.
16. Сохранение технологической карты.
17. Формирование печатной формы техкарты.
18. Экспорт техкарты в PDF.
19. Открытие справочника продуктов.
20. Поиск и выбор продукта из справочника.
21. Просмотр и редактирование карточки продукта.
22. Сохранение изменений в справочнике продуктов.
23. Открытие раздела меню.
24. Создание или открытие дня меню.
25. Автоматическое формирование меню.
26. Добавление, редактирование и удаление блюда в меню.
27. Просмотр потребности в сырье и дефицитных позиций.
28. Печать или экспорт меню.
29. Утверждение и публикация меню.

30. Открытие раздела заявок.

31. Создание новой заявки на склад.

32. Добавление, редактирование и удаление позиций заявки.

33. Сохранение заявки.

34. Отправка заявки на склад.

35. Просмотр списка заявок и карточки выбранной заявки.

36. Изменение статуса заявки.

37. Подтверждение, частичная выдача или отклонение заявки.

38. Просмотр складских остатков.

39. Создание и сохранение приходного документа.

40. Создание и сохранение расходного документа.

41. Создание инвентаризации.

42. Заполнение инвентаризации по текущим остаткам.

43. Ввод фактических данных по инвентаризации.

44. Сохранение, печать и экспорт акта инвентаризации.

45. Запуск синхронизации данных.

46. Просмотр утвержденных техкарт поваром.

47. Просмотр состава, КБЖУ, выхода и технологии приготовления блюда.

48. Просмотр аналитики и отчетности руководителем.

49. Выбор филиала и периода для анализа.

50. Просмотр публичного меню гостем.

51. Выбор филиала на публичной странице.

52. Просмотр меню на день или на неделю.

53. Просмотр состава, КБЖУ и цены блюда.

54. Печать публичного меню.

Совокупность перечисленных операций позволяет выделить следующие функциональные блоки программного средства: блок авторизации и входа в систему, блок навигации и общих сервисных операций, блок технологических карт, блок справочников, блок планирования меню, блок заявок и складских операций, блок инвентаризации, блок производственного просмотра, блок аналитики и отчетности, блок публичного меню.

Выделенные операции и функциональные блоки показывают, что интерфейс программного средства построен по ролевому принципу. Пользователь получает доступ только к тем действиям и разделам, которые соответствуют его задачам, а близкие по назначению операции объединены в единые рабочие области. Это делает взаимодействие с приложением более понятным и упорядоченным.

4.2.3 Проектирование структуры экранов и схемы навигации

На данном этапе на основе пользовательских сценариев, ролей и ранее выделенных функциональных блоков формируется структура экранов программного средства и определяется логика переходов между ними. Проектирование структуры экранов необходимо для того, чтобы задать понятный маршрут перемещения пользователя по системе, разграничить рабочие области по назначению и обеспечить удобный доступ к функциям, соответствующим конкретной роли. В разрабатываемом решении навигация организована по двум основным пользовательским контурам: ВЕБ, предназначенному для публичного доступа и веб-кабинетов сотрудников, и ДЕСКТОП, предназначенному для внутренней работы с расширенным набором функций. Отдельно на схеме выделен серверный контур, обеспечивающий обработку запросов и доступ к централизованной базе данных.

Подписи на переходах между экранами соответствуют номерам операций пользователей, определенных в предыдущем подразделе. Это позволяет связать навигационную схему со структурой интерфейса и с конкретными действиями, выполняемыми пользователями в рамках прецедентов.

Основные экраны системы

1. Экран главной страницы веб-контура.
2. Экран выбора режима работы.
3. Экран входа сотрудников в веб-контуре.
4. Экран определения роли сотрудника.
5. Экран меню для посетителей.
6. Экран выбора филиала в публичном режиме.
7. Экран публичного меню на день или неделю.
8. Веб-кабинет технолога.
9. Веб-кабинет кладовщика.
10. Веб-кабинет повара.
11. Веб-кабинет руководителя.
12. Экран отчетов руководителя в веб-контуре.
13. Экран меню и статусов заявок для технолога.
14. Экран заявок и выдачи для кладовщика.
15. Экран утвержденных техкарт и рабочего меню для повара.
16. Экран входа десктоп-контура.
17. Экран выбора режима работы в десктоп-приложении.
18. Экран синхронизации.
19. Экран определения роли пользователя десктоп-контура.

20. Экран выбора филиала во внутреннем контуре.

21. Десктоп-кабинет технолога.

22. Десктоп-кабинет кладовщика.

23. Десктоп-кабинет повара.

24. Десктоп-кабинет руководителя.

25. Экран сводной аналитики по филиалам.

26. Экран просмотра утвержденных техкарт и меню.

27. Экран работы с техкартами, продуктами, меню, заявками и справочниками.

28. Экран работы со складом, заявками, приходом, расходом и инвентаризацией.

29. Блок операций веб-контура.

30. Блок операций десктоп-контура.

31. Серверный контур, включающий Backend API и PostgreSQL.

Подписи на переходах между экранами соответствуют номерам операций пользователей. Это позволяет связать навигационные схемы с конкретными действиями, выполняемыми пользователями при работе с системой.

К основным экранам веб-контура относятся главная страница, экран выбора режима, экран входа сотрудников, экран определения роли сотрудника, экран меню для посетителей, экран выбора филиала, экран публичного меню, а также кабинеты технолога, кладовщика, повара и руководителя. Для каждой роли предусмотрен собственный рабочий экран: для технолога это экран меню и статусов заявок, для кладовщика — экран заявок и выдачи, для повара — экран утвержденных технологических карт и рабочего меню, для руководителя — экран отчетов по заявкам, остаткам, расходу и инвентаризации.

К основным экранам десктоп-контура относятся экран входа, экран выбора режима работы, экран синхронизации, экран определения роли пользователя, экран выбора филиала, а также кабинеты технолога, кладовщика, повара и руководителя. Для каждой роли предусмотрен свой набор внутренних рабочих экранов: технолог работает с техкартами, продуктами, меню, заявками и справочниками, кладовщик — со складом, приходом, расходом, заявками и инвентаризацией, повар — с утвержденными техкартами и меню, руководитель — со сводной аналитикой по филиалам.

Перечисленные экраны образуют не набор изолированных форм, а взаимосвязанную структуру интерфейса. Для более наглядного представления маршрутов перехода между экранами общая схема была разделена на две самостоятельные части: навигационную схему веб-контура и навигационную схему десктоп-контура



Рисунок 4.2.3.1 – Навигационная схема десктоп-контура

Как показано на рисунке 4.2.3.1, работа во внутреннем приложении начинается с экрана входа, после чего пользователь выбирает режим работы. При необходимости выполняется переход к экрану синхронизации, что отражает поддержку обмена данными между локальным и серверным контуром. После этого система определяет роль пользователя, выполняется выбор филиала, и далее пользователь переходит в соответствующий кабинет.

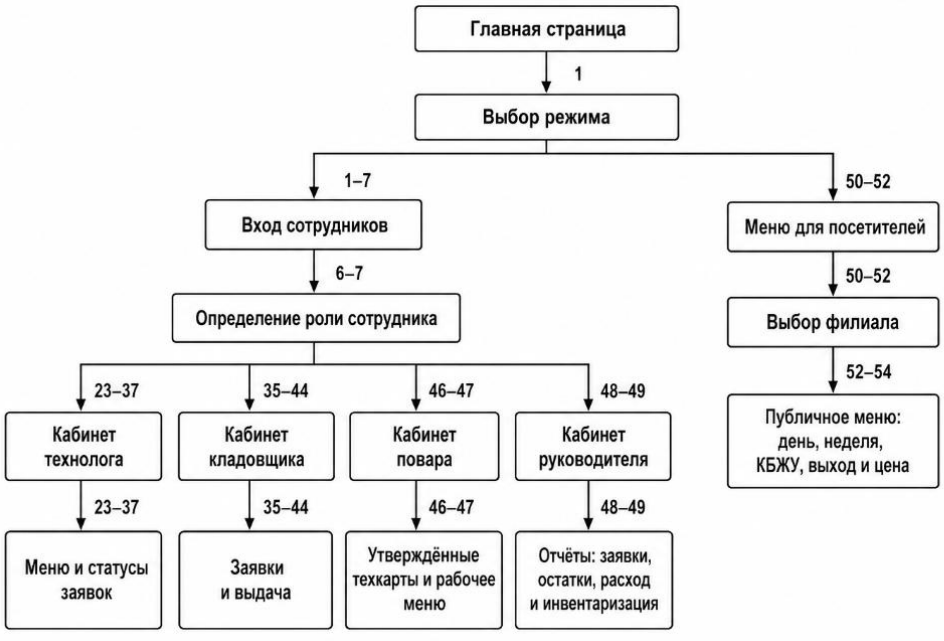


Рисунок 4.2.3.2 – Навигационная схема веб-контура

Как показано на рисунке 4.2.3.2, работа в ВЕБ-контуре начинается с главной страницы, после чего пользователь переходит к экрану выбора режима. Далее навигация разделяется на два основных сценария. Первый сценарий относится к сотрудникам: пользователь выполняет вход, после чего система определяет его роль и направляет в соответствующий кабинет. Технолог переходит к экрану меню и статусов заявок, кладовщик — к экрану заявок и выдачи, повар — к экрану утвержденных техкарт и рабочего меню, руководитель — к экрану отчетов по заявкам, остаткам, расходу и инвентаризации.

Второй сценарий относится к гостю. В этом случае после выбора режима пользователь переходит к экрану меню для посетителей, затем выбирает филиал и получает доступ к публичному меню на текущий день или неделю. В публичной части отображаются основные характеристики блюда, включая КБЖУ, выход и цену. Такое разделение навигации позволяет отделить внешний информационный доступ от внутренних рабочих функций сотрудников.

Технолог получает доступ к экрану работы с техкартами, продуктами, меню, заявками и справочниками. Кладовщик переходит к экрану складских операций, включающему работу с остатками, заявками, приходом, расходом и инвентаризацией. Повар использует экран просмотра утвержденных технологических карт и меню. Руководитель получает доступ к сводной аналитике по филиалам. Такое построение схемы показывает, что ДЕСКТОП-контур ориентирован на выполнение основных внутренних операций и охватывает более широкий набор рабочих функций, чем ВЕБ-часть.

Разделение навигации на две отдельные схемы позволяет сделать структуру экранов более понятной и читаемой. ВЕБ-схема отражает публичный доступ и ограниченные ролевые кабинеты сотрудников, тогда как ДЕСКТОП-схема показывает внутренний контур с расширенным набором рабочих операций. Это помогает точнее отразить распределение функций между экранами и упрощает дальнейшее описание интерфейса программного средств

4.2.4 Разработка дизайна интерфейса

Проектирование пользовательского интерфейса является одним из ключевых этапов разработки программного средства, поскольку именно интерфейс определяет удобство и эффективность взаимодействия пользователя с системой. Интерфейс должен обеспечивать интуитивность, визуальную логичность, доступность основных функций и последовательность поведения на всех экранах. Для достижения этих целей дизайн создаётся поэтапно – от концептуальной структуры до финального стилистического оформления. Такой подход позволяет детально продумать архитектуру экранов, их содержимое и визуальную идентичность.

В рамках данного этапа были разработаны макеты двух ключевых экранов системы MealSync — экрана публичного меню и экрана рабочего кабинета. Эти формы отражают два основных сценария взаимодействия с программным средством: внешний доступ к опубликованному меню и внутренняя работа сотрудников с технологическими и учетными данными. Каждый экран разрабатывался в три этапа: выделение функциональных областей, размещение элементов интерфейса и формирование финального визуального решения. Такой подход позволяет последовательно перейти от общей структуры к детализированному оформлению и обеспечить единообразие интерфейса во всех частях системы. Эскиз экрана публичного меню представлен на рисунке 4.2.4.1.

Этап 1. Разграничение областей.

На первом этапе определяется общая композиция экрана без детальной стилизации. Выделяются основные функциональные зоны:

- верхняя панель — логотип, краткое описание, переключение языка;
- центральная карточка — заголовок и область фильтрации;
- нижняя рабочая область — список или таблица блюд.

Цель этапа заключается в формировании базового каркаса экрана и в определении назначения каждой области. Такое построение особенно важно для публичной части, поскольку пользователь должен сразу понимать, где находятся средства выбора филиала и где отображается само меню.

Этап 2. Размещение кнопок и элементов.

На втором этапе в пределах выделенных областей размещаются элементы управления и информационные блоки:

- логотип и наименование системы;
- пункт перехода к меню для посетителей;
- кнопка перехода ко входу сотрудников;
- элемент выбора языка;
- блок фильтров по дате, филиалу и типу столовой;
- кнопка быстрого перехода к актуальному меню;
- таблица или список блюд с основными характеристиками.

Размещение выполняется без окончательной стилизации. Основная задача этапа — определить точное положение элементов, их взаимную иерархию и логику чтения экрана слева направо и сверху вниз.

Этап 3. Цвет, текст и стиль.

На третьем этапе формируется итоговое визуальное оформление экрана:

- используется светлая цветовая схема с мягким фоновым тоном;

– ключевые действия и активные элементы выделяются зеленым акцентом;
– применяются скругленные карточки и кнопки;
– усиливается контраст между заголовками, подписями и табличными данными;
– выравниваются отступы и интервалы между блоками.

В результате экран приобретает завершенный вид и становится удобным для быстрого просмотра опубликованного меню. Визуальный акцент делается на простоте, читаемости и ясной подаче информации о блюдах.

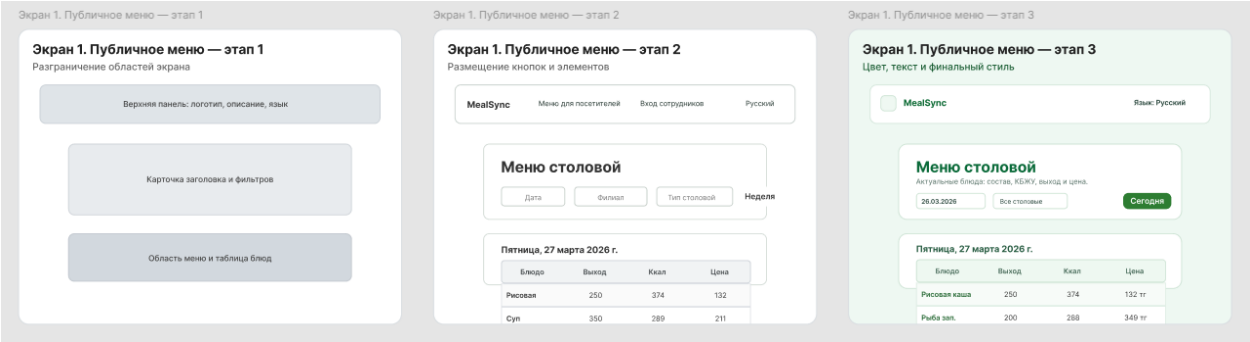


Рисунок 4.2.4.1 – Эскиз публичного меню

Экран рабочего кабинета на рисунке 4.2.4.2:

Этап 1. Разграничение областей.

На первом этапе задается каркас рабочего интерфейса, ориентированного на сотрудников системы. Выделяются следующие зоны:

– верхняя панель действий;
– боковое меню навигации;
– центральная область со списком объектов;
– правая область карточки выбранного объекта.

Цель этапа — определить устойчивую рабочую структуру, которая может использоваться в разных модулях системы: техкарты, продукты, меню, заявки и складские операции.

Этап 2. Размещение кнопок и элементов.

На втором этапе в пределах выделенных областей размещаются основные элементы интерфейса:

– кнопки создания новой записи, нового продукта и синхронизации;
– переключатель языка;
– боковые кнопки перехода между разделами;
– строка поиска;
– таблица со списком технологических карт;
– кнопки добавления и редактирования;
– карточка выбранной техкарты с основными реквизитами.

На этом этапе определяется логика рабочего взаимодействия: пользователь выбирает раздел, выполняет поиск, работает со списком записей и получает доступ к подробной карточке объекта.

Этап 3. Цвет, текст и стиль.

На третьем этапе формируется финальное визуальное решение рабочего кабинета:

- сохраняется единая светлая цветовая схема;
- активные кнопки и выбранные строки выделяются зеленым цветом;
- боковое меню и карточки получают единый стиль скругленных блоков;
- усиливается визуальное разделение между навигацией, списком и карточкой;
- текстовые подписи и табличные данные приводятся к единой иерархии.

Итоговый вариант рабочего кабинета ориентирован на длительную повседневную работу сотрудника. Интерфейс строится по принципу «навигация — список — карточка», что упрощает выполнение повторяющихся операций и делает работу с технологическими и учетными данными более упорядоченной.

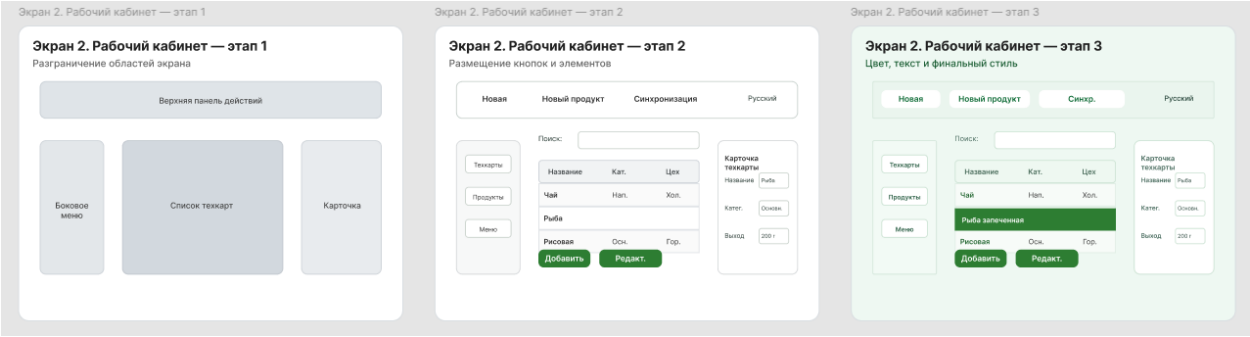


Рисунок 4.2.4.2 – Эскиз рабочего кабинета

Разработанные эскизы показывают поэтапный переход от функциональной схемы экрана к его окончательному визуальному оформлению. Экран публичного меню ориентирован на внешнего пользователя и делает акцент на простом доступе к информации, тогда как экран рабочего кабинета предназначен для сотрудников и поддерживает более плотный, операционный формат работы. Оба макета выполнены в едином визуальном стиле, что обеспечивает целостность интерфейса программного средства.

4.3 Входные, выходные и промежуточные данные

В программном средстве MealSync обработка информации осуществляется в рамках клиент-серверной архитектуры, включающей десктоп-приложение, веб-интерфейс и серверную часть. Поэтому данные в системе представлены не одним файлом, а совокупностью экранных форм, структурированных записей, расчетных значений и итоговых документов.

Входная информация вводится пользователем через формы интерфейса, промежуточные данные формируются в процессе хранения и обработки сведений, а выходные данные представляют собой готовые документы и итоговые формы отображения. Передача данных между клиентскими приложениями и серверной частью выполняется по REST API в формате JSON, а итоговые документы формируются в формате PDF.

Входные данные. К входным данным относятся сведения, которые пользователь непосредственно вводит в систему через экранные формы. Для MealSync это реквизиты технологической карты, наименование блюда, категория, цех, выход порции, состав ингредиентов, значения массы брутто и процента потерь, а также служебные данные, необходимые для последующего сохранения и расчета. Пример входных данных представлен на рисунке 4.3.1.

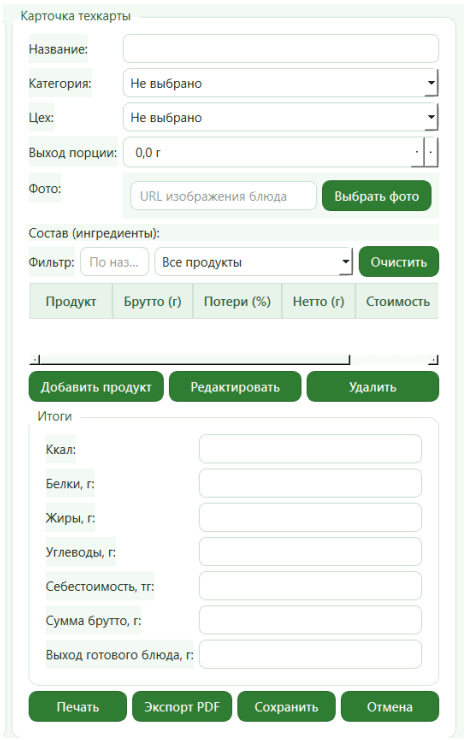


Рисунок 4.3.1 – Форма ввода технологической карты

На рисунке 4.3.1 показана пустая форма технологической карты, предназначенная для ввода исходной информации о блюде. В верхней части формы расположены поля Название, Категория, Цех, Выход порции и блок выбора изображения. Ниже размещена таблица состава ингредиентов с колонками Продукт, Брутто, Потери, Нетто, Стоимость, а также кнопки добавления, редактирования и удаления позиций. В нижней части формы находятся итоговые расчетные поля и кнопки Печать, Экспорт PDF, Сохранить, Отмена. Данный экран отражает структуру входной информации, поступающей в систему от пользователя.

Промежуточные данные формируются и используются системой в процессе обработки входной информации. К ним относятся нормативно-справочные сведения, на основании которых выполняются расчеты и строятся итоговые формы. В MealSync такими данными являются карточки продуктов, содержащие их основные характеристики: категорию, единицу измерения, массу единицы, энергетическую ценность, содержание белков, жиров и углеводов, а также цену. Эти сведения не являются конечным результатом работы системы, однако используются при расчете КБЖУ, себестоимости и стоимости ингредиентов в технологической карте. Пример промежуточных данных представлен на рисунке 4.3.2.



Рисунок 4.3.2 – Карточка продукта как пример промежуточных данных

На рисунке 4.3.2 показана заполненная карточка продукта. В форме отображаются поля Название, Категория, Единица, Масса единицы, Ккал/100 г, Белки/100 г, Жиры/100 г, Углеводы/100 г, Цена за 1 кг. Эти данные используются системой как промежуточная нормативная база для дальнейших расчетов в технологических картах и меню. Следовательно, карточка продукта отражает структуру промежуточных данных, которые не формируются как самостоятельный итоговый документ, но необходимы для корректной работы расчетных механизмов системы.

Выходные данные представляют собой результаты обработки входной информации, отображаемые пользователю или формируемые в виде готовых документов. В MealSync к выходным данным относятся заполненные технологические карты, меню, заявки, отчеты и печатные формы. Они используются для просмотра, передачи, хранения и печати. Пример выходных данных представлен на рисунке 4.3.3.

Технологическая карта (ГОСТ)

Блюдо: **Плов**
Категория: Основные блюда
Цех: Горячий цех
Выход порции: 300.0 г

№	Наименование сырья	Брутто, г	Потери, %	Нетто, г	Цена, тг/кг	Сумма, тг
1	Рис	80.00	0.00	80.00	650.00	52.00
2	Говядина	100.00	5.00	95.00	2200.00	209.00
3	Морковь	60.00	5.00	57.00	140.00	7.98
4	Лук репчатый	40.00	5.00	38.00	120.00	4.56
5	Масло растительное	15.00	0.00	15.00	700.00	10.50
6	Соль	2.00	0.00	2.00	40.00	0.08

Ккал	626.65
Белки, г	24.36
Жиры, г	27.52
Углеводы, г	72.77
Сумма брутто, г	297.00
Выход готового блюда, г	287.00
Себестоимость, тг	284.12

Составил(а): _____ /_____/
Утвердил(а): _____ /_____/
Дата: ____ / ____ / _____

Рисунок 4.3.3 – Печатная форма технологической карты в формате PDF

На рисунке 4.3.3 представлена сформированная технологическая карта в формате PDF. Документ содержит итоговые сведения о блюде: наименование, категорию, цех, выход порции, перечень сырья, значения массы брутто и нетто, процент потерь, стоимость по каждой позиции, а также суммарные показатели калорийности, белков, жиров, углеводов, общей массы и себестоимости. В отличие от входных и промежуточных данных, данная форма является завершенным результатом обработки и предназначена для использования в производственной и документной работе.

Общая характеристика структуры данных. Рассмотренные примеры показывают, что в системе MealSync данные последовательно проходят несколько стадий преобразования. Сначала пользователь вводит исходные сведения через экранные формы, затем система использует промежуточные нормативно-справочные данные и расчетные параметры для внутренней обработки, после чего формирует итоговые выходные документы и формы отображения. Такое разделение позволяет четко определить источник данных, их роль в процессе обработки и конечный результат использования в программном средстве.

4.4 Реализация используемых методов хранения, обработки и передачи информации об объектах предметной области

4.4.1 Методы хранения данных

В программном средстве MealSync для хранения информации используется реляционная база данных. Выбор именно такого способа хранения обусловлен необходимостью организованного представления большого объема взаимосвязанных сведений, относящихся к различным функциональным контурам системы. В базе данных хранятся сведения о пользователях и их ролях, нормативно-справочная информация, данные о продуктах, технологических картах, меню, складских документах, заявках, партиях товаров, операциях прихода и расхода, а также результаты инвентаризации. Представление этих данных в виде набора связанных таблиц позволяет логически разделить информацию по ее назначению и одновременно обеспечить целостность общей структуры хранения.

Реляционный подход удобен для рассматриваемой предметной области, поскольку большинство объектов имеют четко выраженные связи друг с другом. Например, технологическая карта связана с категорией блюда, производственным цехом и перечнем ингредиентов; меню связано с конкретной датой, филиалом и набором блюд; заявка на склад включает позиции продуктов; операции расхода и инвентаризации опираются на складские остатки и партии товаров. Если такие данные хранить разрозненно, это приведет к дублированию информации, ошибкам при обновлении и затруднению контроля. Использование реляционной базы данных позволяет избежать этих проблем за счет выделения самостоятельных сущностей и задания между ними устойчивых связей.

Каждая таблица базы данных предназначена для хранения определенного набора данных, относящихся к конкретному объекту или документу. При этом одни таблицы выполняют роль справочников, другие содержат основные операционные данные, а третьи фиксируют состав документов и историю изменений. Связи между таблицами обеспечивают согласованность информации в рамках всей системы, позволяют обращаться к связанным данным без повторного ввода и создают основу для корректной работы всех модулей программного средства. Именно за счет такой организации хранения становится возможным совместное использование данных технологом, кладовщиком, поваром, руководителем и другими участниками процесса.

Для наглядного представления структуры хранения данных в данном разделе приводится ER-диаграмма базы данных, описание основных таблиц и характеристика связей между ними. ER-диаграмма позволяет отразить состав сущностей и показать, каким

образом между ними распределены отношения 1:1, 1:M и другие типы зависимостей. ER-диаграмма базы данных программного средства MealSync представлена на рисунке 4.4.1.1.

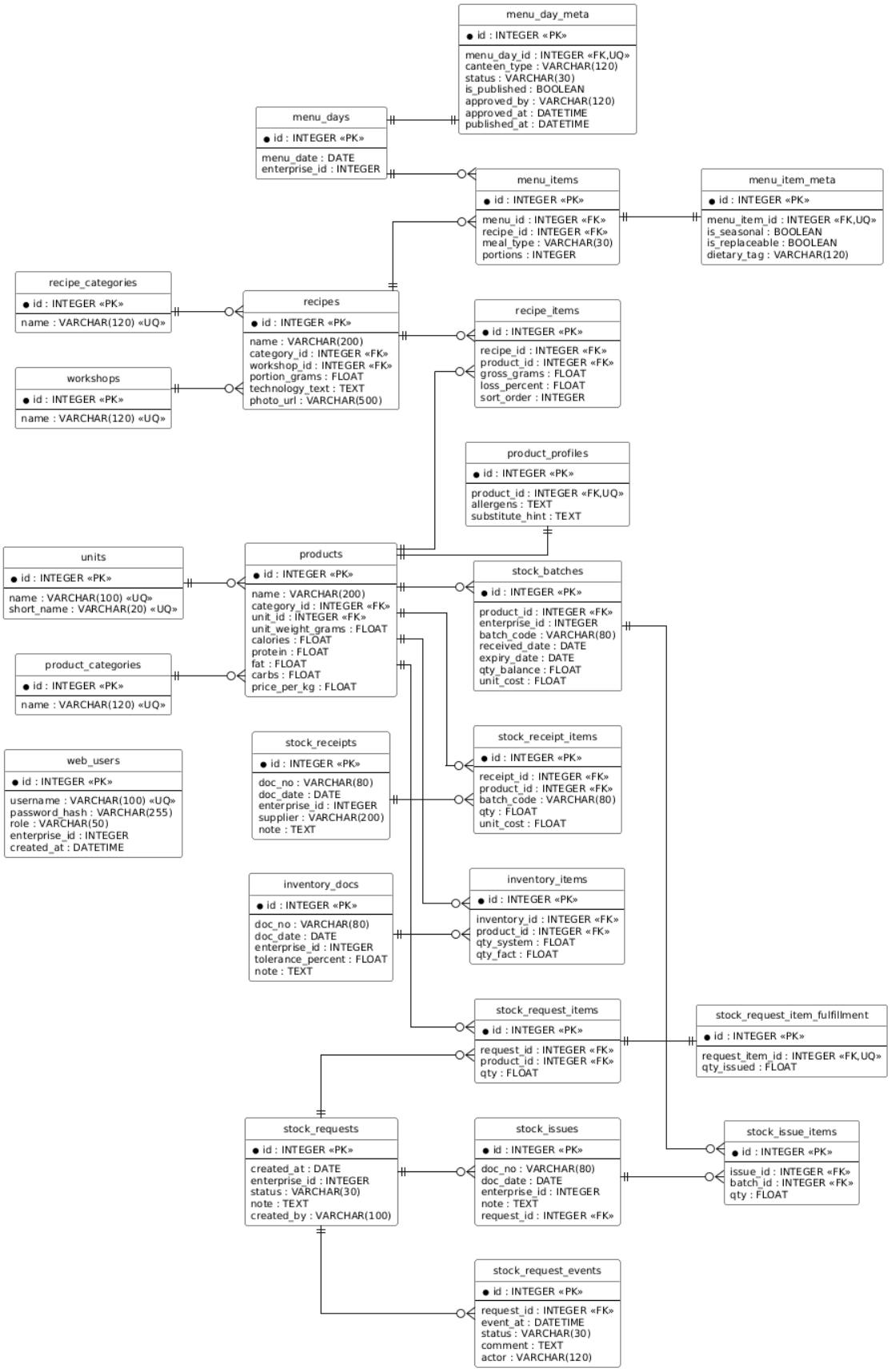


Рисунок 4.4.1.1 – ER-диаграмма базы данных программного средства MealSync

ER-диаграмма позволяет получить общее представление о составе базы данных, однако для более детального рассмотрения необходимо описать каждую основную таблицу отдельно. В таблицах 4.4.1.1-4.4.1.24 приведена структура ключевых отношений базы данных с указанием полей, их типов, свойств и назначения.

Таблица 4.4.1.1 – Отношение «Пользователь»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор пользователя
username	Текстовый	Длина до 100 символов, уникальное	Обязательное	Логин пользователя для входа в систему
password_hash	Текстовый	Длина до 255 символов	Обязательное	Хеш пароля пользователя
role	Текстовый	Длина до 50 символов	Обязательное	Роль пользователя в системе
enterprise_id	Числовой	Длинное целое	Обязательное	Идентификатор филиала пользователя
created_at	Дата/время	Формат YYYY-MM-DD HH:MM	Обязательное	Дата и время создания учетной записи

Таблица 4.4.1.2 – Отношение «Единица измерения»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор единицы измерения
name	Текстовый	Длина до 100 символов, уникальное	Обязательное	Полное наименование единицы измерения
short_name	Текстовый	Длина до 20 символов, уникальное	Обязательное	Краткое обозначение единицы измерения

Таблица 4.4.1.3 – Отношение «Категория продукта»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор категории продукта
name	Текстовый	Длина до 120 символов, уникальное	Обязательное	Наименование категории продукта

Таблица 4.4.1.4 – Отношение «Категория блюда»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор категории блюда
name	Текстовый	Длина до 120 символов, уникальное	Обязательное	Наименование категории блюда

Таблица 4.4.1.5 – Отношение «Цех»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор производственного цеха
name	Текстовый	Длина до 120 символов, уникальное	Обязательное	Наименование производственного цеха

Таблица 4.4.1.6 – Отношение «Продукт»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор продукта
name	Текстовый	Длина до 200 символов	Обязательное	Наименование продукта
category_id	Числовой	Длинное целое	Внешний ключ	Ссылка на категорию продукта
unit_id	Числовой	Длинное целое	Внешний ключ	Ссылка на единицу измерения
unit_weight_grams	Вещественный	Неотрицательное значение	Необязательное	Масса единицы продукта в граммах
calories	Вещественный	Неотрицательное значение	Обязательное	Калорийность продукта
protein	Вещественный	Неотрицательное значение	Обязательное	Содержание белков
fat	Вещественный	Неотрицательное значение	Обязательное	Содержание жиров
carbs	Вещественный	Неотрицательное значение	Обязательное	Содержание углеводов
price_per_kg	Вещественный	Неотрицательное значение	Обязательное	Цена продукта за 1 кг

Таблица 4.4.1.7 – Отношение «Профиль продукта»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор записи профиля продукта
product_id	Числовой	Длинное целое, уникальное	Внешний ключ	Ссылка на продукт
allergens	Текстовый	Произвольная длина	Необязательное	Сведения об аллергенах продукта
substitute_hint	Текстовый	Произвольная длина	Необязательное	Рекомендации по замене продукта

Таблица 4.4.1.8 – Отношение «Технологическая карта»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор технологической карты
name	Текстовый	Длина до 200 символов	Обязательное	Наименование блюда
category_id	Числовой	Длинное целое	Внешний ключ	Ссылка на категорию блюда
workshop_id	Числовой	Длинное целое	Внешний ключ	Ссылка на производственный цех
portion_grams	Вещественный	Неотрицательное значение	Обязательное	Выход порции блюда в граммах
technology_text	Текстовый	Произвольная длина	Необязательное	Описание технологии приготовления блюда
photo_url	Текстовый	Длина до 500 символов	Необязательное	Ссылка на изображение блюда

Таблица 4.4.1.9 – Отношение «Состав технологической карты»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор позиции состава
recipe_id	Числовой	Длинное целое	Внешний ключ	Ссылка на технологическую карту
product_id	Числовой	Длинное целое	Внешний ключ	Ссылка на продукт
gross_grams	Вещественный	Неотрицательное значение	Обязательное	Масса брутто ингредиента
loss_percent	Вещественный	Неотрицательное значение	Обязательное	Процент потерь при обработке
sort_order	Числовой	Целое число	Необязательное	Порядок отображения ингредиента в составе

Таблица 4.4.1.10 – Отношение «Меню по дням»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор дня меню
menu_date	Дата	Формат YYYY-MM-DD	Обязательное	Дата меню
enterprise_id	Числовой	Длинное целое	Обязательное	Идентификатор филиала

Таблица 4.4.1.11 – Отношение «Параметры меню по дню»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор записи параметров меню
menu_day_id	Числовой	Длинное целое, уникальное	Внешний ключ	Ссылка на день меню
canteen_type	Текстовый	Длина до 120 символов	Обязательное	Тип столовой
status	Текстовый	Длина до 30 символов	Обязательное	Статус меню
is_published	Логический	Значения Да/Нет	Обязательное	Признак публикации меню
approved_by	Текстовый	Длина до 120 символов	Необязательное	Пользователь, утвердивший меню
approved_at	Дата/время	Формат YYYY-MM-DD HH:MM	Необязательное	Дата и время утверждения меню
published_at	Дата/время	Формат YYYY-MM-DD HH:MM	Необязательное	Дата и время публикации меню

Таблица 4.4.1.12 – Отношение «Позиция меню»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор позиции меню
menu_id	Числовой	Длинное целое	Внешний ключ	Ссылка на меню по дню
recipe_id	Числовой	Длинное целое	Внешний ключ	Ссылка на технологическую карту
meal_type	Текстовый	Длина до 30 символов	Обязательное	Тип приема пищи
portions	Числовой	Целое число	Обязательное	Количество порций

Таблица 4.4.1.13 – Отношение «Параметры позиции меню»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор записи параметров позиции меню
menu_item_id	Числовой	Длинное целое, уникальное	Внешний ключ	Ссылка на позицию меню
is_seasonal	Логический	Значения Да/Нет	Обязательное	Признак сезонного блюда
is_replaceable	Логический	Значения Да/Нет	Обязательное	Признак возможности замены блюда
dietary_tag	Текстовый	Длина до 120 символов	Необязательное	Диетическая или служебная метка блюда

Таблица 4.4.1.14 – Отношение «Приходный документ»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор приходного документа
doc_no	Текстовый	Длина до 80 символов	Обязательное	Номер приходного документа
doc_date	Дата	Формат YYYY-MM-DD	Обязательное	Дата приходного документа
enterprise_id	Числовой	Длинное целое	Обязательное	Идентификатор филиала
supplier	Текстовый	Длина до 200 символов	Необязательное	Наименование поставщика
note	Текстовый	Произвольная длина	Необязательное	Примечание к документу

Таблица 4.4.1.15 – Отношение «Позиция приходного документа»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор позиции приходного документа
receipt_id	Числовой	Длинное целое	Внешний ключ	Ссылка на приходный документ
product_id	Числовой	Длинное целое	Внешний ключ	Ссылка на продукт
batch_code	Текстовый	Длина до 80 символов	Обязательное	Код партии товара
qty	Вещественный	Неотрицательное значение	Обязательное	Количество поступившего товара
unit_cost	Вещественный	Неотрицательное значение	Обязательное	Стоимость единицы товара

Таблица 4.4.1.16 – Отношение «Партия склада»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор складской партии
product_id	Числовой	Длинное целое	Внешний ключ	Ссылка на продукт
enterprise_id	Числовой	Длинное целое	Обязательное	Идентификатор филиала
batch_code	Текстовый	Длина до 80 символов	Обязательное	Код партии
received_date	Дата	Формат YYYY-MM-DD	Обязательное	Дата поступления партии
expiry_date	Дата	Формат YYYY-MM-DD	Необязательное	Срок годности партии
qty_balance	Вещественный	Неотрицательное значение	Обязательное	Остаток по партии
unit_cost	Вещественный	Неотрицательное значение	Обязательное	Стоимость единицы товара

Таблица 4.4.1.17 – Отношение «Заявка на склад»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор заявки на склад
created_at	Дата	Формат YYYY-MM-DD	Обязательное	Дата создания заявки
enterprise_id	Числовой	Длинное целое	Обязательное	Идентификатор филиала
status	Текстовый	Длина до 30 символов	Обязательное	Текущий статус заявки
note	Текстовый	Произвольная длина	Необязательное	Примечание к заявке
created_by	Текстовый	Длина до 100 символов	Необязательное	Пользователь, создавший заявку

Таблица 4.4.1.18 – Отношение «Позиция заявки на склад»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор позиции заявки
request_id	Числовой	Длинное целое	Внешний ключ	Ссылка на заявку на склад
product_id	Числовой	Длинное целое	Внешний ключ	Ссылка на продукт
qty	Вещественный	Неотрицательное значение	Обязательное	Запрошенное количество продукта

Таблица 4.4.1.19 – Отношение «Исполнение позиции заявки»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор записи исполнения
request_item_id	Числовой	Длинное целое, уникальное	Внешний ключ	Ссылка на позицию заявки
qty_issued	Вещественный	Неотрицательное значение	Обязательное	Фактически выданное количество продукта

Таблица 4.4.1.20 – Отношение «Журнал событий заявки»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор события
request_id	Числовой	Длинное целое	Внешний ключ	Ссылка на заявку на склад
event_at	Дата/время	Формат YYYY-MM-DD HH:MM	Обязательное	Дата и время изменения статуса или события
status	Текстовый	Длина до 30 символов	Обязательное	Статус заявки на момент события
comment	Текстовый	Произвольная длина	Необязательное	Комментарий к изменению статуса
actor	Текстовый	Длина до 120 символов	Необязательное	Пользователь, выполнивший действие

Таблица 4.4.1.21 – Отношение «Документ расхода»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор документа расхода
doc_no	Текстовый	Длина до 80 символов	Обязательное	Номер документа расхода
doc_date	Дата	Формат YYYY-MM-DD	Обязательное	Дата документа расхода
enterprise_id	Числовой	Длинное целое	Обязательное	Идентификатор филиала

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
note	Текстовый	Произвольная длина	Необязательное	Примечание к документу расхода
request_id	Числовой	Длинное целое	Внешний ключ	Ссылка на заявку на склад

Таблица 4.4.1.22 – Отношение «Позиция расхода»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор позиции расхода
issue_id	Числовой	Длинное целое	Внешний ключ	Ссылка на документ расхода
batch_id	Числовой	Длинное целое	Внешний ключ	Ссылка на складскую партию
qty	Вещественный	Неотрицательное значение	Обязательное	Количество списываемого товара

Таблица 4.4.1.23 – Отношение «Документ инвентаризации»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор документа инвентаризации
doc_no	Текстовый	Длина до 80 символов	Обязательное	Номер документа инвентаризации
doc_date	Дата	Формат YYYY-MM-DD	Обязательное	Дата документа инвентаризации
enterprise_id	Числовой	Длинное целое	Обязательное	Идентификатор филиала
tolerance_percent	Вещественный	Неотрицательное значение	Необязательное	Допустимый процент отклонения
note	Текстовый	Произвольная длина	Необязательное	Примечание к документу инвентаризации

Таблица 4.4.1.24 – Отношение «Позиция инвентаризации»

Имя поля	Тип данных	Свойства поля	Тип поля	Назначение
id	Числовой	Длинное целое, автоинкремент	Ключевое	Уникальный идентификатор позиции инвентаризации
inventory_id	Числовой	Длинное целое	Внешний ключ	Ссылка на документ инвентаризации
product_id	Числовой	Длинное целое	Внешний ключ	Ссылка на продукт
qty_system	Вещественный	Неотрицательное значение	Обязательное	Учетное количество продукта
qty_fact	Вещественный	Неотрицательное значение	Обязательное	Фактическое количество продукта

После описания структуры таблиц базы данных необходимо рассмотреть связи между ними. Именно связи позволяют объединить отдельные таблицы в единую информационную модель и обеспечивают согласованное хранение данных в различных функциональных модулях системы. За счет использования связей между таблицами достигается целостность информации, исключается избыточное дублирование сведений и обеспечивается

корректная работа операций, связанных с технологическими картами, меню, складским учетом и инвентаризацией.

Связи между таблицами базы данных

- Таблицы product_categories и products связаны 1:M — одна категория продукта может соответствовать множеству продуктов.
- Таблицы units и products связаны 1:M — одна единица измерения может использоваться для многих продуктов.
- Таблицы products и product_profiles связаны 1:1 — каждому продукту соответствует одна запись с дополнительными характеристиками.
- Таблицы recipe_categories и recipes связаны 1:M — одна категория блюда может соответствовать множеству технологических карт.
- Таблицы workshops и recipes связаны 1:M — один производственный цех может быть связан с множеством технологических карт.
- Таблицы recipes и recipe_items связаны 1:M — одна технологическая карта содержит множество позиций состава.
- Таблицы products и recipe_items связаны 1:M — один продукт может использоваться в составе различных технологических карт.
- Таблицы menu_days и menu_day_meta связаны 1:1 — одному дню меню соответствует одна запись с дополнительными параметрами.
- Таблицы menu_days и menu_items связаны 1:M — одно меню на определенную дату содержит множество позиций блюд.
- Таблицы recipes и menu_items связаны 1:M — одна технологическая карта может включаться в меню многократно.
- Таблицы menu_items и menu_item_meta связаны 1:1 — каждой позиции меню соответствует одна запись с дополнительными признаками.
- Таблицы stock_receipts и stock_receipt_items связаны 1:M — один приходный документ содержит множество товарных позиций.
- Таблицы products и stock_receipt_items связаны 1:M — один продукт может встречаться в разных приходных документах.
- Таблицы products и stock_batches связаны 1:M — один продукт может храниться в виде нескольких партий.
- Таблицы stock_requests и stock_request_items связаны 1:M — одна заявка на склад содержит множество позиций продуктов.
- Таблицы products и stock_request_items связаны 1:M — один продукт может входить в состав различных заявок на склад.

- Таблицы stock_request_items и stock_request_item_fulfillment связаны 1:1 — для каждой позиции заявки хранится одна запись о фактическом исполнении.
- Таблицы stock_requests и stock_request_events связаны 1:M — одна заявка может иметь множество записей об изменении статуса и истории обработки.
- Таблицы stock_requests и stock_issues связаны 1:M — на основании одной заявки может быть оформлено несколько документов расхода.
- Таблицы stock_issues и stock_issue_items связаны 1:M — один документ расхода содержит множество позиций списания.
- Таблицы stock_batches и stock_issue_items связаны 1:M — одна складская партия может участвовать в нескольких операциях списания.
- Таблицы inventory_docs и inventory_items связаны 1:M — один документ инвентаризации содержит множество позиций.
- Таблицы products и inventory_items связаны 1:M — один продукт может участвовать в различных документах инвентаризации.
- Таблица веб_users в текущей реализации хранит сведения о пользователях и их ролях, однако не имеет прямых внешних ключей на другие таблицы. Привязка пользователя к подразделению выполняется через поле enterprise_id, которое хранится как самостоятельный атрибут.

Приведенные связи показывают, что база данных MealSync построена по реляционному принципу с четким разделением сущностей по функциональным областям и с поддержкой устойчивых зависимостей между ними. Это обеспечивает целостность хранения информации и позволяет использовать единый набор данных в модулях технологических карт, меню, складского учета и инвентаризации.

Разработанная структура БД полностью поддерживает функциональные требования приложения: формирование технологических карт, управление их составом, учёт изменений, отображение статистических данных и авторизацию пользователей. Представленная ER-диаграмма наглядно демонстрирует взаимосвязи между таблицами и подтверждает корректность выбранной модели хранения данных. Благодаря такой организации база данных может эффективно масштабироваться и расширяться по мере развития программного средства.

4.4.2 Алгоритмы реализации используемых математических моделей

В программном средстве MealSync математические модели, приведенные в подразделе 2.1.4.2, используются как основа для автоматизации расчетов, выполняемых при работе с технологическими картами, меню, складскими документами и результатами

инвентаризации. Их применение позволяет перевести формализованные зависимости предметной области в последовательность вычислительных операций, выполняемых системой без участия пользователя. За счет этого обеспечивается единообразие расчетов, уменьшается вероятность ошибок, связанных с ручной обработкой данных, и повышается достоверность итоговых показателей, используемых в производственном и учетном контурах.

С программной точки зрения каждая математическая зависимость реализуется как определенный алгоритм обработки входных данных. При этом часть формул применяется непосредственно в расчетных сервисах, где из исходных параметров вычисляются итоговые показатели, а часть используется в виде производных или агрегированных зависимостей на уровне аналитики и формирования документов.

Особенность реализации заключается в том, что одни и те же промежуточные величины используются сразу в расчетных контурах. Масса нетто ингредиента применяется при определении пищевой ценности блюда, расчете его себестоимости и вычислении потребности в сырье для меню. Аналогично данные о фактическом и учетном количестве продукта используются как для фиксации результатов инвентаризации, так и для формирования аналитических показателей по отклонениям. Такое повторное использование промежуточных величин делает систему внутренне согласованной и позволяет исключить расхождение между результатами различных расчетных модулей.

Далее в разделе рассматриваются основные алгоритмы, реализующие используемые математические модели, а именно расчеты показателей технологической карты, потребности в сырье по меню, а также складских и инвентаризационных показателей.

Первый блок расчетов связан с обработкой технологической карты и охватывает формулы (1), (2), (4)–(10). Для каждого ингредиента система получает массу брутто и процент потерь, после чего рассчитывает массу нетто по формуле (1). Абсолютная величина потерь по формуле (2) определяется как разность между массой брутто и массой нетто и используется как производный показатель. После этого на основе массы нетто вычисляется вклад ингредиента в общую калорийность, белки, жиры и углеводы по формулам (4)–(7), а также стоимость данного ингредиента по формуле (9). Полная пищевая ценность и себестоимость блюда по формулам (7) и (10) получаются суммированием значений по всем ингредиентам рецептуры. Нормативный выход блюда по формуле (3) в текущем проекте хранится как значение поля portion_grams технологической карты, а энергетическая ценность по формуле (8) представлена через суммирование справочных значений калорийности ингредиентов, заданных в карточках продуктов.

```python
gross = _to_float(item.gross_grams)
loss = _to_float(item.loss_percent)
net = max(gross * (1 - loss / 100.0), 0.0)

gross_total += gross
net_total += net

coeff = net / 100.0
kcal = _to_float(product.calories) * coeff
protein = _to_float(product.protein) * coeff
fat = _to_float(product.fat) * coeff
carbs = _to_float(product.carbs) * coeff
cost = _to_float(product.price_per_kg) * (net / 1000.0)

total_kcal += kcal
total_protein += protein
total_fat += fat
total_carbs += carbs
total_cost += cost
```

Рисунок 4.4.2.1 – Фрагмент программной реализации расчетов технологической карты

Последовательность выполнения указанных действий представлена на рисунке 4.4.2.2.

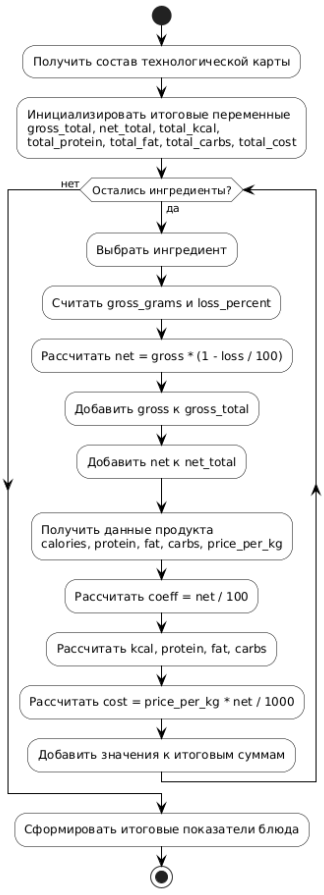


Рисунок 4.4.2.2 – Диаграмма активности алгоритма расчета показателей технологической карты

Второй блок расчетов связан с формированием меню и определением потребности в сырье. Здесь используются формулы (11) и (12). В системе для каждой позиции меню хранятся количество порций и расчетные показатели блюда, поэтому общая стоимость меню по формуле (11) может быть получена как агрегированный показатель на основе себестоимости блюда и числа порций. Формула (12), определяющая потребность в сырье, реализована непосредственно: для каждого блюда система получает число порций, затем по всем ингредиентам рассчитывает массу нетто и умножает ее на количество порций, после чего суммирует результат по каждому продукту.

```python
planned_need: defaultdict[int, float] = defaultdict(float)
for row in rows:
    portions = max(0, int(recipe_portions.get(int(row.recipe_id), 0)))
    gross = _to_float(row.gross_grams)
    loss = _to_float(row.loss_percent)
    net = max(gross * (1 - loss / 100.0), 0.0)
    planned_need[int(row.product_id)] += net * portions
```

Рисунок 4.4.2.3 – Фрагмент программной реализации расчета потребности в сырье

Логика этого расчета показана на рисунке 4.4.2.4.

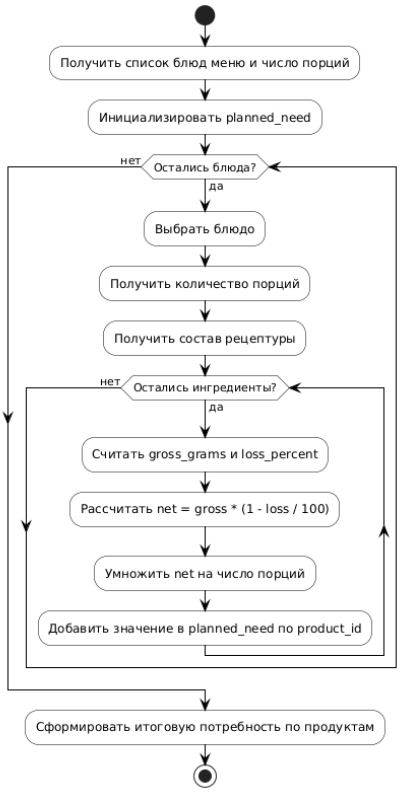


Рисунок 4.4.2.4 – Диаграмма активности алгоритма расчета потребности в сырье по меню

Третий блок расчетов относится к складскому контуру и аналитике. Здесь используются формулы (13)–(17). Балансовая формула движения запасов (13) реализуется через изменение остатка qty_balance при операциях прихода и расхода и последующее агрегирование остатка по партиям. Формула абсолютного отклонения (14) применяется при анализе результатов инвентаризации и вычисляется как разность между фактическим и учетным количеством. Формула относительного отклонения (15) в текущей версии не вынесена в отдельную функцию, однако может быть получена на основе уже рассчитанных величин qty_fact, qty_system и diff. Формула коэффициента исполнения заявки (16) также не оформлена отдельной функцией, но в системе хранятся qty_requested и qty_issued, а их сравнение используется при определении полного или частичного исполнения заявки. Формула средней энергетической ценности (17) относится к аналитическим расчетам и может быть получена на основе уже сформированных показателей calories для отдельных блюд.

```python
batch.qty_balance = float(batch.qty_balance or 0) + float(item.qty)
batch.qty_balance = float(batch.qty_balance or 0) - float(item.qty)
```

```python
"diff": round(float((row.qty_fact or 0.0) - (row.qty_system or 0.0)), 3)
```

```python
requested = float(item.qty or 0)
new_total = float(fulfill.qty_issued or 0) + float(payload.qty)
fulfill.qty_issued = min(new_total, requested)
```

Рисунок 4.4.2.5 – Фрагменты программной реализации складских и инвентаризационных расчетов

Последовательность выполнения складских и инвентаризационных расчетов представлена на рисунке 4.4.2.6.

Приведенные алгоритмы показывают, что математические модели из подраздела 2.1.4.2 используются в проекте как практическая основа для вычислительной логики системы. Часть формул реализована напрямую в расчетных сервисах, а часть используется как производная аналитическая зависимость на основе уже рассчитанных показателей. Это обеспечивает совпадение между математическим описанием предметной области и фактической программной реализацией ключевых расчетов.

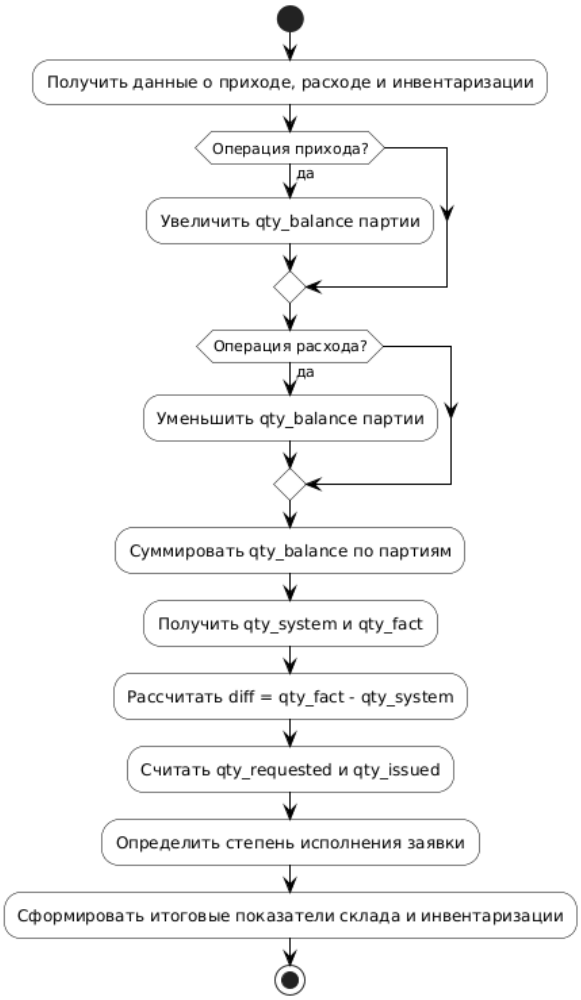


Рисунок 4.4.2.6 – Диаграмма активности алгоритма расчета складских остатков и инвентаризационных отклонений

4.4.3 Алгоритмы применения методов графического анализа данных

В программном средстве MealSync графический анализ применяется для представления результатов работы модуля отчётности руководителя. С позиции программиста процесс формирования отчётов разделён между серверной частью FastAPI и веб-интерфейсом.

Серверная часть отвечает за получение исходных данных из центральной базы данных PostgreSQL, их фильтрацию, группировку и подготовку аналитических показателей. Веб-интерфейс получает сформированные результаты через REST API и преобразует их в таблицы, индикаторы и графические представления.

Доступ к отчётности руководителя предоставляется только пользователю с соответствующей ролью. После открытия раздела веб-интерфейс формирует запрос к маршруту /analytics/dashboard. В запрос передаются анализируемый период, идентификатор предприятия и пороговое значение дефицита. Если период не указан, сервер формирует отчёт за последние 30 дней.

Серверный модуль аналитики последовательно выполняет запросы к таблицам продуктов, складских партий, меню, рецептов, выдач, заявок и инвентаризаций. Для обработки данных применяются операции фильтрации, группировки, суммирования, подсчёта записей и сортировки.

В результате сервер формирует единый набор аналитической информации, содержащий:

- текущие складские остатки;
- список дефицитных продуктов;
- плановую потребность и обеспеченность продуктами;
- статистику расхода сырья;
- расхождения инвентаризации;
- наиболее популярные блюда;
- блюда с наибольшей себестоимостью;
- количество заявок по статусам.

Перед отправкой клиентскому приложению структура данных проверяется схемами Pydantic. После проверки результат передаётся веб-интерфейсу в формате JSON.

Веб-интерфейс обрабатывает полученные данные и формирует раздел отчётности руководителя. Для каждого аналитического набора используется отдельный способ графического отображения. Длина горизонтальных индикаторов определяется относительно максимального значения соответствующего набора. Это позволяет визуально сравнивать продукты, блюда и показатели без выполнения дополнительных запросов к серверу.

Отчёт «Дефицит и потребность» формируется на основании текущих складских остатков и рассчитанной потребности меню. В него включаются продукты, количества которых недостаточно для выполнения производственного плана. Веб-интерфейс отображает наименование продукта и величину недостающего количества.

Пример сформированного отчёта представлен на рисунке 4.4.3.1.

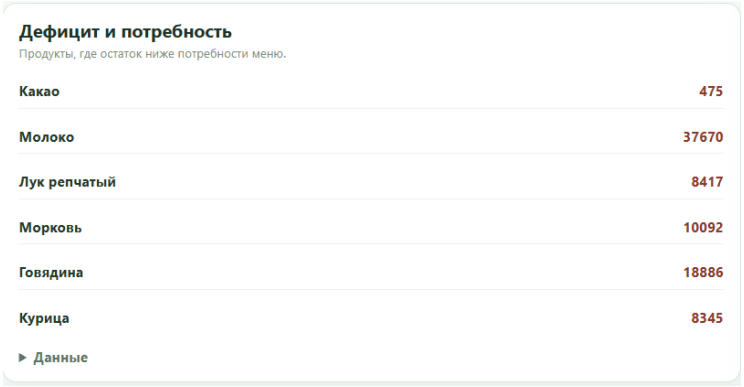


Рисунок 4.4.3.1 — Отчёт «Дефицит и потребность»

Отчёт «Плановая потребность и обеспеченность» содержит рассчитанную потребность в продуктах, текущие остатки и процент покрытия потребности. Перед передачей клиентскому приложению записи сортируются по возрастанию обеспеченности. Поэтому первыми отображаются продукты, запасы которых необходимо пополнить в первую очередь.

Пример отчёта представлен на рисунке 4.4.3.2.

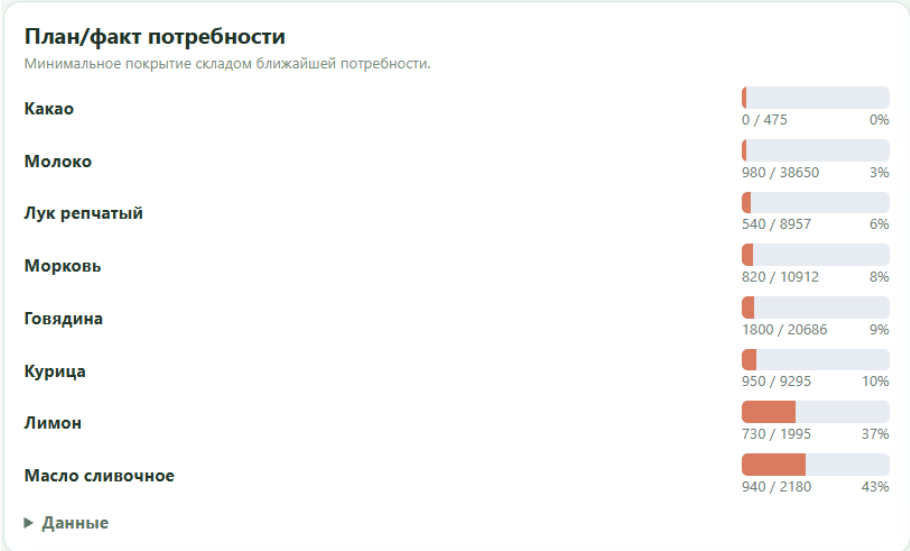


Рисунок 4.4.3.2 — Отчёт «Плановая потребность и обеспеченность продуктами»

Для формирования отчёта «Расхождения инвентаризации» сервер сравнивает системное и фактическое количество продуктов. Для каждой позиции определяется величина отклонения. Полученные значения позволяют выявлять недостачи и излишки.

Пример сформированного отчёта представлен на рисунке 4.4.3.3.

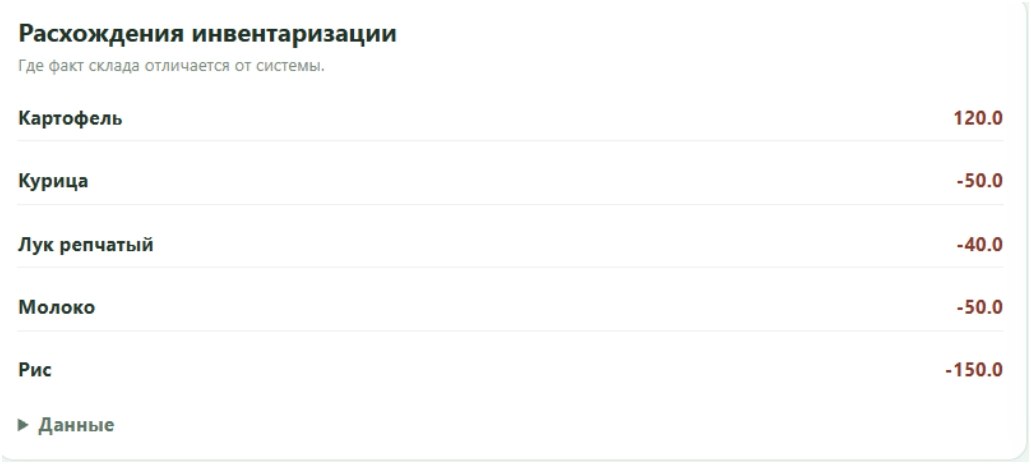


Рисунок 4.4.3.3 — Отчёт «Расхождения инвентаризации»

Отчёт «Популярные блюда» формируется путём группировки позиций меню по рецептам и суммирования количества порций за выбранный период. После сортировки сервер

передаёт веб-интерфейсу блюда с наибольшими значениями. В интерфейсе результат представляется в виде горизонтальных индикаторов.

Пример отчёта представлен на рисунке 4.4.3.4.



Рисунок 4.4.3.4 — Отчёт «Популярные блюда»

Для формирования отчёта «Топ себестоимости блюд» сервер получает состав рецептов и рассчитывает стоимость используемых ингредиентов с учётом их массы и технологических потерь. После расчёта блюда сортируются по убыванию себестоимости.

Пример сформированного отчёта представлен на рисунке 4.4.3.5.

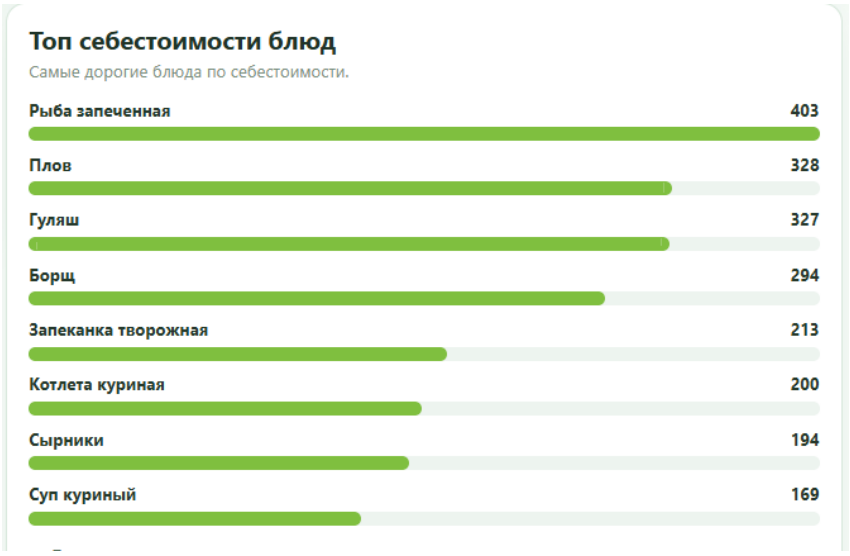


Рисунок 4.4.3.5 — Отчёт «Топ себестоимости блюд»

Отчёт «Сводка по заявкам» формируется путём группировки складских заявок по текущим статусам. Сервер подсчитывает количество заявок для каждого статуса, после чего веб-интерфейс отображает полученное распределение.

Пример отчёта представлен на рисунке 4.4.3.6.



Рисунок 4.4.3.6 — Отчёт «Сводка по заявкам»

Таким образом, графический анализ данных в MealSync реализован как последовательное взаимодействие серверного модуля аналитики и веб-интерфейса. Сервер отвечает за получение и обработку исходной информации, а клиентская часть — за её графическое и табличное представление. Разделение расчётной логики и отображения упрощает сопровождение программного кода и обеспечивает единообразное формирование отчётности руководителя.

4.4.4 Алгоритмы использования технологий передачи данных

В программном средстве передача данных организована по клиент-серверному принципу. Центральным компонентом системы является сервер приложения, к которому подключаются пользовательские клиенты: десктоп-приложение и веб-интерфейс. Такой подход позволяет использовать систему с разных устройств, поскольку основные данные и прикладная логика размещаются на серверной стороне, а пользователи получают доступ к ним через установленное приложение или браузер.

Передача данных между компонентами выполняется через серверное API. Клиентские приложения формируют запросы к серверу, передают необходимые параметры и получают результаты обработки в структурированном виде. В качестве формата обмена используется JSON, что обеспечивает единый способ передачи информации между десктоп-приложением, веб-интерфейсом и серверной частью. Для сетевого взаимодействия применяется протокол HTTP/HTTPS, а серверный адрес задается в конфигурации приложения или используется через опубликованный веб-адрес системы.

В веб-контуре пользователь открывает интерфейс MealSync в браузере с любого устройства, имеющего доступ к серверу. После загрузки страницы браузер выполняет запросы к серверному API, получает данные о меню, технологических картах, заявках, складских остатках и аналитических отчетах, после чего отображает их в пользовательском

интерфейсе. Публичная часть системы также работает через сервер и позволяет просматривать опубликованное меню без обращения к внутренним модулям управления.

В десктоп-контуре приложение может работать в сетевом режиме, отправляя запросы к серверной части, а также использовать локальное хранилище для автономной работы. При подключении к серверу данные передаются через REST API: пользовательские действия в интерфейсе преобразуются в запросы создания, чтения, обновления или обработки записей. Если используется автономный режим, изменения временно сохраняются локально, а затем передаются на сервер при выполнении синхронизации.

Серверная часть выполняет роль посредника между клиентскими приложениями и базой данных. Она принимает входящие запросы, проверяет права пользователя, определяет контекст филиала, выполняет бизнес-логику и обращается к централизованному хранилищу данных. После обработки сервер возвращает клиенту результат: список записей, расчетные показатели, статус операции, данные аналитики или сформированный документ. Благодаря этому клиентские приложения не обращаются к базе данных напрямую, а работают через единый серверный слой.

Схема передачи данных между компонентами программного средства MealSync представлена на рисунке 4.4.4.1.

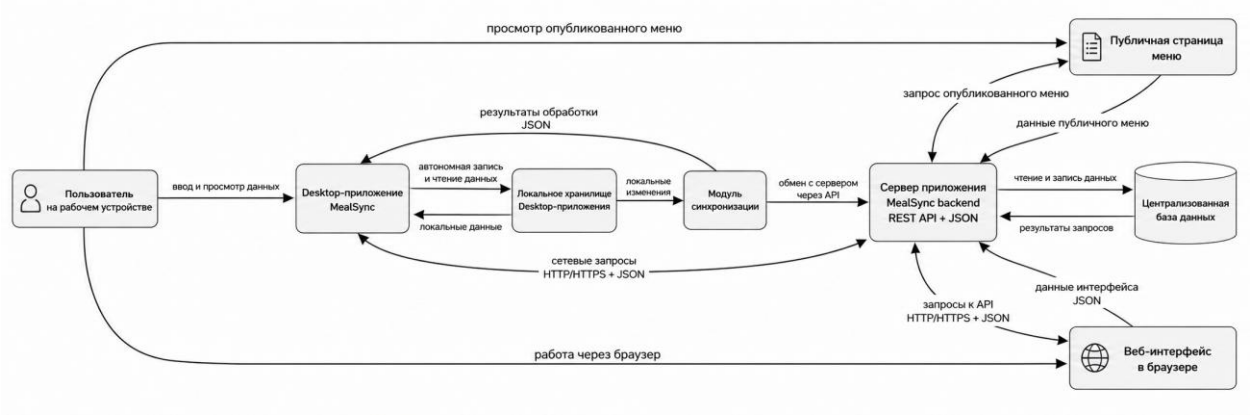


Рисунок 4.4.4.1 – Схема передачи данных между компонентами программного средства MealSync

Как показано на рисунке 4.4.4.1, пользователи взаимодействуют с системой через десктоп-приложение или веб-интерфейс. Оба клиентских компонента передают данные на сервер приложения, где выполняются проверка, обработка и обращение к базе данных. Результаты обработки возвращаются обратно клиенту и отображаются в виде экранных форм, таблиц, отчетов, публичного меню или печатных документов.

Алгоритм передачи данных включает несколько последовательных этапов. Сначала пользователь выполняет действие в интерфейсе: открывает справочник, сохраняет

технологическую карту, формирует меню, создает заявку, проводит складскую операцию или просматривает отчет. Затем клиентский компонент формирует запрос к серверу и передает данные в формате JSON. Сервер принимает запрос, выполняет авторизацию пользователя и проверяет его роль. После этого серверная часть обращается к базе данных, выполняет необходимые операции чтения или записи и формирует ответ. Полученный ответ возвращается клиенту, где преобразуется в удобное для пользователя представление.

Для работы с данными используются разные виды запросов. Запросы чтения применяются при загрузке списков продуктов, технологических карт, меню, складских остатков и аналитических отчетов. Запросы создания используются при добавлении новых продуктов, технологических карт, заявок и складских документов. Запросы обновления применяются при редактировании существующих записей, изменении статусов заявок, публикации меню и фиксации результатов инвентаризации. Отдельные операции, например экспорт документа или публикация меню, реализуются как специализированные серверные действия.

Передача данных между системой и внешними потребителями реализуется через веб-интерфейс и публичные страницы. Внутренние пользователи получают доступ к функциональным разделам после авторизации, а внешние потребители могут просматривать опубликованное меню через браузер. Таким образом, серверная часть обеспечивает как внутренний обмен данными между модулями системы, так и предоставление информации конечным пользователям.

Таким образом, использование серверного API, формата JSON, централизованной базы данных, веб-доступа и механизма синхронизации позволяет организовать устойчивую передачу данных между компонентами MealSync. Такая архитектура обеспечивает возможность работы системы на разных устройствах, поддерживает согласованность информации и позволяет использовать единый источник данных для десктопа- и веб-контуров.

4.5 Описание архитектурного решения

4.5.1 Структурная организация программной системы

Разработанное программное средство MealSync / iRecipe имеет клиент-серверную архитектуру. С позиции программиста система разделена на клиентскую, серверную подсистемы и подсистему хранения данных. Такое разделение обеспечивает независимую разработку компонентов, разграничение ответственности модулей и возможность изменения отдельных частей системы без полной переработки программного средства.

Структурная организация программной системы представлена на диаграмме компонентов UML, приведённой на рисунке 4.5.1.1.

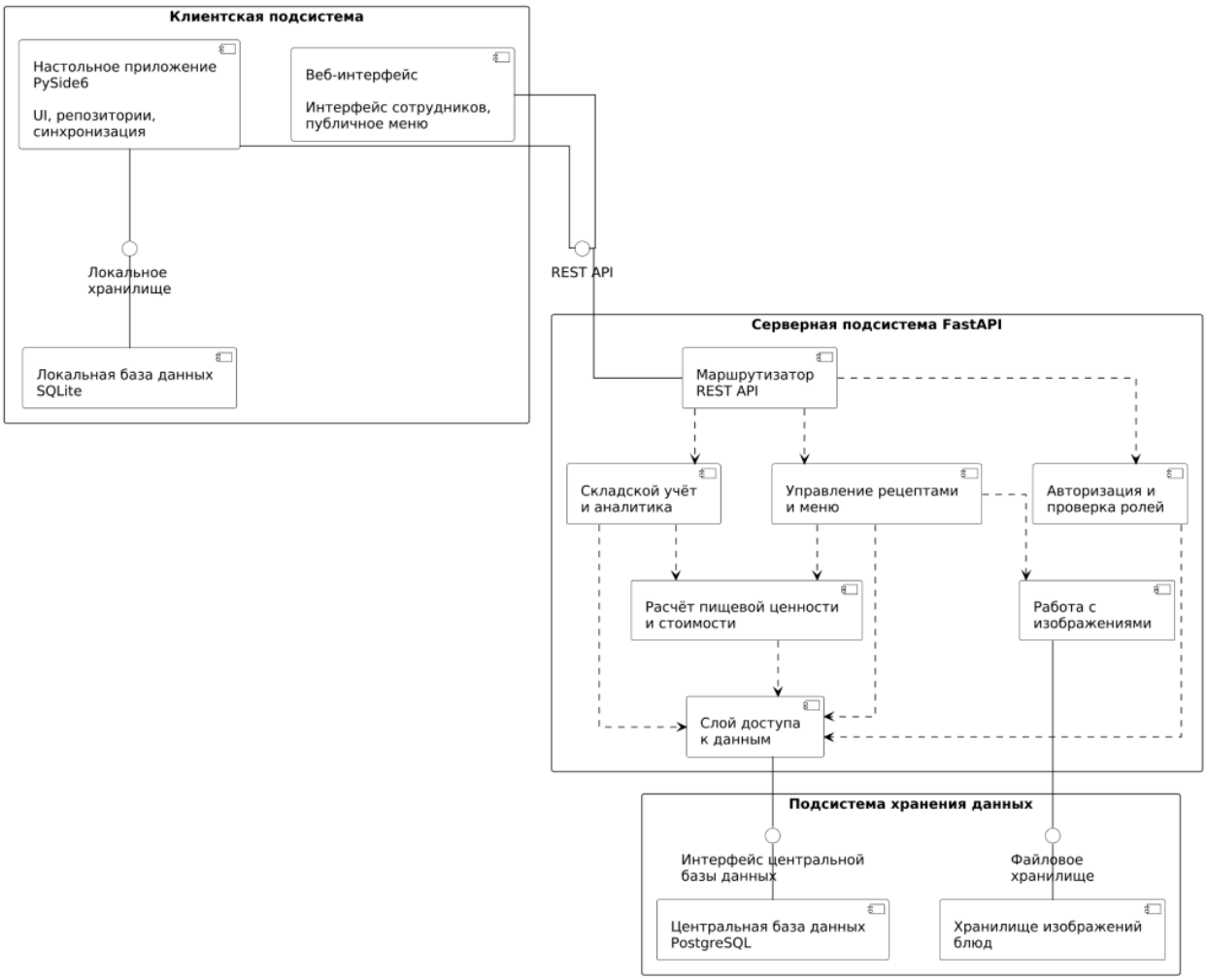


Рисунок 4.5.1.1 – Диаграмма компонентов программного средства

Клиентская подсистема включает настольное приложение PySide6 и веб-интерфейс. Настольное приложение содержит графический интерфейс, репозитории доступа к данным и средства синхронизации. Оно может взаимодействовать с серверной подсистемой через REST API, а также выполнять операции с локальной базой данных SQLite. Наличие локального хранилища позволяет сотруднику продолжать работу при временном отсутствии подключения к серверу. После восстановления соединения локальные изменения могут быть переданы в центральное хранилище.

Веб-интерфейс включает интерфейс сотрудников и публичное меню. Он не обращается к хранилищам данных напрямую. Получение и изменение информации выполняется посредством запросов к REST API. Благодаря этому правила обработки данных и проверки прав доступа остаются централизованными на серверной стороне.

Основой серверной подсистемы является приложение, разработанное с использованием фреймворка FastAPI. Маршрутизатор REST API принимает запросы клиентских приложений и передаёт их соответствующим функциональным компонентам.

Компонент управления рецептами и меню реализует операции с технологическими картами, ингредиентами и меню. Для формирования характеристик блюд он использует компонент расчёта пищевой ценности и стоимости. Компонент складского учёта и аналитики отвечает за обработку поступлений, выдач, остатков, заявок, инвентаризаций и аналитических показателей.

Компонент авторизации и проверки ролей выполняет идентификацию пользователей и ограничивает доступ к операциям системы. Права предоставляются в зависимости от роли сотрудника, например технолога, кладовщика, повара или руководителя.

Компонент работы с изображениями обеспечивает проверку, сохранение и получение фотографий блюд. Изображения размещаются в отдельном файловом хранилище, поэтому двоичные файлы не сохраняются непосредственно в центральной базе данных.

Взаимодействие серверных компонентов с PostgreSQL выполняется через слой доступа к данным. Данный слой изолирует функциональные компоненты от особенностей физического хранения информации и обеспечивает централизованное выполнение операций чтения и записи. Зависимости между компонентами на диаграмме обозначены пунктирными стрелками, а предоставляемые интерфейсы REST API и хранилищ — круглыми обозначениями и сплошными линиями.

Физическое размещение программных компонентов представлено на диаграмме развёртывания UML, приведённой на рисунке 4.5.1.2.

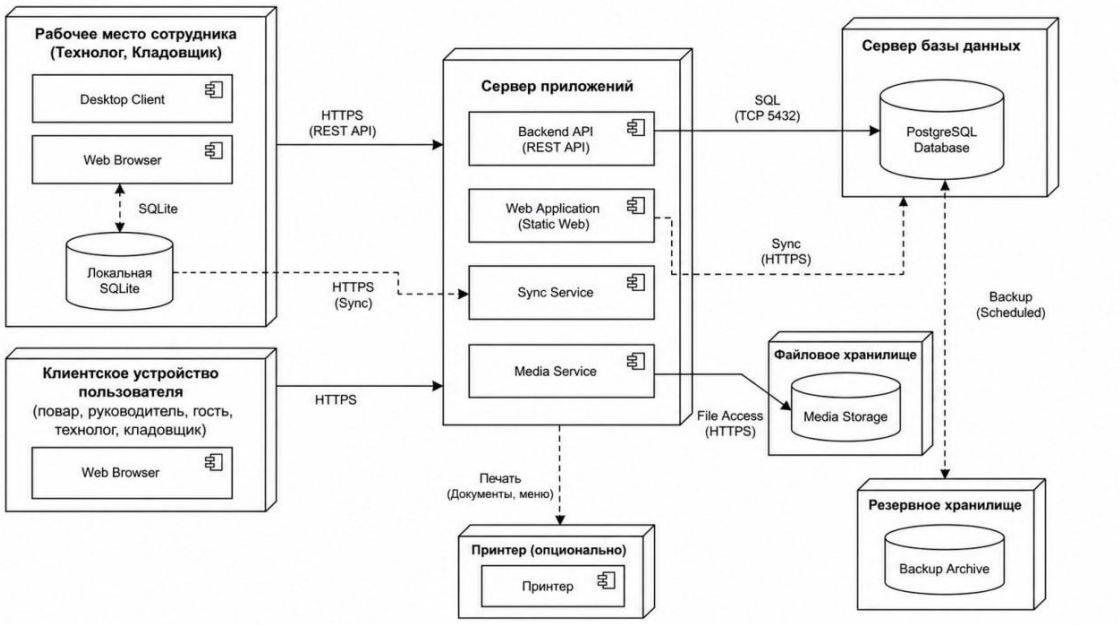


Рисунок 4.5.1.2 – Диаграмма развертывания программного средства

На рабочем месте сотрудника размещается настольный клиент и веб-браузер. Настольное приложение предназначено преимущественно для технолога и кладовщика. На рабочем месте также располагается локальная база данных SQLite, которая используется настольным клиентом для локального хранения информации. Обмен данными с сервером приложений выполняется по протоколу HTTPS посредством REST API.

Пользователи, которым не требуется установка настольного клиента, получают доступ к системе через веб-браузер. К ним относятся повар, руководитель, гость, технолог и кладовщик. Клиентское устройство взаимодействует с сервером приложений по протоколу HTTPS.

На сервере приложений развёрнуты серверный REST API, статическое веб-приложение, сервис синхронизации и сервис обработки изображений. Серверный REST API выполняет основную бизнес-логику и предоставляет клиентам единый программный интерфейс. Статическое веб-приложение содержит файлы веб-интерфейса, загружаемые браузером. Сервис синхронизации обеспечивает обмен данными между локальным хранилищем настольного приложения и центральной базой данных. Сервис обработки изображений обеспечивает доступ к фотографиям блюд.

Центральная база данных PostgreSQL размещается на отдельном сервере базы данных. Сервер приложений взаимодействует с ней посредством SQL-соединения по протоколу TCP через порт 5432. Выделение базы данных в отдельный узел позволяет централизовать хранение информации и независимо управлять производительностью и защитой данных.

Фотографии блюд размещаются в файловом хранилище. Доступ к ним выполняется через сервер приложений.

Диаграмма развёртывания также предусматривает возможность подключения принтера к серверу приложений. Принтер может использоваться для печати документов, технологических карт и меню, однако его наличие не является обязательным для работы системы.

Таким образом, структурная организация программного средства обеспечивает разделение пользовательского интерфейса, бизнес-логики и хранения данных. Клиентские компоненты взаимодействуют с сервером через REST API, функциональные серверные модули используют единый слой доступа к данным, а физические хранилища размещаются независимо от клиентских приложений. Принятая архитектура упрощает сопровождение программного кода, расширение функциональности и развёртывание системы на различных рабочих местах.

4.5.2 Архитектура программного кода

В программном средстве MealSync архитектура программного кода организована по модульному принципу. Кодовая база разделена на несколько логических частей, каждая из которых отвечает за отдельный участок работы системы: пользовательский интерфейс, обработку данных, выполнение расчетов, синхронизацию, серверные запросы, работу с базой данных и формирование файлов. Такое разделение позволяет упростить сопровождение проекта, снизить связанность между отдельными элементами программы и обеспечить возможность дальнейшего расширения функциональности без полной переработки структуры приложения.

Так как система включает как десктоп-приложение, так и серверную часть, структура кода рассматривается отдельно для каждого из этих контуров. Десктоп-часть отвечает за взаимодействие пользователя с приложением, локальное хранение данных и синхронизацию с сервером. Серверная часть отвечает за прием и обработку запросов, выполнение бизнес-логики, доступ к централизованной базе данных и предоставление данных веб-интерфейсу. Для наглядного представления архитектуры программного кода в данном разделе используются диаграммы классов UML, отражающие основные классы и связи между ними.

Диаграмма классов десктоп-части программного средства MealSync представлена на рисунке 4.5.2.1.

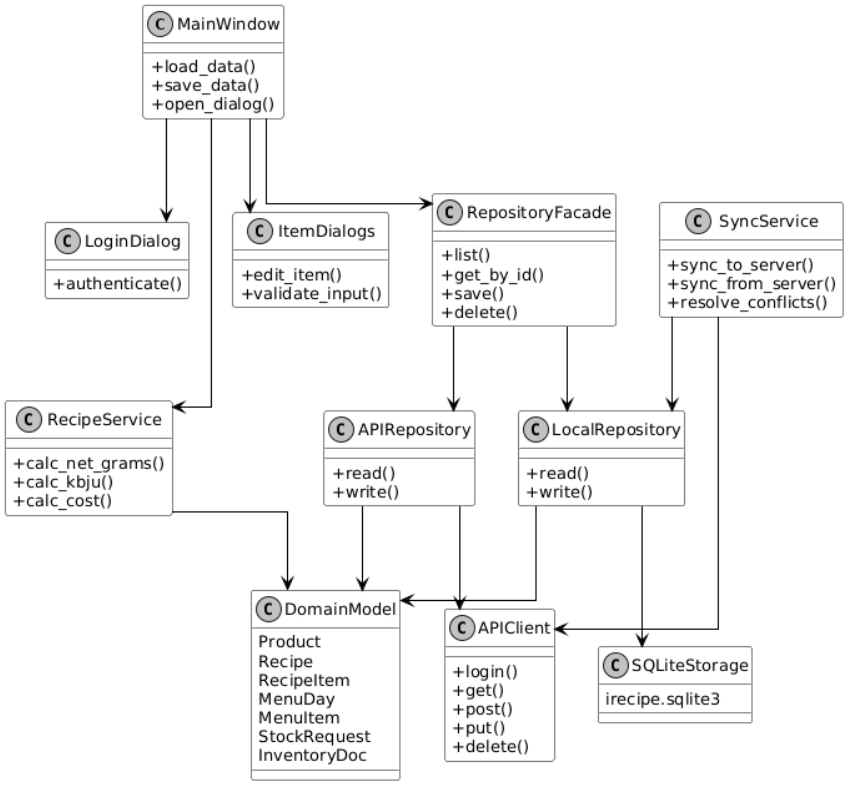


Рисунок 4.5.2.1 – Диаграмма классов десктоп-части программного средства

На рисунке 4.5.2.1 показана структура классов, используемых в десктоп-приложении. Центральным классом пользовательского интерфейса является MainWindow, который отвечает за загрузку данных, сохранение изменений и открытие диалоговых окон. Для авторизации пользователя используется класс LoginDialog, а редактирование отдельных объектов выполняется через класс ItemDialogs. Расчетная логика технологических карт вынесена в класс RecipeService, где реализованы методы расчета массы нетто, КБЖУ и себестоимости.

Для доступа к данным используется класс RepositoryFacade, который выступает единым интерфейсом между пользовательским интерфейсом и источниками данных. В зависимости от режима работы он может обращаться либо к LocalRepository, либо к APIRepository. Класс LocalRepository выполняет чтение и запись данных в локальное хранилище SQLiteStorage, а класс APIRepository использует APIClient для передачи данных в серверную часть. Класс APIClient содержит методы авторизации и выполнения сетевых запросов get, post, put, delete.

Отдельно выделен класс SyncService, отвечающий за синхронизацию локальных и серверных данных. Он передает локальные изменения на сервер, получает серверные изменения и выполняет разрешение конфликтов. Класс DomainModel на диаграмме объединяет основные сущности предметной области, используемые в десктоп-части: Product, Recipe, RecipeItem, MenuDay, MenuItem, StockRequest, InventoryDoc. Такое объединение позволяет показать связь с моделями данных без перегрузки диаграммы отдельными классами каждой сущности.

Диаграмма классов серверной части программного средства MealSync представлена на рисунке 4.5.2.2.

На рисунке 4.5.2.2 показана структура классов серверной части. Верхний уровень представлен классом FastAPIApp, который отвечает за подключение маршрутов, статических файлов и выполнение действий при запуске приложения. Запросы пользователей обрабатываются через APIRouter, который объединяет группы маршрутов: авторизацию, продукты, технологические карты, меню, склад, аналитику и публичную часть.

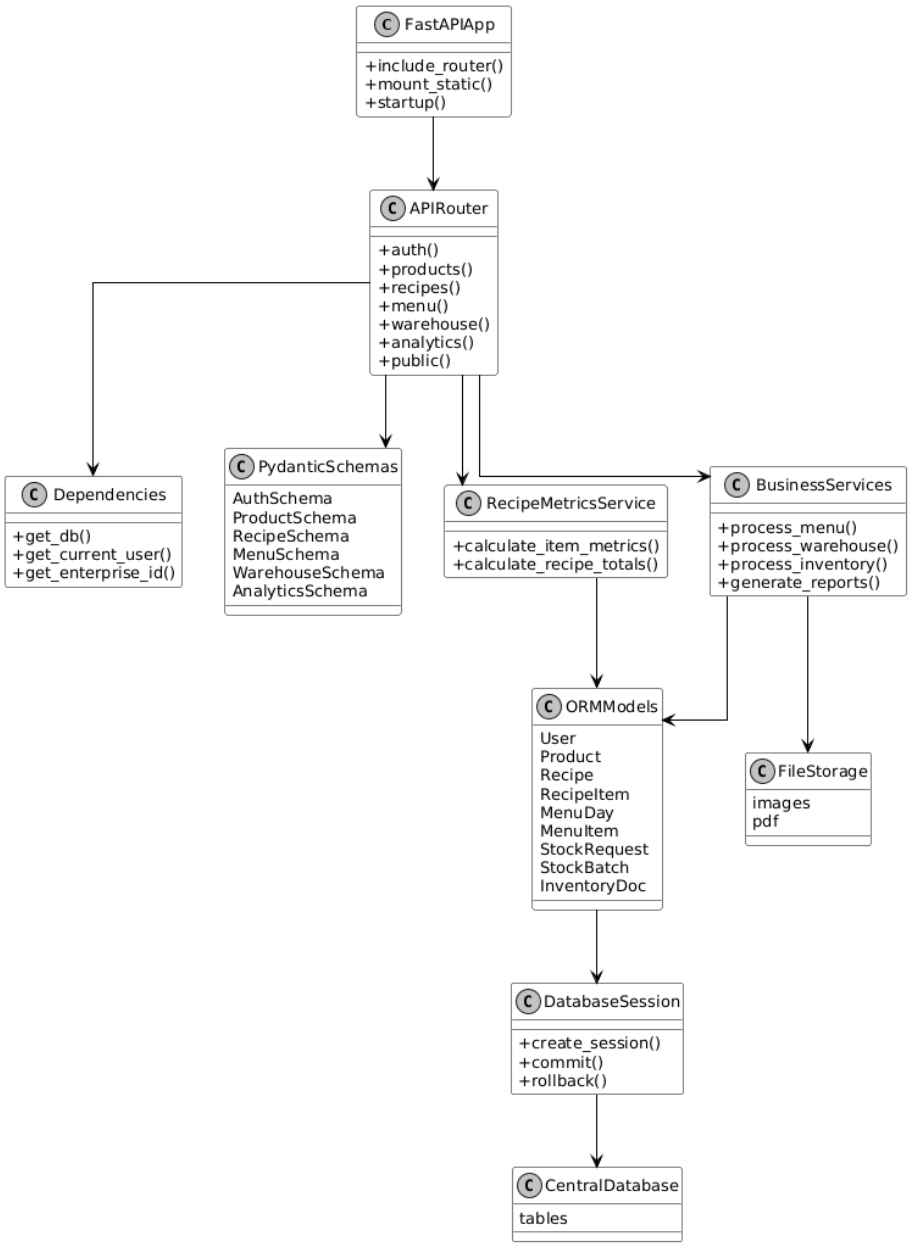


Рисунок 4.5.2.2 – Диаграмма классов серверной части программного средства

Класс Dependencies содержит зависимости, используемые обработчиками запросов. К ним относятся получение сессии базы данных, определение текущего пользователя и определение контекста филиала. Класс PydanticSchemas объединяет схемы данных, которые применяются для проверки входных данных и формирования выходных ответов API. В него входят схемы авторизации, продуктов, технологических карт, меню, склада и аналитики.

Расчетная логика серверной части представлена классом RecipeMetricsService. Он выполняет расчет показателей технологических карт, включая метрики отдельных ингредиентов и итоговые показатели блюда. Более общая бизнес-логика объединена в

классе BusinessServices, который отвечает за обработку меню, складских операций, инвентаризации и формирование отчетов.

Класс ORMModels представляет основные модели базы данных: User, Product, Recipe, RecipeItem, MenuDay, MenuItem, StockRequest, StockBatch, InventoryDoc. Эти модели используются серверными сервисами для чтения и изменения данных. Работа с подключением к базе данных вынесена в класс DatabaseSession, который создает сессию, фиксирует изменения и выполняет откат транзакций при необходимости. Данные сохраняются в CentralDatabase, представляющей централизованную базу данных системы. Для изображений и сформированных PDF-документов используется класс FileStorage.

Таким образом, диаграммы классов показывают, что программный код MealSync построен с разделением ответственности между пользовательским интерфейсом, сервисной логикой, доступом к данным, синхронизацией и серверной обработкой запросов. Такое построение упрощает сопровождение проекта, позволяет независимо развивать десктоп- и серверную части, а также обеспечивает единый подход к обработке данных во всех функциональных модулях системы.

5 ТЕСТИРОВАНИЕ И ОПТИМИЗАЦИЯ

5.1 План тестирования

Тестирование проекта MealSync представляет собой важный этап разработки, направленный на проверку стабильной работы десктоп-клиента, backend API, веб-интерфейса, базы данных и механизмов синхронизации. С учетом архитектуры системы был разработан комплексный план тестирования, включающий следующие направления:

1. Функциональное тестирование включает проверку:
   - авторизации пользователей и разграничения ролей;
   - ведения справочников, продуктов и технологических карт;
   - формирования, утверждения и публикации меню;
   - обработки складских заявок и учета остатков;
   - работы публичного меню;
   - синхронизации данных между локальной SQLite-базой и серверным API.

Особое внимание уделяется корректности расчетов КБЖУ, себестоимости блюд, списанию продуктов со склада и работе системы в разных режимах хранения данных.

2. Тестирование пользовательского интерфейса направлено на проверку:
   - корректности отображения десктоп-интерфейса PySide6;
   - удобства навигации между модулями технолога, кладовщика, повара и руководителя;
   - корректной работы веб-страниц;
   - отображения публичного меню на разных устройствах;
   - читаемости таблиц, форм и карточек блюд.

3. Нагрузочное тестирование направлено на оценку производительности системы:
   - при массовом обращении к публичному меню;
   - при одновременной работе сотрудников с API;
   - при большом количестве технологических карт, продуктов и заявок;
   - при синхронизации локальных и серверных данных;
   - при формировании отчетов и аналитики.

4. Тестирование безопасности включает проверку:
   - авторизации через API;
   - защиты endpoints от несанкционированного доступа;
   - корректной работы ролей пользователей;

- устойчивости к SQL-инъекциям и XSS;
- хранения паролей и работы токенов доступа;
- изоляции данных по enterprise_id

5.2 Функциональное тестирование

Функциональное тестирование направлено на проверку соответствия системы заявленным требованиям представлено в таблице 5.2.1. Оно охватывает основные бизнес-процессы: работу с меню, технологическими картами, продуктами, складскими заявками, публичным меню и синхронизацией данных.

Методы проведения:

- ручное тестирование основных пользовательских сценариев;
- проверка API-запросов;
- тестирование граничных значений;
- проверка сценариев с ошибочными входными данными;
- тестирование работы десктоп-клиента в локальном и API-режимах.

Таблица 5.2.1 – Функциональное тестирование

№	Тестируемый сценарий	Входные данные	Ожидаемый результат	Фактический результат	Статус
1	Авторизация технолога	Логин: technolog, пароль: 1234	Пользователь входит в систему, открываются доступные модули	Успешно	Pass
2	Создание технологической карты	Название блюда, категория, выход, ингредиенты	Техкарта сохраняется, рассчитываются КБЖУ и себестоимость	Успешно	Pass
3	Формирование меню дня	Дата, прием пищи, список блюд	Меню сохраняется и доступно для публикации	Успешно	Pass
4	Создание заявки на склад	Продукты и количества	Заявка создается в статусе черновика и может быть отправлена	Успешно	Pass
5	Просмотр публичного меню	enterprise_id, дата	Пользователь видит опубликованные блюда	Успешно	Pass

По результатам проведения функционального тестирования было выявлено:

- успешно пройдено: 34 теста;
- не пройдено: 3 теста;
- итоговая успешность: 91,9%.

Критические проблемы:

1. Ошибка обновления списка меню после смены предприятия.
2. Некорректная обработка пустого состава технологической карты.

3. Задержка синхронизации при большом количестве локальных изменений.

После доработки логики обновления интерфейса, проверки состава техкарт и обработки конфликтов синхронизации повторное тестирование подтвердило корректную работу основных функций системы.

5.3 Тестирование пользовательского интерфейса

UI-тестирование проверяет удобство, отзывчивость и визуальную корректность интерфейса десктоп- и веб-частей проекта представлено в таблице 5.3.1.

Методы проведения:

- ручная проверка десктоп-интерфейса PySide6;
- проверка веб-интерфейса в Chrome, Edge и Firefox;
- проверка публичного меню на десктоп и мобильных экранах;
- тестирование форм ввода, таблиц, кнопок и фильтров.

Таблица 5.3.1 – UI-тестирование

№	Тестируемый сценарий	Устройство / среда	Ожидаемый результат	Фактический результат	Статус
1	Открытие десктоп-клиента	Windows, PySide6	Главное окно отображается корректно	Успешно	Pass
2	Работа формы авторизации	Десктоп	Ошибки входа отображаются понятно, успешный вход открывает систему	Успешно	Pass
3	Просмотр публичного меню	Chrome, мобильный экран	Меню адаптируется под ширину экрана	Успешно	Pass
4	Работа таблиц продуктов и техкарт	Десктоп	Данные читаемы, строки выбираются корректно	Успешно	Pass
5	Навигация staff-интерфейса	Chrome / Edge	Разделы доступны, действия выполняются без перезагрузки страницы	Успешно	Pass

Основные замечания:

1. На узких экранах таблицы публичного меню требовали дополнительной адаптации.
2. В отдельных формах десктоп-клиента длинные названия блюд выходили за пределы ячеек.
3. При ошибке API веб-интерфейс не всегда показывал понятное сообщение пользователю.

После корректировки CSS, улучшения обработки ошибок и настройки отображения длинных текстов интерфейс стал корректно отображаться на тестовых устройствах и в основных браузерах.

5.4 Нагрузочное тестирование

Нагрузочное тестирование проводилось для проверки стабильности backend API, веб-интерфейса и операций синхронизации при повышенной активности пользователей представлено в таблице 5.4.1.

Методы проведения:

- имитация одновременных запросов к публичному меню;
- проверка времени отклика API;
- тестирование операций создания меню и заявок;
- мониторинг нагрузки на базу данных;
- проверка синхронизации большого объема данных.

Таблица 5.4.1 – Нагрузочное тестирование

№	Тестируемый сценарий	Нагрузка	Ожидаемый результат	Фактический результат	Статус
1	Просмотр публичного меню	300 запросов/мин	Время отклика менее 1 сек	0,7 сек	Pass
2	Авторизация сотрудников	100 входов/мин	Нет зависаний и ошибок сессии	Успешно	Pass
3	Создание складских заявок	150 операций/мин	Заявки сохраняются корректно	Успешно	Pass
4	Массовая синхронизация	1000 записей	Данные передаются без потерь	Успешно	Pass
5	Запрос аналитики	50 запросов/мин	Отчет формируется без ошибки	Успешно	Pass

Ключевые метрики производительности в таблице 5.4.2.

Таблица 5.4.2 – Результат проверки производительности

Параметр	Ожидаемое значение	Фактический результат
Время отклика публичного меню	≤ 1 сек	0,7 сек
Время авторизации	≤ 1 сек	0,5 сек
Синхронизация 1000 записей	≤ 10 сек	8,4 сек
Пиковая нагрузка CPU	≤ 75%	63%

Нагрузочное тестирование показало, что система выдерживает плановую нагрузку. Основным узким местом стала синхронизация большого объема локальных изменений, однако после оптимизации запросов и пакетной обработки данных время выполнения было снижено до допустимого уровня.

5.5 Тестирование безопасности

Тестирование безопасности направлено на проверку защиты пользовательских данных, API-методов и разграничения доступа по ролям представлено в таблице 5.5.1.

Методы проведения:

- проверка авторизации и токенов доступа;

- ручное тестирование SQL-инъекций и XSS;
- проверка доступа к API без авторизации;
- проверка ограничения данных по enterprise_id;
- анализ обработки пользовательского ввода.

Таблица 5.5 – Тестирование безопасности

№	Тестируемый сценарий	Метод проверки	Ожидаемый результат	Фактический результат	Статус
1	Доступ к API без авторизации	Запрос без токена	Ошибка доступа	Успешно	Pass
2	SQL-инъекция в форму поиска	Ввод ' OR 1=1 --	Запрос не нарушает работу БД	Успешно	Pass
3	XSS в названии блюда	Ввод <script>alert(1)</script>	Ввод обрабатывается безопасно	Успешно	Pass
4	Проверка ролей	Вход под поваром/кладовщиком/технологом	Доступны только разрешенные разделы	Успешно	Pass
5	Проверка enterprise_id	Запрос данных другого предприятия	Доступ ограничен	Успешно	Pass

Результаты проверки:

- критических уязвимостей не выявлено;
- обнаружены незначительные замечания по валидации отдельных текстовых полей;
- доступ к защищенным API-методам без авторизации заблокирован;
- роли пользователей работают корректно.

Принятые меры:

1. Усилена валидация пользовательского ввода.
2. Проверена корректность разграничения доступа по ролям.
3. Добавлена обработка ошибок авторизации в веб- и десктоп-интерфейсах.
4. Проверена изоляция данных разных предприятий.

5.6 Общие выводы

Система MealSync успешно прошла основные виды тестирования:

- функциональная полнота: 97%;
- UI-совместимость: основные десктоп- и веб-сценарии работают корректно;
- производительность: система выдерживает плановую нагрузку;
- безопасность: доступ к данным защищен авторизацией, ролями и проверкой контекста предприятия;
- синхронизация: локальный и серверный контуры данных работают согласованно.

По итогам тестирования можно сделать вывод, что проект соответствует заявленным требованиям и готов к использованию в условиях столовой или предприятия общественного питания.

6 РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ

Разработанная информационная система MealSync предназначена для автоматизации работы столовой. Система обеспечивает ведение технологических карт, формирование меню, обработку заявок на склад, учёт складских документов, просмотр технологических карт поваром и анализ деятельности предприятия. Программная система размещается на удалённом сервере организации. Для просмотра общедоступного меню пользователю достаточно открыть сайт в браузере. Сотрудники столовой могут работать через личный кабинет на сайте или установить настольное приложение на рабочий компьютер.

Адрес системы предоставляет администратор организации. В настоящем руководстве используется адрес: https://mealsync.example.kz.

Перед использованием системы сотруднику должны быть выданы логин, пароль и роль. В системе предусмотрены следующие роли:

- технолог;
- кладовщик;
- повар;
- руководитель
- гость

Доступные разделы и действия автоматически определяются после авторизации пользователя.

6.1 Вход на сайт

Для начала работы необходимо открыть браузер, ввести адрес MealSync и перейти на сайт. На главной странице отображаются кнопки «Меню для посетителей» и «Вход сотрудников», как показано на рисунке 6.1.1.

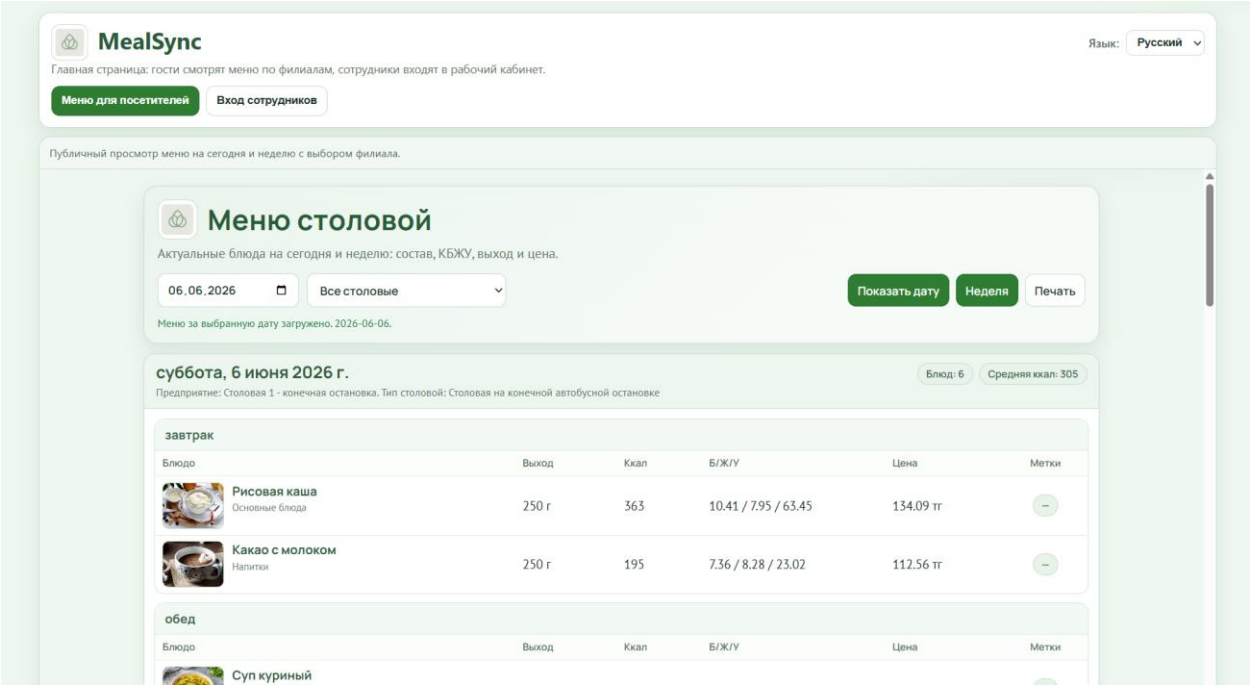


Рисунок 6.1.1 – Главная страница MealSync

Кнопка «Меню для посетителей» предназначена для просмотра опубликованного меню без авторизации. Для перехода в личный кабинет сотруднику необходимо нажать кнопку «Вход сотрудников».

После перехода открывается форма авторизации, представленная на рисунке 6.1.2. В верхней части страницы пользователь может выбрать предприятие и язык интерфейса. В поля «Логин» и «Пароль» необходимо ввести полученные учётные данные, после чего нажать кнопку «Войти».

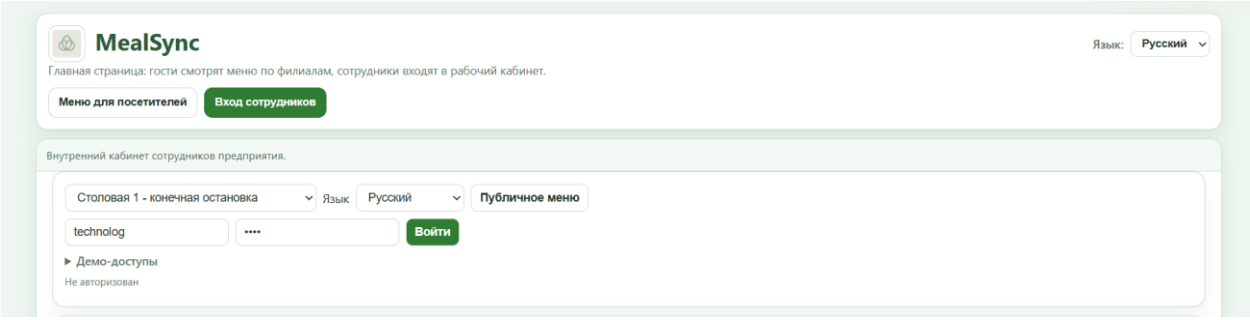


Рисунок 6.1.2 – Форма авторизации сотрудника

После успешной авторизации в верхней части страницы отображаются имя пользователя, его роль и выбранное предприятие. Состав доступных разделов изменяется в зависимости от роли сотрудника.

6.2 Просмотр публичного меню

Гость является неавторизованным пользователем системы MealSync. Для него не создаётся отдельная учётная запись и не предоставляется личный кабинет. Гость может использовать веб-интерфейс только для просмотра опубликованного меню столовой. Работа с настольным приложением для данной роли не предусмотрена.

Для просмотра меню гость должен открыть сайт MealSync и на главной странице будет публичное мню, как показано на рисунке 6.1.1. Вход сотрудника с использованием логина и пароля при этом не требуется.

Для просмотра меню на определённую дату гость должен:

1. Выбрать необходимую столовую в поле выбора предприятия.
2. Указать дату в календаре.
3. Нажать кнопку «Показать дату».

После выполнения указанных действий система отображает опубликованное меню выбранной столовой. Для каждого блюда выводятся его наименование, категория, фотография и количество порций, как показано на рисунке 6.2.1.

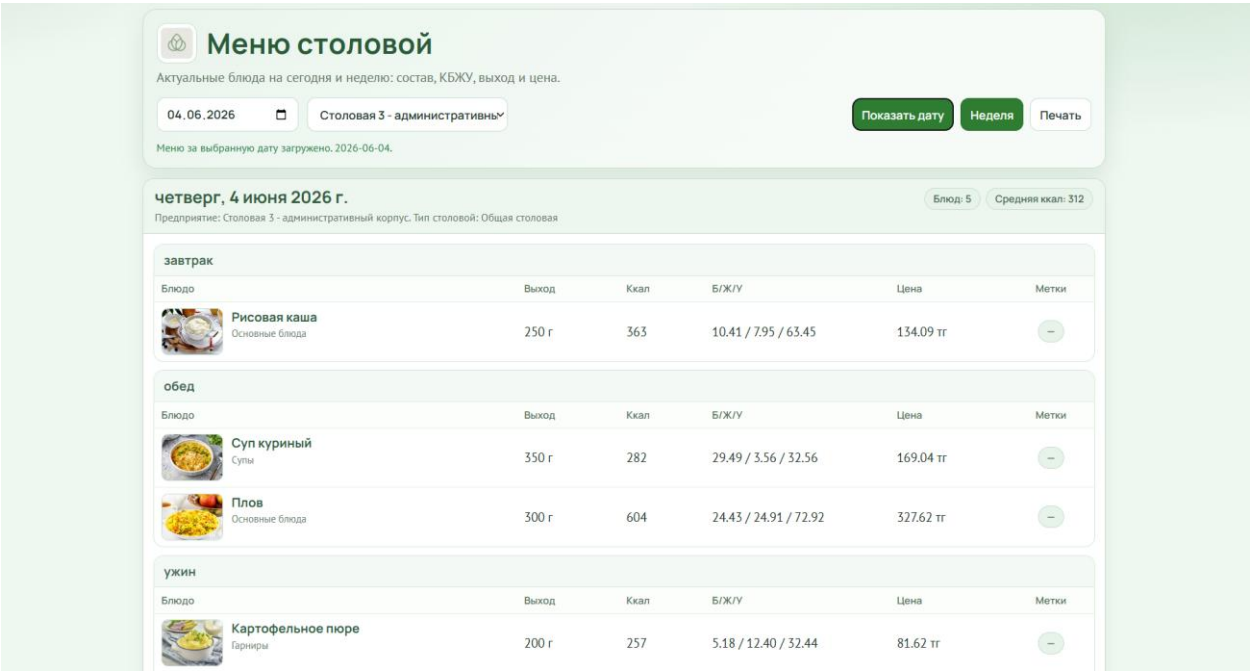


Рисунок 6.2.1 – Просмотр гостем меню на выбранную дату

Для просмотра меню сразу на семь дней необходимо нажать кнопку «Неделя». Система отобразит отдельный блок для каждого дня, содержащего опубликованное меню.

Если на выбранную дату меню ещё не было опубликовано сотрудником столовой, система сообщает об отсутствии доступных данных. В таком случае гость может выбрать другую дату или другую столовую.

Для получения бумажной версии меню необходимо нажать кнопку «Печать». После этого откроется стандартное окно печати браузера, в котором можно выбрать принтер или сохранить меню в виде PDF-файла показано на рисунке 6.2.2.

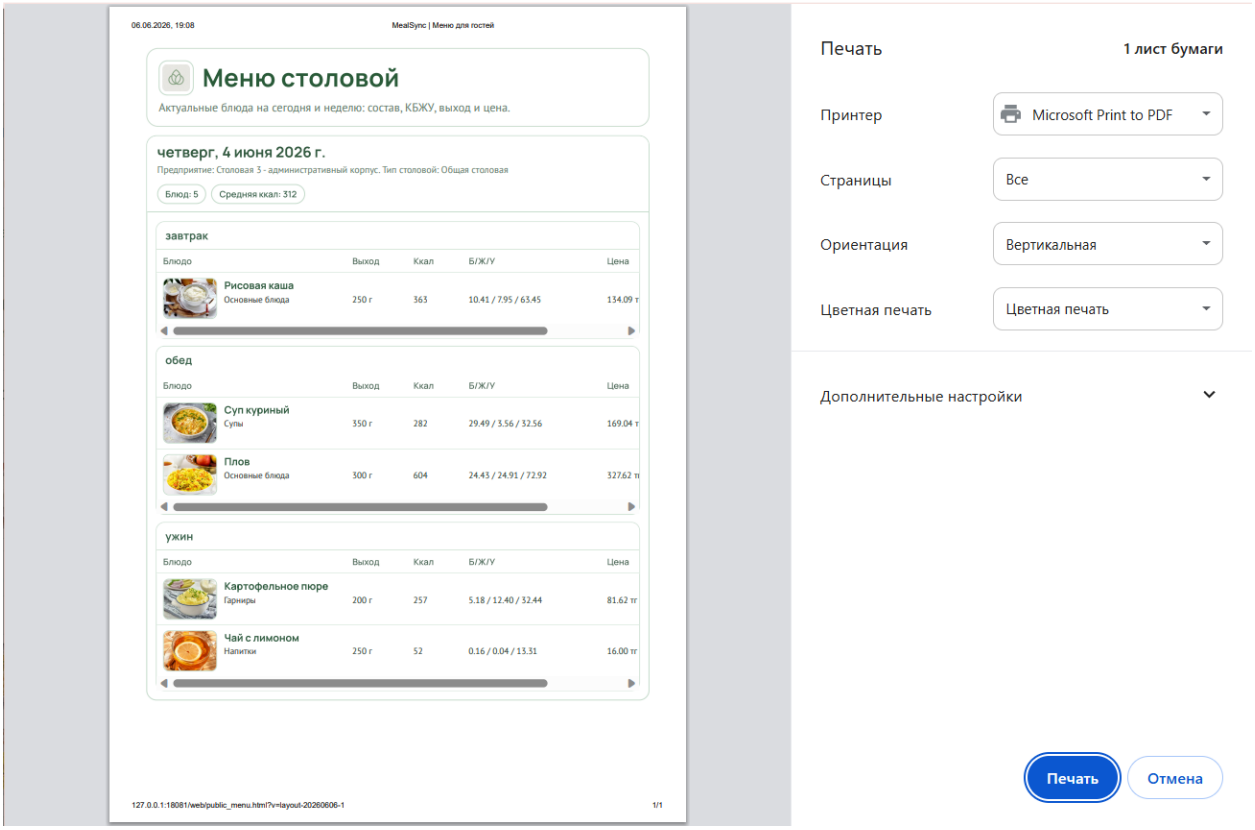


Рисунок 6.2.2 – Просмотр меню в виде PDF-файла

6.3 Установка настольного приложения

Кнопка установки настольного приложения доступна в веб-кабинетах технолога, кладовщика и повара. После авторизации необходимо нажать кнопку «Установить на ПК», расположенную в верхней части личного кабинета, как показано на рисунке 6.5.

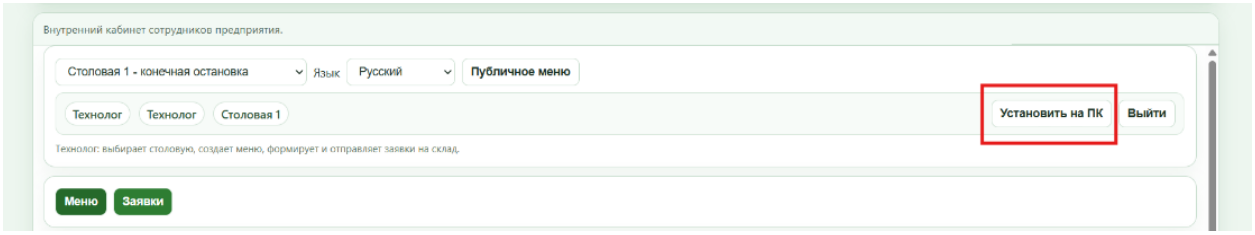


Рисунок 6.3.1 – Кнопка загрузки настольного приложения

После нажатия кнопки браузер загружает архив MealSyncDesktop.zip представлен на рисунке 6.3.2. Загруженный архив необходимо открыть и извлечь его содержимое в отдельную папку на компьютере.

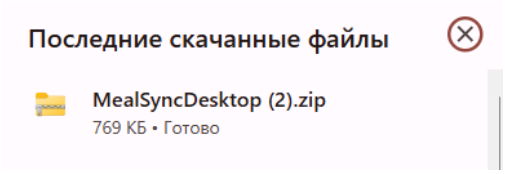


Рисунок 6.3.2 – Архив в загрузках

Для запуска приложения следует открыть извлечённую папку и запустить файл `start_mealsync_desktop.bat`. Окно входа в настольное приложение показано на рисунке 6.3.3.

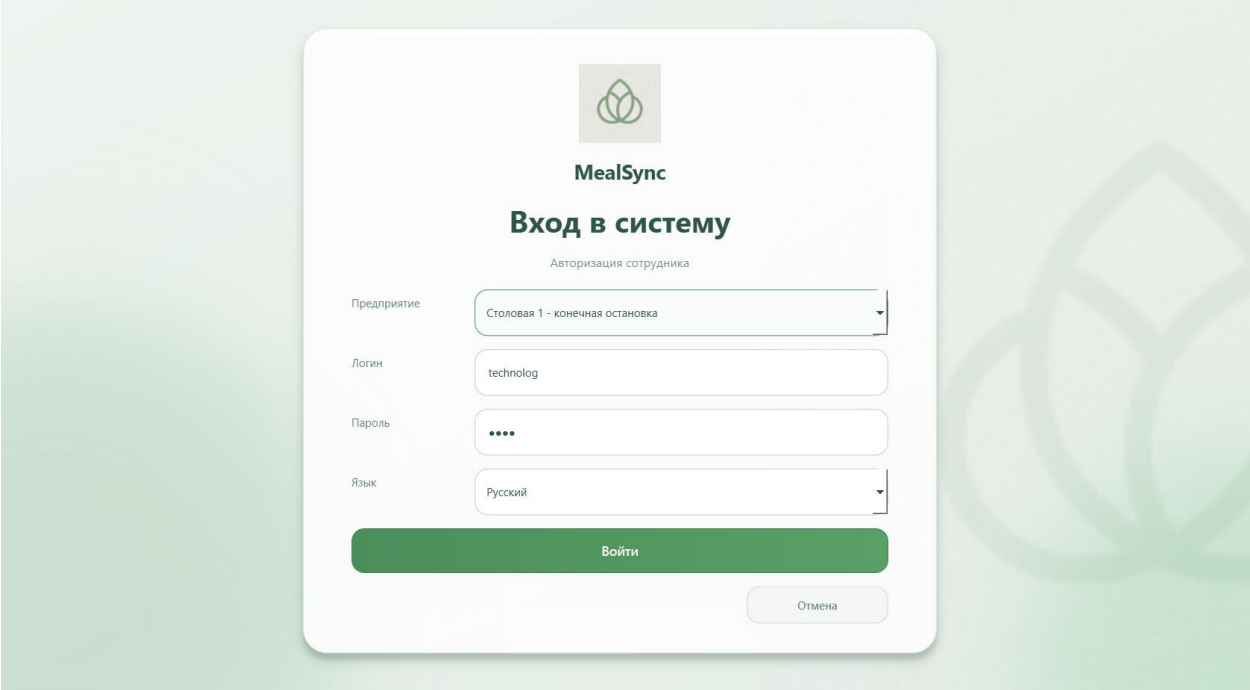


Рисунок 6.3.3 – Авторизация в настольном приложении MealSync

В окне входа необходимо выбрать предприятие, ввести логин и пароль, выбрать язык интерфейса и нажать кнопку «Войти». Настольное приложение подключается к серверу организации, поэтому изменения, внесённые в приложении, становятся доступны пользователям веб-версии

6.4 Работа технолога

Технолог может использовать веб-кабинет для оперативной работы с меню и заявками, а настольное приложение – для управления технологическими картами, продуктами, справочниками и меню.

6.4.1 Работа технолога в веб-интерфейсе

После входа технологу доступны разделы «Меню» и «Заявки». В разделе «Меню» отображается список дней меню и блюда выбранного дня, как показано на рисунке 6.7. В таблице блюд выводятся фотографии, наименования, приёмы пищи, количество порций, выход, калорийность и цена.

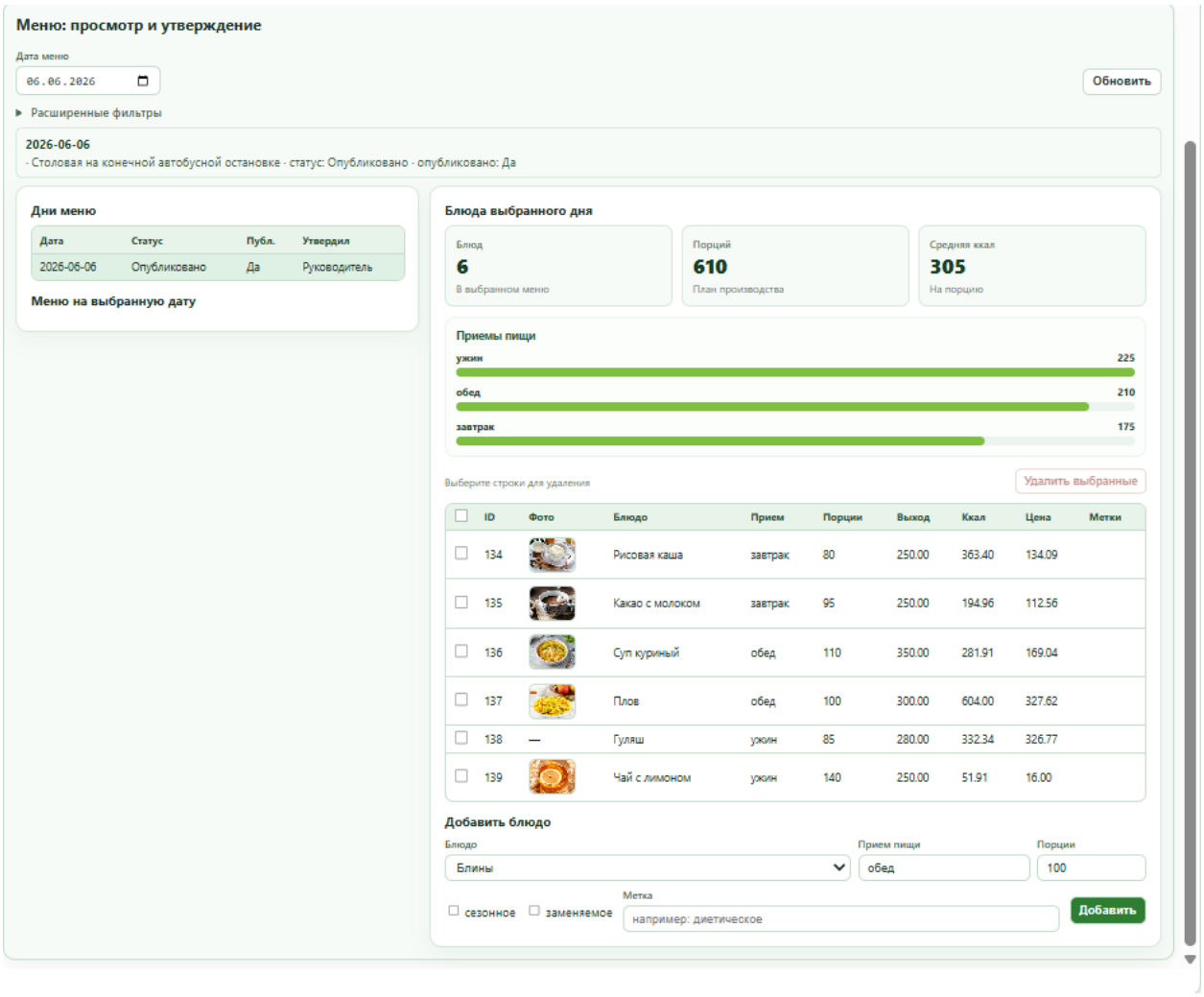


Рисунок 6.4.1.1 – Раздел «Меню» в веб-кабинете технолога

Для создания меню технолог должен:

1. Выбрать дату меню.
2. Нажать кнопку «Создать».
3. Выбрать блюдо из списка.
4. Указать приём пищи и количество порций.
5. Нажать кнопку «Добавить».

После формирования меню технолог может выбрать день и нажать кнопку «Утвердить». Для отображения меню посетителям необходимо нажать кнопку «Опубликовать».

Раздел «Заявки» используется технологом для формирования потребности в продуктах. Интерфейс раздела представлен на рисунке 6.4.1.2, создание новой заявки на рисунке 6.4.1.3.

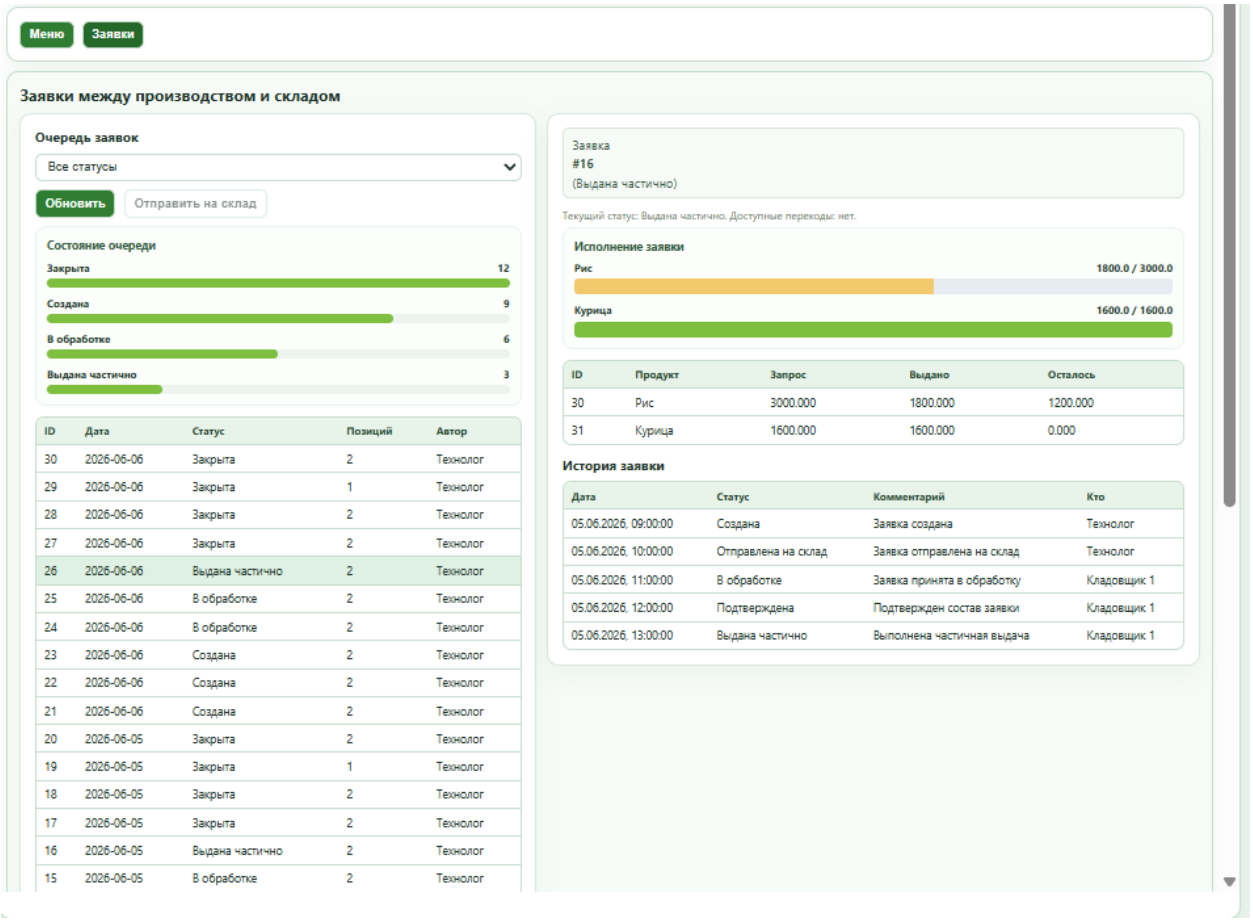


Рисунок 6.4.1.2 – Заявки между производством и складом у технолога в веб-интерфейсе

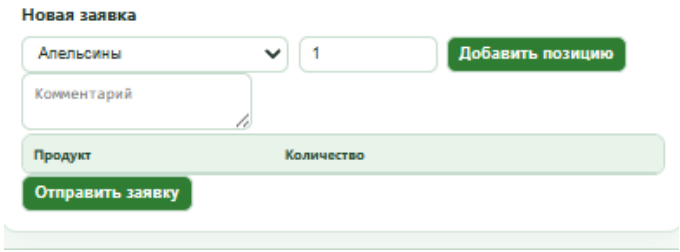


Рисунок 6.4.1.3 – Формирование новой заявки технологом в веб-интерфейсе

Для создания заявки необходимо выбрать продукт, указать требуемое количество и нажать кнопку «Добавить позицию». После добавления всех необходимых продуктов следует нажать кнопку «Отправить заявку». Созданная заявка передаётся кладовщику.

6.4.2 Работа технолога в настольном приложении

Главное окно настольного приложения технолога показано на рисунке 6.4.2.1. Технологу доступны разделы «Техкарты», «Продукты», «Меню», «Заявки» и «Справочники».

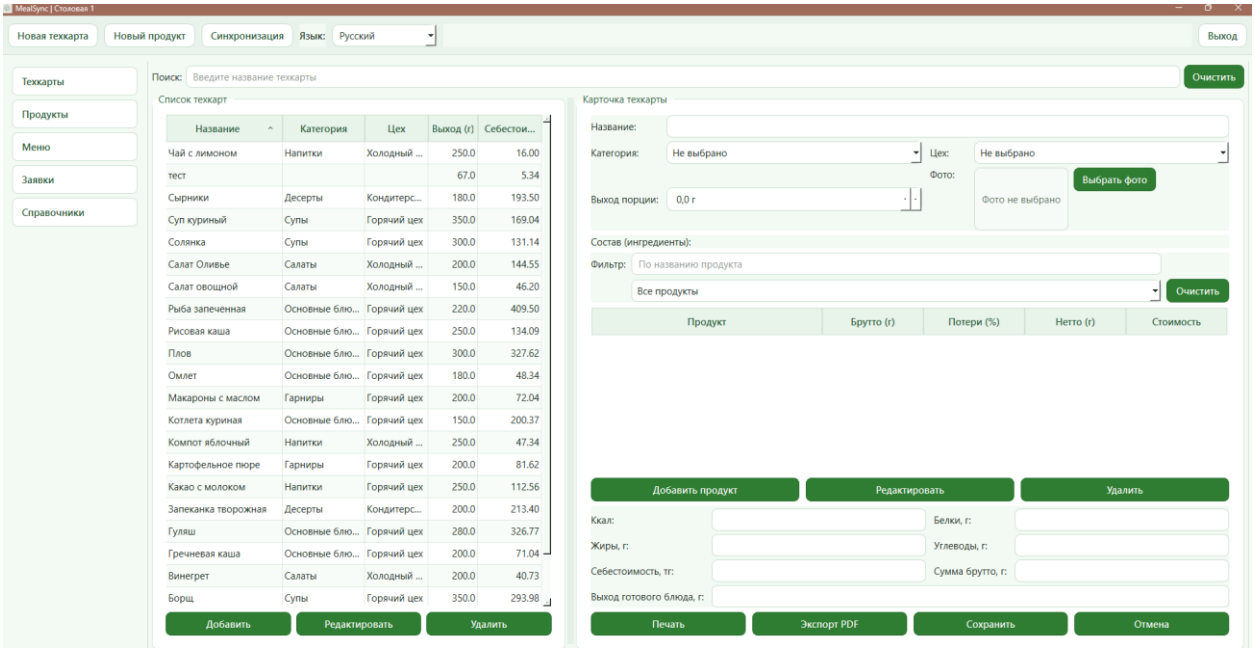


Рисунок 6.4.2.1 – Настольный интерфейс технолога

В разделе «Техкарты» технолог может создавать, редактировать и удалять технологические карты блюд. Для создания новой карты необходимо нажать кнопку «Новая техкарта» или «Добавить», заполнить сведения о блюде, добавить продукты, указать технологию приготовления и выбрать фотографию блюда. После заполнения данных следует нажать кнопку «Сохранить».

Экран редактирования технологической карты представлен на рисунке 6.4.2.2.

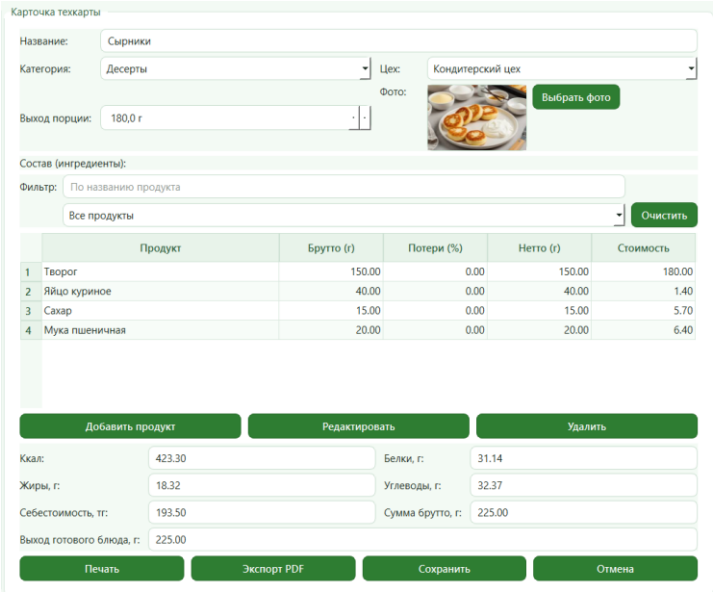
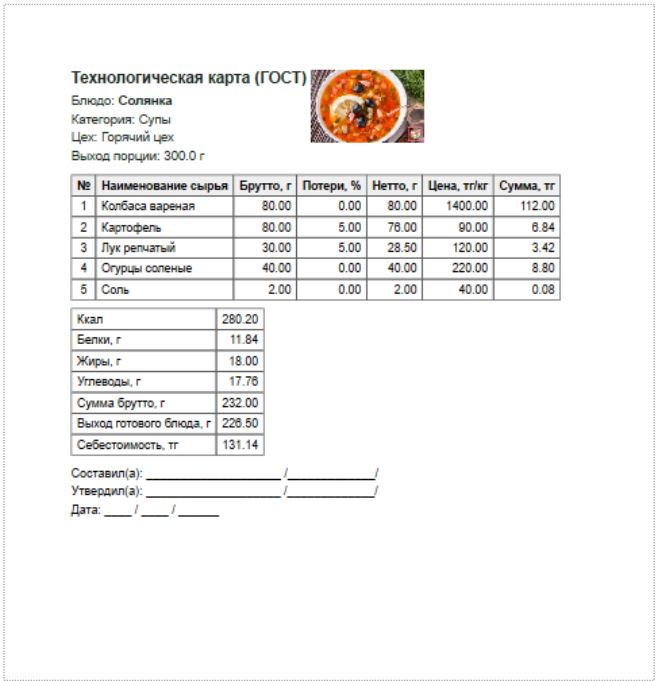


Рисунок 6.4.2.2 – Редактирование технологической карты блюда

В настольном приложении предусмотрен экспорт данных в форматы PDF и Excel. Экспорт в PDF используется для формирования готовых к печати технологических карт, меню и актов инвентаризации с сохранением установленного оформления документа на рисунке 6.4.2.3.



Технологическая карта (ГОСТ)

Блюдо: **Солянка**
Категория: Супы
Цех: Горячий цех
Выход порции: 300.0 г

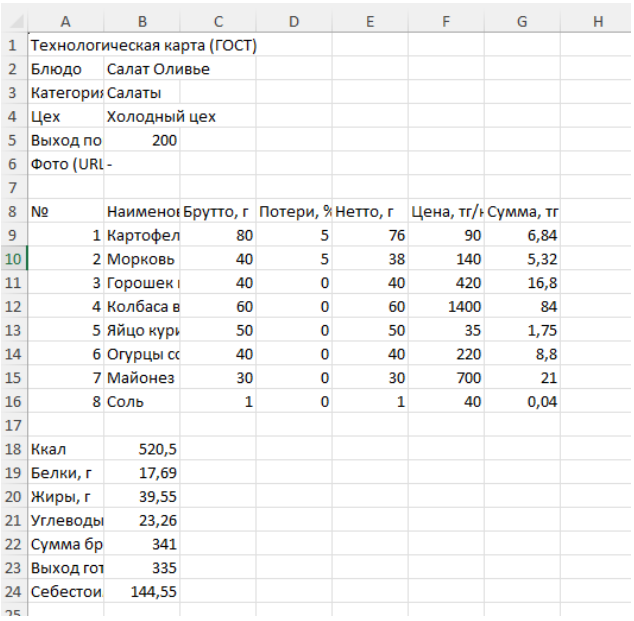
№	Наименование сырья	Брутто, г	Потери, %	Нетто, г	Цена, тг/кг	Сумма, тг
1	Колбаса вареная	80.00	0.00	80.00	1400.00	112.00
2	Картофель	80.00	5.00	76.00	90.00	6.84
3	Лук репчатый	30.00	5.00	28.50	120.00	3.42
4	Огурцы соленые	40.00	0.00	40.00	220.00	8.80
5	Соль	2.00	0.00	2.00	40.00	0.08

Ккал	280.20
Белки, г	11.84
Жиры, г	18.00
Углеводы, г	17.76
Сумма брутто, г	232.00
Выход готового блюда, г	226.50
Себестоимость, тг	131.14

Составил(а): _____/_____/
Утвердил(а): _____/_____/
Дата: ____/____/_____

Рисунок 6.4.2.3 – Экспортированная технологическая карта в PDF

Экспорт в Excel позволяет сохранить табличные данные для последующего анализа, редактирования и подготовки отчётности на рисунке 6.4.2.4. Для выполнения экспорта пользователь выбирает необходимый документ, нажимает соответствующую кнопку экспорта и указывает место сохранения файла.



	A	B	C	D	E	F	G	H
1	Технологическая карта (ГОСТ)							
2	Блюдо	Салат Оливье						
3	Категория	Салаты						
4	Цех	Холодный цех						
5	Выход по	200						
6	Фото (URL	-						
7								
8	№	Наименов	Брутто, г	Потери, %	Нетто, г	Цена, тг/к	Сумма, тг	
9	1	Картофел	80	5	76	90	6,84	
10	2	Морковь	40	5	38	140	5,32	
11	3	Горошек	40	0	40	420	16,8	
12	4	Колбаса в	60	0	60	1400	84	
13	5	Яйцо кури	50	0	50	35	1,75	
14	6	Огурцы со	40	0	40	220	8,8	
15	7	Майонез	30	0	30	700	21	
16	8	Соль	1	0	1	40	0,04	
17								
18	Ккал	520,5						
19	Белки, г	17,69						
20	Жиры, г	39,55						
21	Углеводы	23,26						
22	Сумма бр	341						
23	Выход гот	335						
24	Себестои	144,55						

Рисунок 6.4.2.4 – Экспортированная технологическая карта в Excel

В разделе «Продукты» технолог может добавлять продукты и указывать их категорию, единицу измерения, пищевую ценность и цену. Интерфейс управления продуктами показан на рисунке 6.4.2.5.

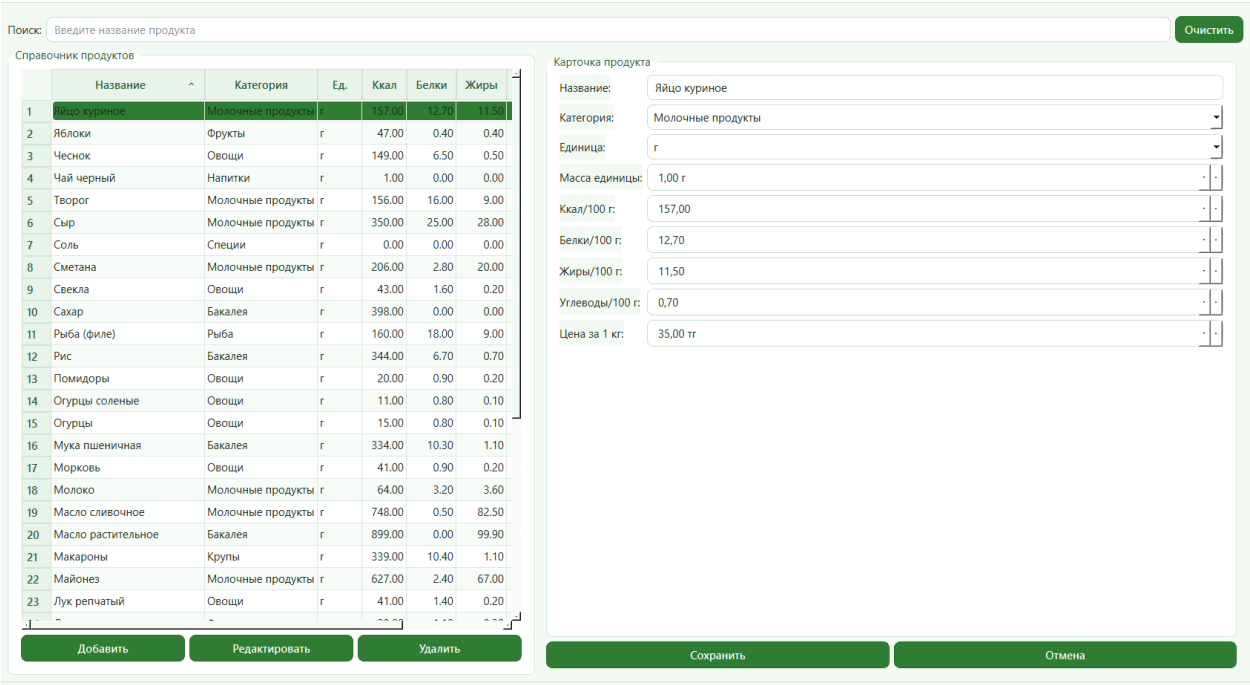


Рисунок 6.4.2.5 – Управление перечнем продуктов

В разделе «Меню» технолог выбирает дату, создаёт меню на день, добавляет блюда и проверяет наличие требуемых продуктов на складе. Раздел «Меню» представлен на рисунке 6.4.2.6.

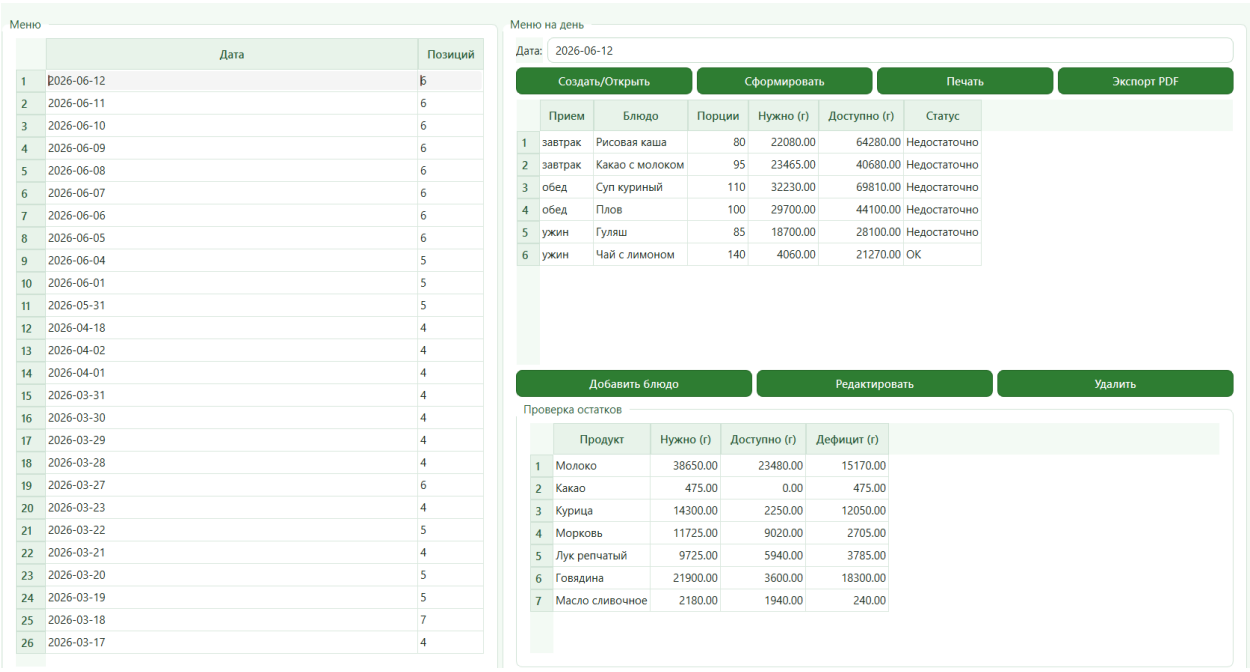


Рисунок 6.4.2.6 – Формирование меню в настольном приложении

В разделе «Заявки» технолог создаёт заявки на выдачу продуктов со склада на рисунке 6.4.2.7. После сохранения заявка становится доступна кладовщику в веб и настольном интерфейсах.

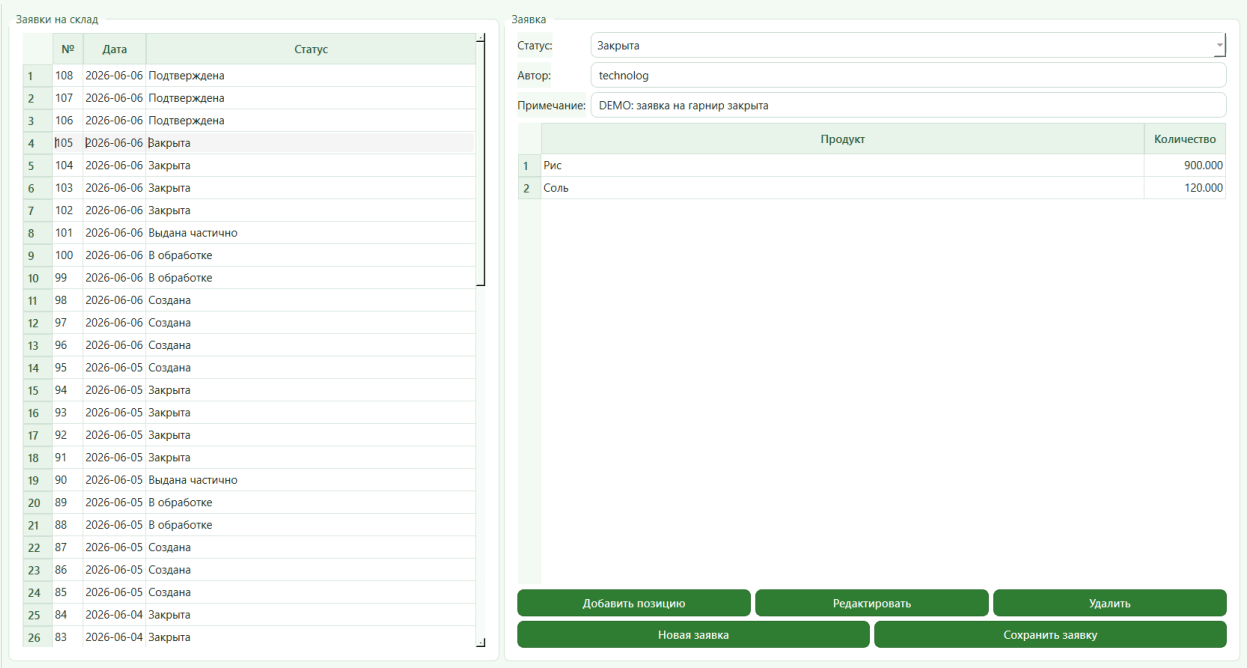


Рисунок 6.4.2.7 – Создание заявки в настольном приложении у технолога

В разделе «Справочники» технолог может управлять единицами измерения, категориями продуктов, категориями блюд и производственными цехами на рисунке 6.4.1.8. Данные справочников используются при заполнении карточек продуктов и создании технологических карт, обеспечивая единообразие вводимой информации.

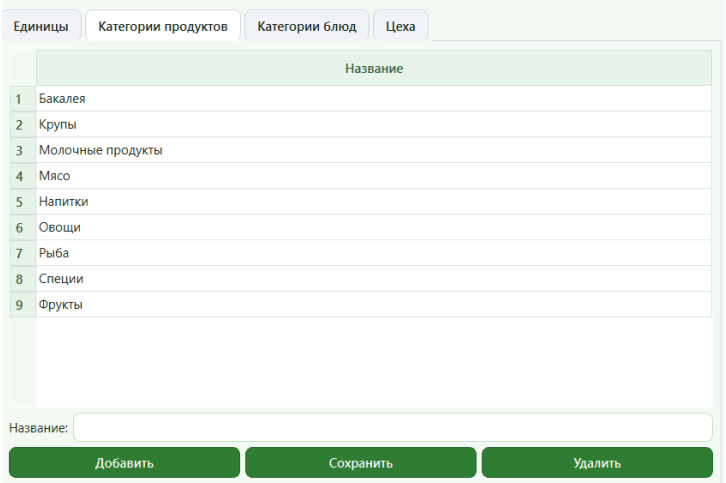


Рисунок 6.4.8 – Справочники

Для завершения работы в настольном приложении необходимо нажать кнопку «Выход», расположенную на панели управления. После нажатия текущий сеанс завершается, и пользователь возвращается к окну авторизации.

6.5 Работа кладовщика

Кладовщик использует веб-интерфейс для обработки поступивших заявок. Настольное приложение предоставляет дополнительные функции складского учёта: регистрацию приходов, расходов и инвентаризаций.

6.5.1 Работа кладовщика в веб-интерфейсе

После авторизации кладовщику доступен раздел «Заявки». В левой части страницы находится очередь заявок, а в правой части отображаются позиции выбранной заявки, история изменений и элементы управления выдачей, как показано на рисунке 6.5.1.1

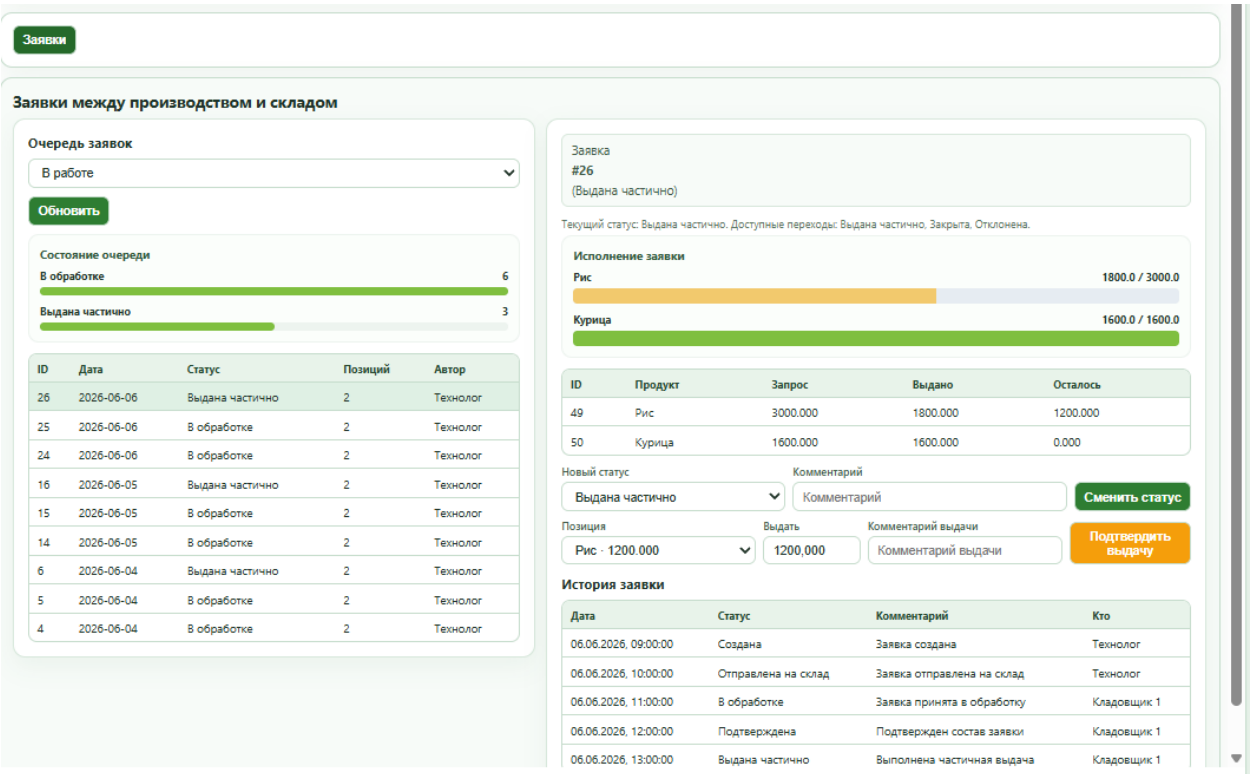


Рисунок 6.5.1.1 – Обработка заявки кладовщиком в веб-интерфейсе

Для обработки заявки кладовщик должен:

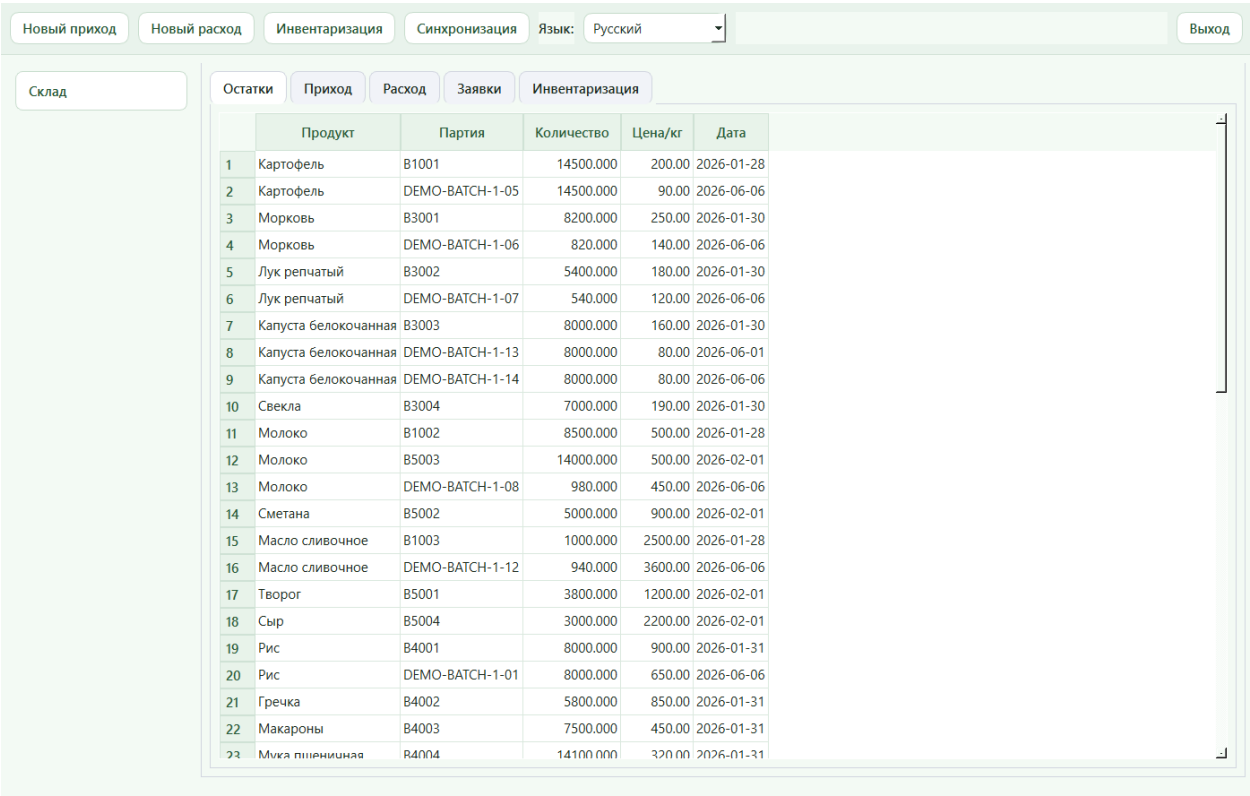
1. Выбрать заявку из очереди.
2. Проверить перечень и требуемое количество продуктов.
3. Выбрать новый статус заявки.
4. При необходимости указать комментарий.
5. Нажать кнопку «Сменить статус».
6. Выбрать выдаваемую позицию и указать количество.
7. Нажать кнопку «Подтвердить выдачу».

После подтверждения выдачи система уменьшает складские остатки и сохраняет операцию в истории заявки.

6.5.2 Работа кладовщика в настольном приложении

Главное окно настольного приложения кладовщика показано на рисунке. Кладовщику доступен раздел «Склад», содержащий вкладки «Остатки», «Приход», «Расход», «Заявки» и «Инвентаризация».

На вкладке «Остатки» отображается текущее количество продуктов по складским партиям на рисунке 6.5.2.1.



	Продукт	Партия	Количество	Цена/кг	Дата
1	Картофель	B1001	14500.000	200.00	2026-01-28
2	Картофель	DEMO-BATCH-1-05	14500.000	90.00	2026-06-06
3	Морковь	B3001	8200.000	250.00	2026-01-30
4	Морковь	DEMO-BATCH-1-06	820.000	140.00	2026-06-06
5	Лук репчатый	B3002	5400.000	180.00	2026-01-30
6	Лук репчатый	DEMO-BATCH-1-07	540.000	120.00	2026-06-06
7	Капуста белокочанная	B3003	8000.000	160.00	2026-01-30
8	Капуста белокочанная	DEMO-BATCH-1-13	8000.000	80.00	2026-06-01
9	Капуста белокочанная	DEMO-BATCH-1-14	8000.000	80.00	2026-06-06
10	Свекла	B3004	7000.000	190.00	2026-01-30
11	Молоко	B1002	8500.000	500.00	2026-01-28
12	Молоко	B5003	14000.000	500.00	2026-02-01
13	Молоко	DEMO-BATCH-1-08	980.000	450.00	2026-06-06
14	Сметана	B5002	5000.000	900.00	2026-02-01
15	Масло сливочное	B1003	1000.000	2500.00	2026-01-28
16	Масло сливочное	DEMO-BATCH-1-12	940.000	3600.00	2026-06-06
17	Творог	B5001	3800.000	1200.00	2026-02-01
18	Сыр	B5004	3000.000	2200.00	2026-02-01
19	Рис	B4001	8000.000	900.00	2026-01-31
20	Рис	DEMO-BATCH-1-01	8000.000	650.00	2026-06-06
21	Гречка	B4002	5800.000	850.00	2026-01-31
22	Макароны	B4003	7500.000	450.00	2026-01-31
23	Мука пшеничная	B4004	14100.000	320.00	2026-01-31

Рисунок 6.5.2.1 – Остатки продуктов

Для регистрации поступления продуктов кладовщик открывает вкладку «Приход», нажимает кнопку «Новый приход», указывает номер документа, дату, поставщика и добавляет поступившие продукты. После заполнения документа необходимо нажать кнопку «Сохранить приход». Пример документа поступления представлен на рисунке 6.5.2.2.

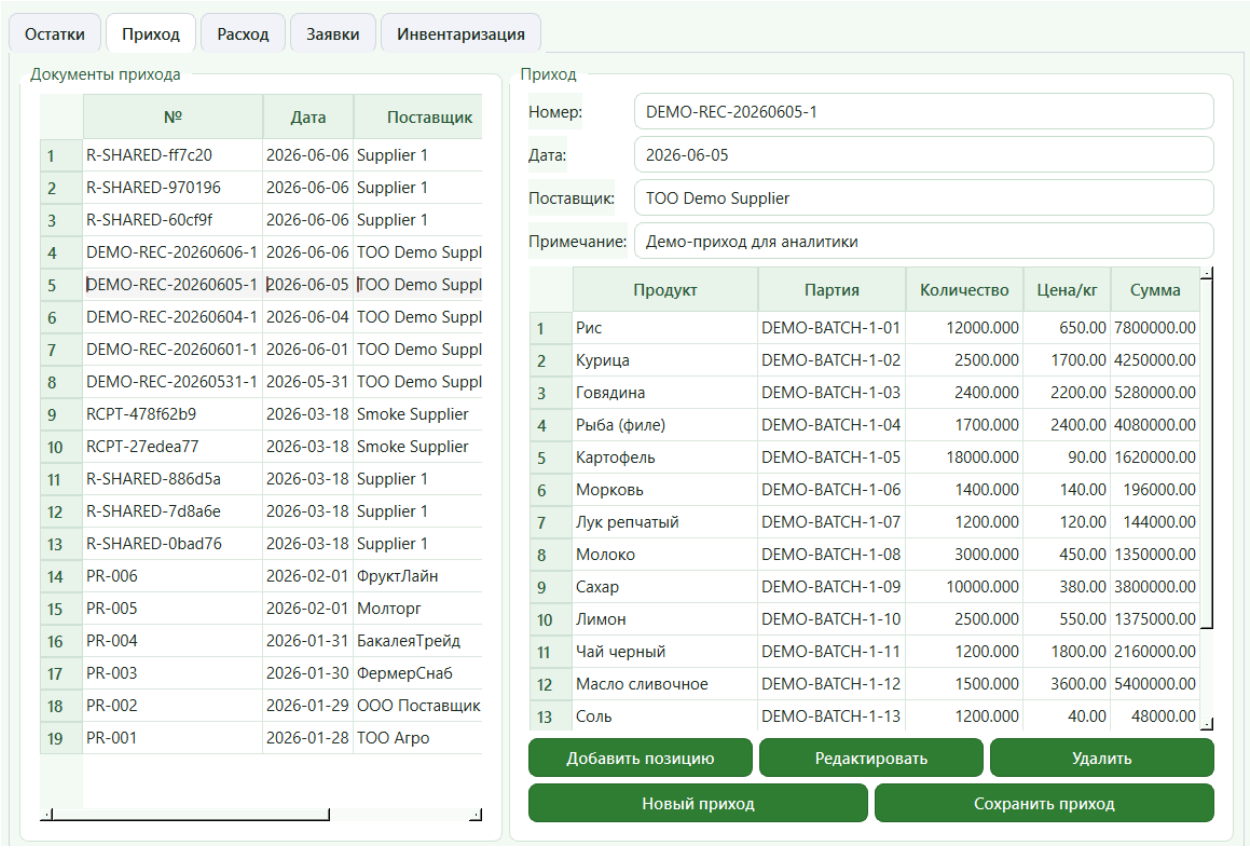


Рисунок 6.5.2.2 – Регистрация прихода продуктов

Для оформления выдачи продуктов используется вкладка «Расход». Кладовщик создаёт новый расход, выбирает соответствующую заявку, добавляет выдаваемые складские партии и сохраняет документ.

На вкладке «Инвентаризация» кладовщик создаёт акт, заполняет системные остатки, вводит фактическое количество продуктов и сохраняет результаты проверки. Интерфейс инвентаризации показан на рисунке 6.5.2.3.

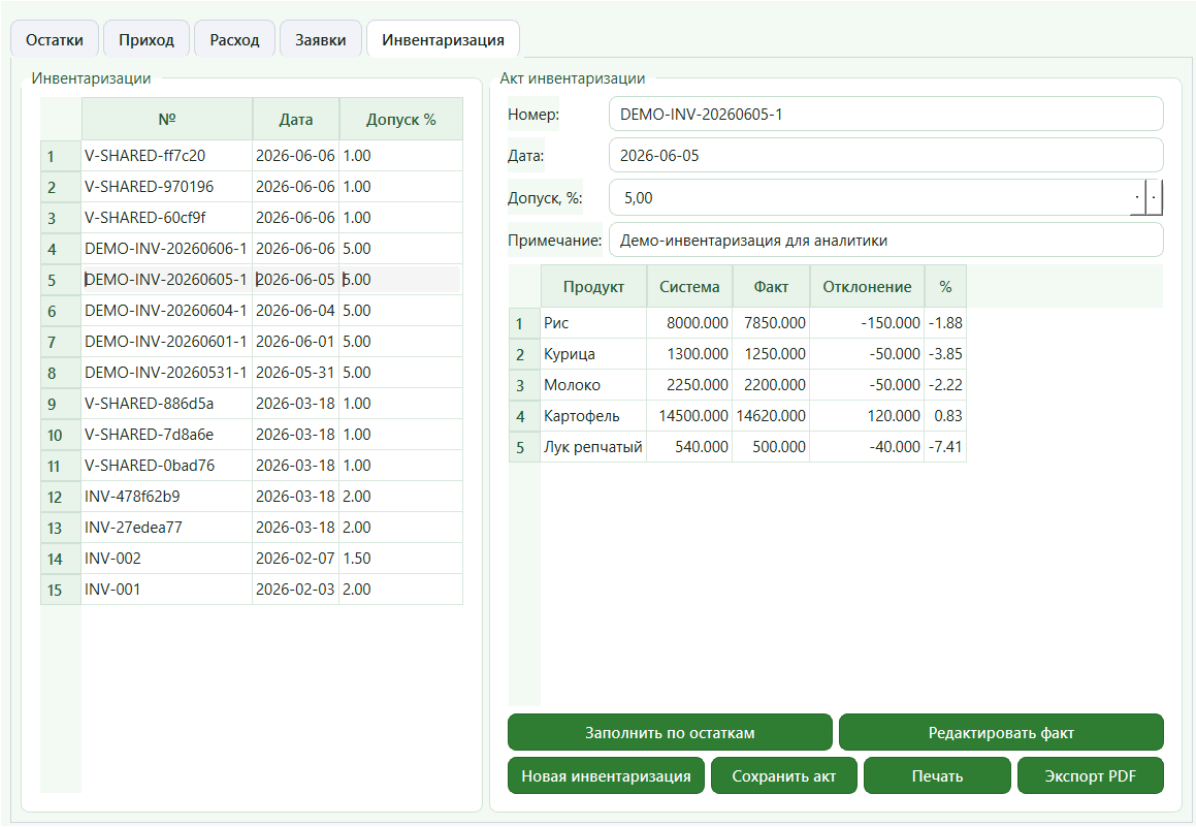


Рисунок 6.5.2.3 – Проведение складской инвентаризации

Для завершения работы в настольном приложении необходимо нажать кнопку «Выход», расположенную на панели управления. После нажатия текущий сеанс завершается, и пользователь возвращается к окну авторизации.

6.6 Работа повара

Повар использует веб-интерфейс и настольное приложение для просмотра утверждённых технологических карт и меню. Изменение технологических карт повару недоступно.

6.6.1 Работа повара в веб-интерфейсе

После авторизации повару открывается раздел просмотра утверждённых технологических карт. В левой части страницы расположен поиск и перечень блюд. После выбора блюда в правой части отображаются фотография, выход, пищевая ценность, стоимость, состав и технология приготовления, как показано на рисунке 6.6.1.

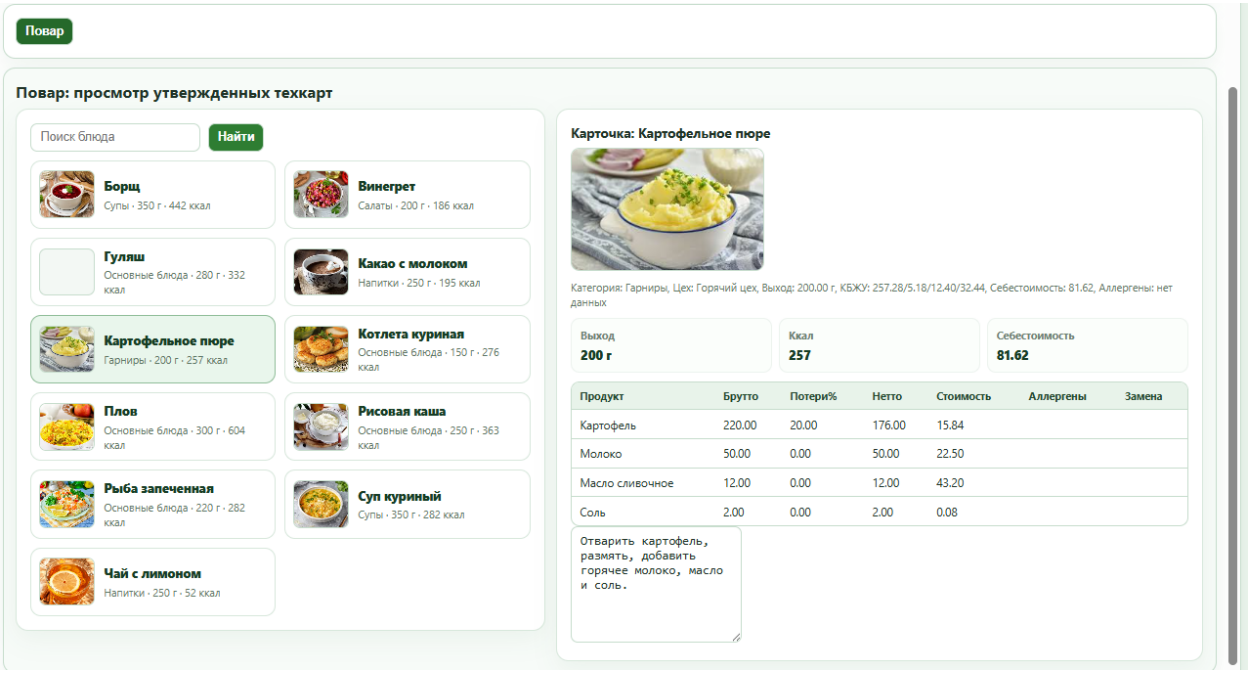


Рисунок 6.6.1 – Просмотр технологической карты поваром в веб-интерфейсе

Для поиска блюда повар вводит часть наименования в поле «Поиск блюда» и нажимает кнопку «Найти». Затем необходимо выбрать карточку нужного блюда и ознакомиться с его составом и технологией приготовления.

6.6.2 Работа повара в настольном приложении

В настольном приложении повару доступны разделы «Техкарты», «Продукты» и «Меню» в режиме просмотра. Кнопки создания, редактирования и удаления для повара скрыты.

В разделе «Техкарты» повар может выбрать блюдо и просмотреть его состав, выход, пищевую ценность, фотографию и технологию приготовления. Интерфейс настольного приложения повара представлен на рисунке 6.6.2.

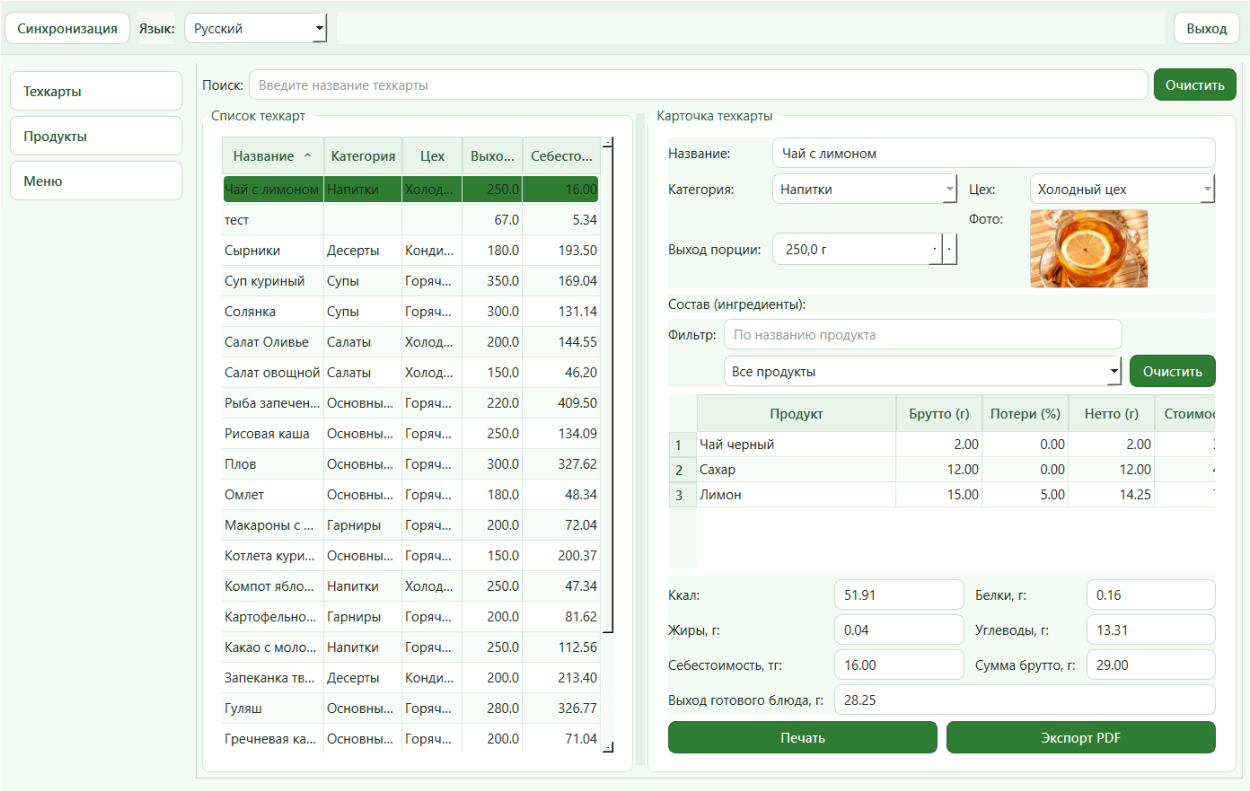


Рисунок 6.6.2 – Просмотр технологической карты в настольном приложении повара

В разделе «Меню» повар может просматривать блюда, запланированные на выбранную дату, количество порций и сведения о наличии продуктов. При необходимости меню можно распечатать или экспортировать в PDF.

6.7 Работа руководителя

Руководитель работает через веб-интерфейс. После авторизации ему доступен раздел «Отчётность», представленный на рисунке 6.7.1, 6.7.2.

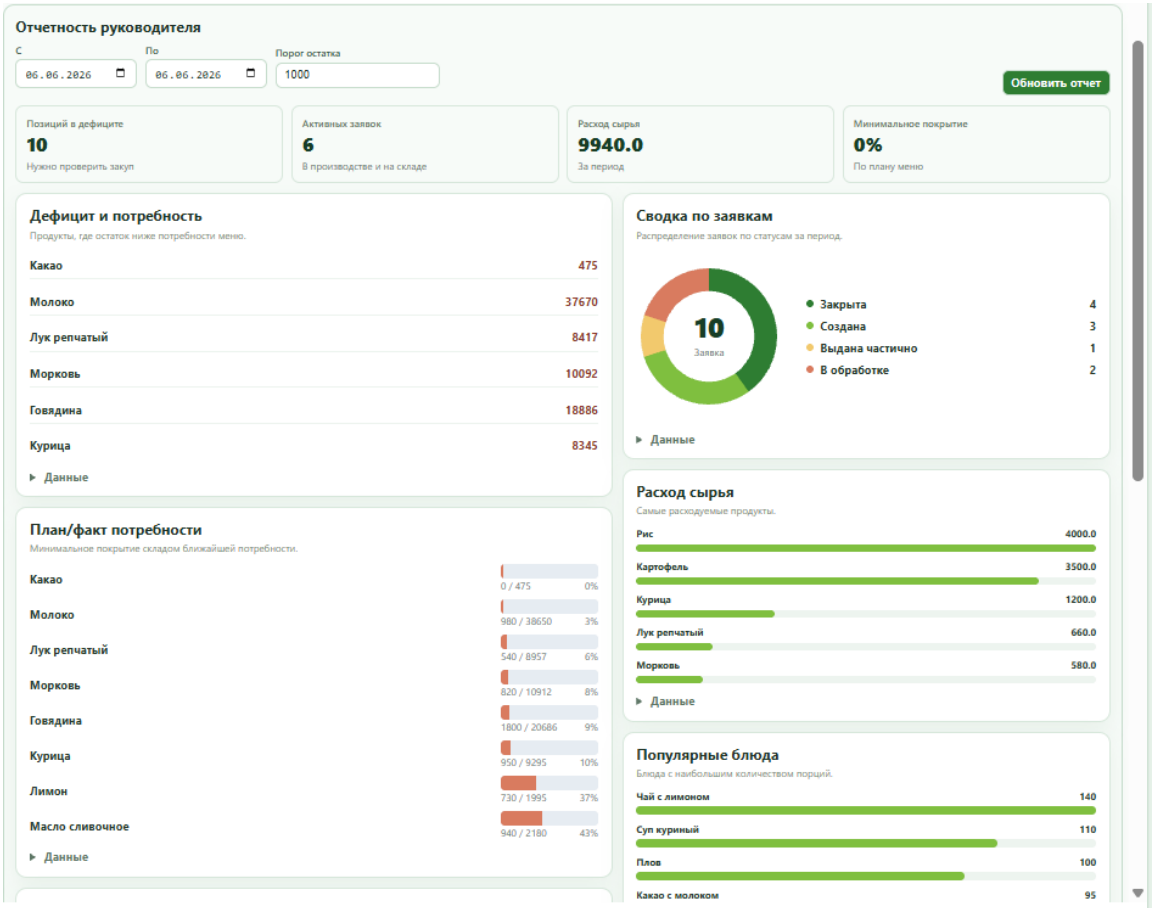


Рисунок 6.7.1 – Начало раздела отчётности руководителя

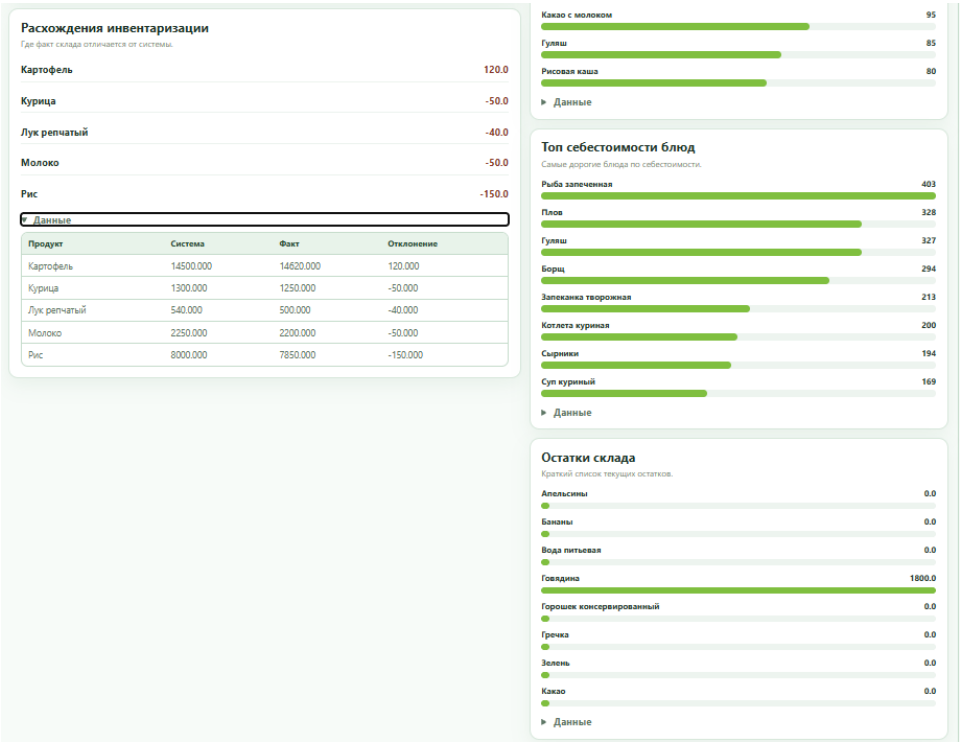


Рисунок 6.7.2 – Раздел отчётности руководителя

Для формирования отчёта руководитель выбирает начальную и конечную даты, указывает порог складского остатка и нажимает кнопку «Обновить отчёт».

В разделе отображаются:

- продукты с недостаточным остатком;
- количество активных заявок;
- расход сырья за выбранный период;
- обеспеченность меню складскими остатками;
- расхождения инвентаризации;
- популярные блюда;
- блюда с наибольшей себестоимостью;
- текущие складские остатки.

6.8 Завершение работы

После завершения работы в веб-интерфейсе необходимо нажать кнопку «Выйти» в верхней части страницы. Для завершения работы в настольном приложении следует нажать кнопку «Выход» или закрыть главное окно программы

Не рекомендуется оставлять личный кабинет открытым на компьютере, доступном другим сотрудникам.

6.9 Действия при возникновении ошибок

Если сайт не открывается, необходимо проверить подключение компьютера к сети Интернет и правильность адреса сайта. Если проблема сохраняется, следует обратиться к администратору организации.

Если система сообщает об ошибке авторизации, необходимо проверить выбранное предприятие, логин и пароль. После нескольких неудачных попыток следует обратиться к администратору для проверки учётной записи.

Если данные в веб-интерфейсе не обновились после действий в настольном приложении, необходимо нажать кнопку «Обновить» в соответствующем разделе.

Если при сохранении документа отображается сообщение об ошибке, необходимо проверить заполнение обязательных полей и корректность введённых значений. При повторном возникновении ошибки пользователю следует записать последовательность выполненных действий, сделать снимок экрана с сообщением и передать сведения администратору системы.

ЗАКЛЮЧЕНИЕ

В ходе выполнения выпускной квалификационной работы было разработано программное средство MealSync, предназначенное для автоматизации процессов общественного питания. Система охватывает работу с технологическими картами, продуктами, меню, складскими заявками, приходными и расходными документами, инвентаризацией, аналитическими отчетами и публичным отображением меню. Поставленная цель работы достигнута полностью: разработана и реализована клиент-серверная система, которая может использоваться сотрудниками через десктоп-приложение и веб-интерфейс, а внешними пользователями — через публичную страницу меню.

В ходе работы была проанализирована предметная область и определены основные роли пользователей: технолог, руководитель, кладовщик, повар и внешний пользователь. Для каждой роли были выделены доступные сценарии работы, что позволило сформировать функциональные требования к программному средству. На этапе проектирования были описаны входные, промежуточные и выходные данные, разработана структура базы данных, построены UML-диаграммы и определена архитектура взаимодействия компонентов системы.

В результате была реализована информационная система MealSync. В состав программного средства вошли десктоп-приложение, серверная часть на базе FastAPI, веб-интерфейс кабинета сотрудника, публичное меню, локальное хранилище SQLite, централизованная база данных, модуль синхронизации, аналитический модуль и модуль формирования PDF-документов. Такая структура обеспечивает разделение пользовательского интерфейса, бизнес-логики, хранения данных и формирования отчетных материалов.

Особое внимание в работе было уделено расчетным алгоритмам. В системе реализованы вычисления массы нетто ингредиентов, КБЖУ, себестоимости блюд, потребности в сырье по меню, складских остатков и отклонений по результатам инвентаризации. За счет автоматизации этих расчетов уменьшается вероятность ошибок при ручной обработке данных и обеспечивается согласованность между технологическими картами, меню и складскими документами.

Была разработана база данных, включающая 24 основные таблицы. Она обеспечивает хранение сведений о пользователях, продуктах, категориях, единицах измерения, технологических картах, меню, складских операциях, заявках, партиях товаров и инвентаризации. Для описания структуры хранения была построена ER-диаграмма, а также приведено описание таблиц, полей, первичных и внешних ключей.

Разработанное программное средство имеет следующий объем: 72 основных файла кода в десктоп, backend и веб-частях, около 15 326 строк программного кода, 69 Python-файлов, 166 классов, 520 функций и методов, 9 модулей серверных API-endpoint и 3 основные веб-страницы. Эти показатели отражают фактический объем реализованной программной системы без учета служебных файлов, изображений, базы данных и кэша.

Проверка работоспособности системы была выполнена путем ручного тестирования основных пользовательских сценариев. Были проверены авторизация, разграничение доступа по ролям, работа с продуктами и технологическими картами, расчетные функции, формирование меню, складские операции, инвентаризация, аналитические отчеты, синхронизация и экспорт PDF. По результатам тестирования были выполнены доработки демонстрационных данных и аналитических отчетов, что позволило обеспечить полноценное отображение информации по текущей дате и выбранному филиалу.

Основные выводы по результатам работы:

1. Разработана и реализована программная система MealSync, предназначенная для автоматизации технологического, складского и аналитического контуров предприятия общественного питания.
2. Поставленная цель выпускной квалификационной работы достигнута полностью, а сформулированные задачи решены в рамках реализованной клиент-серверной архитектуры.
3. Реализованы основные функциональные модули: технологические карты, продукты, меню, публичное меню, складской учет, заявки, инвентаризация, аналитика, синхронизация и формирование PDF-документов.
4. Разработана структура базы данных из 24 таблиц, обеспечивающая хранение и взаимосвязь основных сущностей предметной области.
5. Реализованы расчетные алгоритмы, необходимые для автоматического определения массы нетто, КБЖУ, себестоимости, потребности в сырье, складских остатков и инвентаризационных отклонений.
6. Проведенное тестирование подтвердило корректность работы основных пользовательских сценариев и устойчивость системы при работе с тестовыми данными.

Практическая значимость разработанного программного средства заключается в сокращении ручных операций при ведении технологических карт, формировании меню, подготовке заявок на склад и анализе остатков. MealSync позволяет централизованно хранить данные, выполнять автоматические расчеты, контролировать движение продуктов и предоставлять актуальную информацию как сотрудникам, так и внешним пользователям через публичное меню.

Апробация программного средства была выполнена на тестовом стенде. Были запущены серверная часть, десктоп-приложение и веб-интерфейс, проверены основные роли пользователей и заполнены демонстрационные данные для отображения технологических карт, меню, складских документов и аналитических отчетов. Публикации, акты внедрения и регистрация промышленного внедрения в рамках данной работы не оформлялись.

Дальнейшее развитие MealSync может быть связано с добавлением мобильного интерфейса, интеграцией с внешними учетными системами, расширением аналитических отчетов, внедрением уведомлений, автоматическим резервным копированием и поддержкой нескольких организаций в единой серверной инфраструктуре. Заложенная архитектура позволяет развивать систему без полной переработки ее основных компонентов.

Таким образом, MealSync представляет собой завершенное программное средство, соответствующее поставленной цели и задачам выпускной квалификационной работы. Разработанная система имеет практическую значимость для автоматизации процессов общественного питания и может использоваться как основа для дальнейшего развития и внедрения в эксплуатацию.

СПИСОК ИСПОЛЬЗУЕМЫХ ИСТОЧНИКОВ

1 ГОСТ 31987-2012. Услуги общественного питания. Технологические документы на продукцию общественного питания. Общие требования к оформлению, построению и содержанию. – Введ. 2015-01-01. – Текст : электронный. – URL: https://docs.cntd.ru/document/1200103473 (дата обращения: 22.02.2026).

2 ГОСТ 31985-2013. Услуги общественного питания. Термины и определения. – Введ. 2015-01-01. – Текст : электронный. – URL: https://docs.cntd.ru/document/1200103472 (дата обращения: 24.02.2026).

3 ГОСТ 19.201-78. Единая система программной документации. Техническое задание. Требования к содержанию и оформлению. – Введ. 1980-01-01. – Текст : электронный. – URL: https://docs.cntd.ru/document/1200007624 (дата обращения: 12.03.2026).

4 ГОСТ 7.32-2017. Система стандартов по информации, библиотечному и издательскому делу. Отчет о научно-исследовательской работе. Структура и правила оформления. – Введ. 2018-07-01. – Текст : электронный. – URL: https://docs.cntd.ru/document/1200157208 (дата обращения: 26.03.2026).

5 OMG Unified Modeling Language. Version 2.5.1 : specification / Object Management Group. – 2017. – Текст : электронный. – URL: https://www.omg.org/spec/UML/2.5.1/PDF (дата обращения: 27.03.2026).

6 Python 3 Documentation : official documentation / Python Software Foundation. – Текст : электронный. – URL: https://docs.python.org/3/ (дата обращения: 02.06.2026).

7 FastAPI Documentation : official documentation. – Текст : электронный. – URL: https://fastapi.tiangolo.com/ (дата обращения: 02.06.2026).

8 PostgreSQL Documentation : official documentation / The PostgreSQL Global Development Group. – Текст : электронный. – URL: https://www.postgresql.org/docs/ (дата обращения: 03.04.2026).

9 SQLAlchemy Documentation : official documentation. – Текст : электронный. – URL: https://docs.sqlalchemy.org/ (дата обращения: 02.06.2026).

10 Qt for Python Documentation : official documentation / The Qt Company. – Текст : электронный. – URL: https://doc.qt.io/qtforpython-6/ (дата обращения: 02.03.2026).

ПРИЛОЖЕНИЯ

Приложение А

Разработка программного комплекса «MealSync» для автоматизации бизнес-процессов организации общественного питания ТОО «СУПЕРВКУС»

Организация-заказчик: Товарищество с ограниченной ответственностью "СУПЕРВКУС" (ТОО "СУПЕРВКУС").

Обоснование разработки. Столовая «Супервкус» осуществляет ежедневное приготовление большого количества блюд. В организации отсутствовало специализированное программное обеспечение, охватывающее полный производственный контур: ведение технологических карт, расчет пищевой ценности блюд (КБЖУ) и себестоимости, формирование меню, учет складских остатков, обработку заявок на склад и проведение инвентаризации. Документация велась вручную и в разрозненных файлах Excel, что приводило к высокой вероятности ошибок, затрудненному обновлению рецептур, несоответствиям по остаткам, потере данных и увеличению времени на оформление документов. Существующие аналоги (1С Общепит, R-Keeper) являются комплексными и дорогостоящими системами, требующими лицензирования и сложного внедрения. Разрабатываемый программный комплекс «MealSync» реализует единую цифровую среду работы для технолога, повара, кладовщика и руководителя: настольный клиент для производственных операций, веб-модуль для публикации меню и рабочих кабинетов сотрудников, а также серверный API-слой с общей базой данных. В проекте обеспечена поддержка нескольких столовых: справочники и техкарты ведутся централизованно, а складской учет, заявки, расход и инвентаризация выполняются раздельно по каждому предприятию. Реализация комплекса позволяет снизить трудозатраты на оформление документов, повысить точность расчетов и обеспечить актуальность данных во всех каналах доступа.

Область применения программного средства. Комплекс «MealSync» применяется в столовых, кулинариях и на небольших предприятиях общественного питания. Система применяется для ведения рецептурной базы, расчета КБЖУ и себестоимости блюд, планирования и публикации меню, управления заявками между производством и складом, учета остатков и формирования отчетности по филиалам. Эксплуатация возможна в локальном формате (desktop) и в клиент-серверном формате (desktop + web + API) с единым источником данных.

Функциональные требования к desktop-приложению:

1. Система должна обеспечивать авторизацию сотрудника по логину и паролю с определением роли пользователя (технолог, кладовщик, повар, руководитель).
2. Для ролей с межфилиальным доступом система должна предоставлять выбор предприятия при входе.
3. Система должна вести единые справочники (продукты, единицы измерения, категории блюд, цеха) с операциями добавления, редактирования, удаления и поиска.
4. Система должна обеспечивать ведение технологических карт: создание, редактирование, удаление, просмотр состава блюда и технологии приготовления.
5. По каждой технологической карте система должна выполнять автоматические расчёты: брутто, потери, нетто, выход блюда, КБЖУ, себестоимость.
6. Система должна обеспечивать формирование меню по дням/неделям на основе технологических карт с распределением по приёмам пищи.
7. Система должна обеспечивать публикацию/утверждение меню и передачу данных в веб-часть для отображения посетителям и сотрудникам.

8. Для складского контура система должна обеспечивать учёт прихода, партий, остатков, расхода и движение сырья по документам.
9. Система должна обеспечивать работу с заявками между технологом и кладовщиком: создание, изменение статусов, частичная и полная выдача.
10. Система должна обеспечивать проведение инвентаризации с фиксацией факта, расчётом расхождений и допусков.
11. Система должна формировать печатные формы и экспорт данных в PDF и Excel.
12. Система должна поддерживать локальное и серверное хранение данных.

Функциональные требования к web-приложению:

1. Система должна предоставлять публичную страницу меню без авторизации с просмотром меню на сегодня и на неделю.
2. Публичная часть должна поддерживать фильтрацию по предприятию, дате.
3. Публичная карточка блюда должна отображать: наименование, категорию, выход порции, КБЖУ, цену.
4. Система должна предоставлять отдельный вход для сотрудников с ролевым доступом.
5. После входа пользователь должен видеть только модули, разрешённые его роли.
6. Роль технолог должна иметь доступ к просмотру/утверждению меню и формированию заявок на склад.
7. Роль кладовщик должна иметь доступ к обработке заявок: просмотр, смена статуса, подтверждение выдачи.
8. Роль повар должна иметь доступ к просмотру утверждённых техкарт: состав, нормы закладки, выход, КБЖУ, технологические шаги.
9. Роль руководитель должна иметь доступ к аналитике и отчётности по каждому предприятию.
10. Для ролей с межфилиальным доступом система должна предоставлять переключение предприятия в интерфейсе.
11.

Требования к графическому пользовательскому интерфейсу:

1. Панель навигации (боковое меню). Панель навигации обеспечивает переход между модулями в соответствии с ролью пользователя и текущим контекстом предприятия. Для технолога доступны разделы «Техкарты», «Продукты», «Меню», «Заявки», «Справочники»; для кладовщика предусмотрен модуль «Склад», содержащий вкладки «Остатки», «Приход», «Расход», «Заявки» и «Инвентаризация» для повара — просмотр утвержденных техкарт и производственного меню; для руководителя — аналитические панели и сводная отчетность по филиалам. В верхней области интерфейса размещается переключение языка и, для межфилиальных ролей, выбор активного предприятия.
2. Рабочая область (центральная панель). Рабочая область отображает содержимое выбранного модуля: таблицы реестров, карточки объектов, формы ввода, статусы документов и итоговые показатели. Интерфейс динамически изменяется при выборе раздела, смене роли, переключении филиала и фильтрации данных. Для всех форм обеспечивается единая логика взаимодействия: выбор записи в реестре, просмотр детальной карточки, выполнение операций редактирования и подтверждения. Для операций изменения данных отображаются системные сообщения о результате и предупреждения при ошибках валидации.
3. Панель работы с технологическими картами. В модуле техкарт отображается реестр блюд с фильтрацией и карточка выбранной технологической карты с расчетными блоками.
   • Реестр поддерживает сортировку по наименованию, категории, цеху, выходу,

себестоимости и текстовый поиск по названию блюда.
• Доступные операции: «Создать», «Редактировать», «Удалить», «Сохранить», «Печать», «Экспорт PDF», «Экспорт Excel».
• Форма карты включает поля: наименование блюда, категория, цех, выход порции, состав ингредиентов (продукт, брутто, потери, нетто, стоимость), технологическое описание, служебные метки.
• Итоговый блок выполняет автоматический расчет КБЖУ и себестоимости (на 100 г и на порцию), а также суммарных значений брутто/нетто и выхода готового блюда.

4. Панель формирования меню. Модуль меню предназначен для планирования меню на день/неделю, утверждения и публикации, а также подготовки печатных форм.
• Доступны параметры: дата/период, филиал, тип столовой, статус меню (черновик/утверждено/опубликовано).
• Добавление блюд выполняется из актуального перечня утвержденных техкарт с привязкой к приему пищи и количеству порций.
• Рабочая таблица меню отображает состав по приемам пищи, выход, КБЖУ, цену, сезонные/заменяемые метки, итоговые показатели по дню и периоду.
• Операции модуля: «Сохранить», «Утвердить», «Опубликовать», «Печать», «Экспорт», «Сформировать заявку на склад».
• Публичная web-страница отображает только опубликованное меню с выбором филиала и периода просмотра.
5. Панель работы со справочниками. В системе ведутся единые нормативные справочники: продукты, категории блюд, цеха, единицы измерения.
• Каждый справочник отображается в табличной форме с поиском, сортировкой и контролем уникальности ключевых атрибутов.
• Операции: «Добавить», «Редактировать», «Удалить», «Сохранить изменения».
• Формы ввода содержат обязательную валидацию полей, контроль диапазонов числовых значений и запрет сохранения некорректных данных.
• Изменения нормативных справочников синхронизируются между desktop и web модулями и используются во всех расчетных и документных процессах.
6. Панель складского учета и заявок. Модуль складского контура обеспечивает учет движения сырья и взаимодействие между производством и складом.
• Разделы «Остатки», «Приход», «Расход», «Заявки», «Инвентаризация» доступны в соответствии с ролью и правами доступа.
• Заявки поддерживают жизненный цикл статусов: создана, отправлена, в обработке, подтверждена, выдана частично, закрыта, отклонена.
• В карточке заявки отображаются состав позиций, запрошенное/выданное количество, история изменений статусов, комментарии ответственных лиц.
• Инвентаризация фиксирует учетное количество, фактическое количество, отклонение.
• По каждому филиалу формируются отчеты по остаткам, расходу, заявкам и инвентаризационным расхождениям с экспортом в PDF/Excel.
7. Панель web-доступа для сотрудников и посетителей. Web-интерфейс разделяется на публичную часть и кабинет сотрудников с ролевым доступом.
• Публичная часть предоставляет просмотр меню на сегодня и неделю, фильтрацию по филиалу и типу столовой, печатную форму для посетителей.
• Кабинет сотрудников отображает только модули, разрешенные роли: технолог — меню и заявки; кладовщик — заявки и складские операции; повар — просмотр утвержденных техкарт; руководитель — аналитика и сводные отчеты.
• Для межфилиальных ролей реализован выбор активного предприятия без перезапуска с динамической перезагрузкой данных.
• Реализован контроль сессии: при ошибке авторизации (в том числе 401) выполняется автоматический выход и отображается сообщение о необходимости повторного входа.

Технологии разработки:

Технология	Описание
Язык программирования	Python — основной язык реализации desktop-клиента, backend-сервиса и служебных скриптов.
GUI-фреймворк	PySide6 — реализация desktop-интерфейса; HTML/CSS/JavaScript — реализация веб-интерфейса пользователей и сотрудников.
База данных	Реляционная СУБД (SQLite для локального режима, PostgreSQL для серверного режима и multi-user эксплуатации).
Серверная часть и API	FastAPI (REST API), JWT-аутентификация, маршруты для web/desktop-клиентов и обмена в единой базе.
Архитектурный шаблон	Многоуровневая архитектура: UI/Data/Services (desktop) + API/Services/ORM/Schemas (backend) + web presentation layer.

Сроки разработки ПС:

Ожидается завершение разработки системы до июня 2026 года.

Условия тестирования:

Тестирование выполняется для двух контуров системы: desktop-клиент и web/API-контур, работающих с общей моделью данных по трем предприятиям (филиалам). Проверка проводится поэтапно: модульное тестирование, интеграционное тестирование, системное тестирование и приемочные проверки пользовательских сценариев.

Модульное тестирование проводится для расчетных и сервисных модулей: расчет нетто по брутто и потерям, расчет КБЖУ, расчет себестоимости, валидация форм, переходы статусов заявок, формирование итогов меню и складских остатков.

Интеграционное тестирование проверяет взаимодействие desktop> API> БД> web, включая двустороннюю синхронизацию, обработку офлайн-режима desktop и последующую догрузку изменений после восстановления связи.

Системное тестирование охватывает ролевую модель (технолог, кладовщик, повар, руководитель), изоляцию данных по предприятиям, корректность общих справочников и корректность филиальных данных склада/документов.

Приемочное тестирование выполняется по сквозным бизнес-сценариям: создание техкарты, формирование меню, публикация меню, создание заявки, выдача со склада, инвентаризация, получение отчетов, экспорт и печать.

Проверка отказоустойчивости должна включать: недоступность API, истечение сессии, ошибочные учетные данные, конфликтные изменения при синхронизации, граничные значения числовых полей.

Проверка интерфейса проводится отдельно для desktop (Windows) и web (актуальные браузеры), корректность отображения кодировки и скрытие недоступных по роли функций.

Проверка производительности выполняется на тестовом наборе повышенного объема и включает время открытия ключевых реестров, время построения отчетов и время операций экспорта.

Аппаратная среда тестирования: процессор не ниже Intel Core i3/Ryzen 3, ОЗУ от 8 ГБ, SSD с запасом не менее 1 ГБ под БД/отчеты/логи, сетевое подключение LAN/Wi-Fi для многопользовательских сценариев.

Критерии успешного прохождения: все обязательные сценарии выполняются без критических ошибок, расчеты совпадают с эталонными значениями, ролевые ограничения соблюдаются, данные между desktop и web синхронизируются корректно.

Типы данных для тестирования ПС:

Тип данных	Состав тестовых данных	Цель проверки
Пользователи и роли	Учетные записи	Авторизация, разграничение прав, выбор предприятия
Предприятия	Несколько предприятий с раздельными складскими данными	Изоляция филиальных данных и корректная фильтрация
Нормативные справочники	Единицы измерения, категории блюд, цеха, типы столовых	Единая нормативная база для всех предприятий
Продукты	Реальные продукты (картофель, молоко, рис, масло и др.) + граничные значения КБЖУ/цены	Корректность справочника, валидация числовых полей
Технологические карты	Не менее 20 техкарт с реальным составом и технологией	Проверка расчетов брутто/нетто/КБЖУ/себестоимости
Состав ингредиентов	Позиции с разными потерями (%), массами и ценами	Проверка формул и итоговых значений
Меню	Дневные и недельные меню по 3 филиалам, разные приемы пищи	Планирование, публикация, отображение в web
Заявки на склад	Заявки с полным и частичным отпуском, разные статусы	Проверка жизненного цикла статусов и логики выдачи
Приход и партии	Документы прихода, партии с датами и стоимостью	Проверка формирования остатков и движения партий
Расход	Документы списания/выдачи по заявкам и вне заявок	Проверка уменьшения остатков и отчетов расхода
Инвентаризация	Документы отклонением и допуском	Проверка расчета расхождений и отчетной формы
Публичное меню web	Опубликованные и неопубликованные дни, выбор предприятия, даты и режима просмотра	Проверка корректности отображения для посетителей
Экспортные данные	PDF/Excel для технологических карт, меню и актов инвентаризации	Проверка форматов, полноты и читаемости отчетов
Локализация	Языковые интерфейсные строки	Проверка переключения языка и отсутствия искажений
Негативные данные	Пустые поля, отрицательные значения, недопустимые проценты/даты	Проверка валидации и сообщений об ошибках

Программное и аппаратное обеспечение для тестирования:

Компонент	Технические требования
Операционная система	Windows 10/11 для desktop-клиента; Linux/Windows для backend-сервиса в контейнерном и нативном запуске.
Процессор	Intel Core i3 или аналог
Хранилище	Не менее 2 ГБ свободного пространства для базы данных, логов, экспортов PDF/XLSX и резервных копий
Оперативная память	от 4 ГБ
Дополнительно	Принтер для проверки печатных форм и сетевое подключение для многопользовательского режима.

Директор ТОО СуперВкус — В.И. Куимов

Руководитель ВКР — О.В. Литвинова

Приложение Б

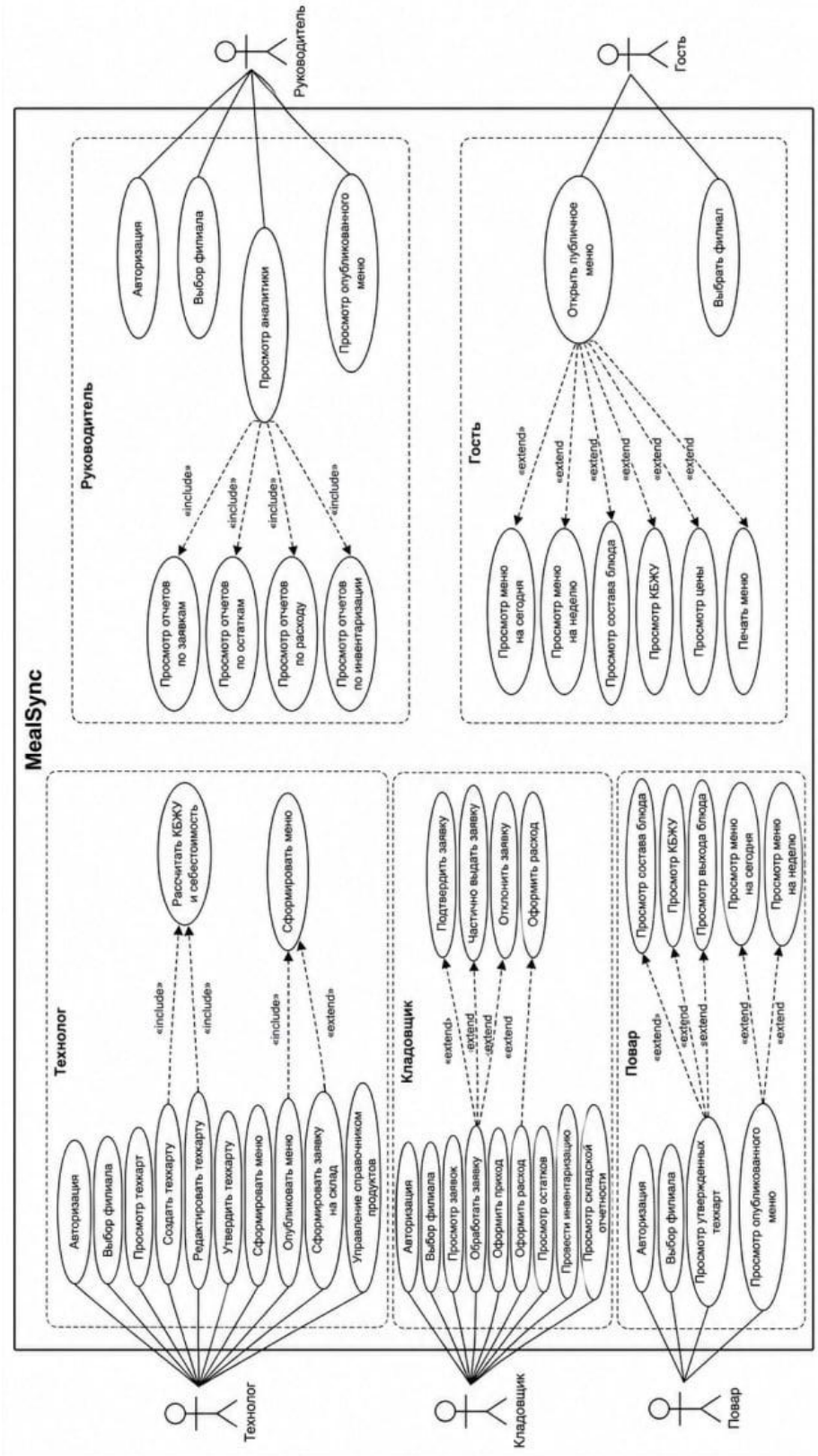


Рисунок Б.1 – Диаграмма вариантов всех пользователей

Приложение В

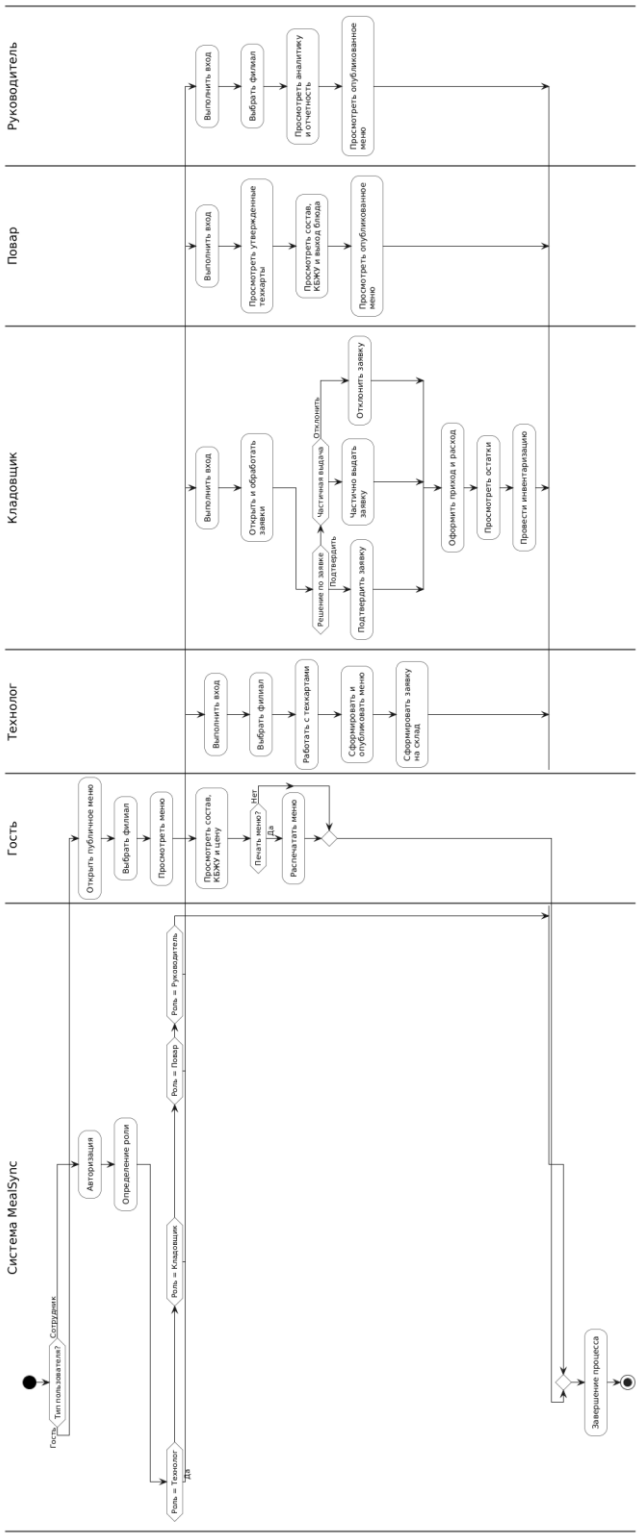


Рисунок В.1 – Диаграмма активности взаимодействия пользователей с программным средством

Приложение Г

В приложении представлен фрагмент программного кода, реализующего работу с технологическими картами блюд. Приведённый код демонстрирует структуру хранения рецептов, проверку поступающих данных, расчёт пищевой ценности и выполнение основных операций с рецептами через программный интерфейс.

Модели хранения рецептов и ингредиентов:

```python
from sqlalchemy import Float, ForeignKey, Integer, String, Text
from sqlalchemy.orm import Mapped, mapped_column

from irecipe_api.db.base import Base


class Recipe(Base):
    __tablename__ = "recipes"

    id: Mapped[int] = mapped_column(primary_key=True)
    name: Mapped[str] = mapped_column(String(200), index=True)
    category_id: Mapped[int] = mapped_column(
        ForeignKey("recipe_categories.id"), nullable=True
    )
    workshop_id: Mapped[int] = mapped_column(
        ForeignKey("workshops.id"), nullable=True
    )
    portion_grams: Mapped[float] = mapped_column(Float, default=0)
    technology_text: Mapped[str] = mapped_column(Text, nullable=True)
    photo_url: Mapped[str] = mapped_column(String(500), nullable=True)


class RecipeItem(Base):
    __tablename__ = "recipe_items"

    id: Mapped[int] = mapped_column(primary_key=True)
    recipe_id: Mapped[int] = mapped_column(
        ForeignKey("recipes.id", ondelete="CASCADE"), index=True
    )
    product_id: Mapped[int] = mapped_column(ForeignKey("products.id"))
    gross_grams: Mapped[float] = mapped_column(Float, default=0)
    loss_percent: Mapped[float] = mapped_column(Float, default=0)
    sort_order: Mapped[int] = mapped_column(Integer, default=0)
```

Модель `Recipe` используется для хранения общей информации о блюде: названия, категории, цеха, массы порции, технологии приготовления и фотографии. Модель `RecipeItem` содержит ингредиенты рецепта, их массу брутто и процент технологических потерь.

Схемы проверки данных рецептов:

```python
from pydantic import BaseModel
```

```
class RecipeOut(BaseModel):
    id: int
    name: str
    category_id: int | None
    workshop_id: int | None
    portion_grams: float
    technology_text: str | None = None
    photo_url: str | None = None

    class Config:
        from_attributes = True


class RecipeCreate(BaseModel):
    name: str
    category_id: int | None = None
    workshop_id: int | None = None
    portion_grams: float = 0.0
    technology_text: str | None = None
    photo_url: str | None = None


class RecipeUpdate(RecipeCreate):
    pass


class RecipeItemOut(BaseModel):
    id: int
    recipe_id: int
    product_id: int
    gross_grams: float
    loss_percent: float
    sort_order: int

    class Config:
        from_attributes = True


class RecipeItemCreate(BaseModel):
    product_id: int
    gross_grams: float
    loss_percent: float
    sort_order: int = 0


class RecipeItemUpdate(BaseModel):
    product_id: int | None = None
    gross_grams: float | None = None
    loss_percent: float | None = None
    sort_order: int | None = None
```

Схемы Pydantic определяют формат данных, принимаемых и возвращаемых сервером. Они обеспечивают автоматическую проверку типов значений и предотвращают передачу некорректных данных.

Расчёт показателей рецепта:

```python
from typing import Any
from sqlalchemy.orm import Session
```

```
from irecipe_api.models.product import Product
from irecipe_api.models.recipe import Recipe, RecipeItem


def _to_float(value: Any) -> float:
    try:
        return float(value or 0)
    except (TypeError, ValueError):
        return 0.0


def compute_recipe_metrics(
    db: Session,
    recipe: Recipe,
) -> dict[str, Any]:
    items = (
        db.query(RecipeItem)
        .filter(RecipeItem.recipe_id == recipe.id)
        .order_by(RecipeItem.sort_order.asc())
        .all()
    )

    product_ids = [item.product_id for item in items]
    products = {
        product.id: product
        for product in db.query(Product)
        .filter(Product.id.in_(product_ids))
        .all()
    }

    calories = 0.0
    protein = 0.0
    fat = 0.0
    carbs = 0.0
    cost = 0.0
    net_total = 0.0

    for item in items:
        product = products.get(item.product_id)
        if not product:
            continue

        gross = _to_float(item.gross_grams)
        loss = _to_float(item.loss_percent)
        net = max(gross * (1 - loss / 100), 0.0)
        coefficient = net / 100

        net_total += net
        calories += _to_float(product.calories) * coefficient
        protein += _to_float(product.protein) * coefficient
        fat += _to_float(product.fat) * coefficient
        carbs += _to_float(product.carbs) * coefficient
        cost += _to_float(product.price_per_kg) * net / 1000

    return {
        "recipe_id": recipe.id,
        "recipe_name": recipe.name,
        "net_total_grams": round(net_total, 2),
        "calories": round(calories, 2),
        "protein": round(protein, 2),
        "fat": round(fat, 2),
        "carbs": round(carbs, 2),
```

```
        "cost": round(cost, 2),
    }
```

Функция получает ингредиенты рецепта и рассчитывает их массу нетто с учётом технологических потерь. На основании массы нетто определяются калорийность, содержание белков, жиров, углеводов и итоговая стоимость блюда.

Операции управления рецептами:

```python
from fastapi import APIRouter, Depends, HTTPException
from sqlalchemy.orm import Session

from irecipe_api.api.deps import get_db, require_roles
from irecipe_api.models.recipe import Recipe, RecipeItem
from irecipe_api.models.user import User
from irecipe_api.schemas.recipe import (
    RecipeCreate,
    RecipeItemCreate,
    RecipeItemOut,
    RecipeOut,
    RecipeUpdate,
)

router = APIRouter(prefix="/recipes", tags=["recipes"])

INTERNAL_ROLES = ("technologist", "storekeeper", "cook", "manager")
EDIT_ROLES = ("technologist", "manager")


@router.get("", response_model=list[RecipeOut])
def list_recipes(
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(*INTERNAL_ROLES)),
) -> list[RecipeOut]:
    return db.query(Recipe).order_by(Recipe.name.asc()).all()


@router.post("", response_model=RecipeOut)
def create_recipe(
    payload: RecipeCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(*EDIT_ROLES)),
) -> RecipeOut:
    recipe = Recipe(**payload.model_dump())
    db.add(recipe)
    db.commit()
    db.refresh(recipe)
    return recipe


@router.put("/{recipe_id}", response_model=RecipeOut)
def update_recipe(
    recipe_id: int,
    payload: RecipeUpdate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(*EDIT_ROLES)),
) -> RecipeOut:
    recipe = db.query(Recipe).filter(Recipe.id == recipe_id).first()
```

```
    if not recipe:
        raise HTTPException(status_code=404, detail="Recipe not found")

    for key, value in payload.model_dump().items():
        setattr(recipe, key, value)

    db.commit()
    db.refresh(recipe)
    return recipe


@router.delete("/{recipe_id}")
def delete_recipe(
    recipe_id: int,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(*EDIT_ROLES)),
) -> dict[str, bool]:
    recipe = db.query(Recipe).filter(Recipe.id == recipe_id).first()

    if not recipe:
        raise HTTPException(status_code=404, detail="Recipe not found")

    db.query(RecipeItem).filter(
        RecipeItem.recipe_id == recipe_id
    ).delete()

    db.delete(recipe)
    db.commit()
    return {"ok": True}


@router.post("/{recipe_id}/items", response_model=RecipeItemOut)
def create_recipe_item(
    recipe_id: int,
    payload: RecipeItemCreate,
    db: Session = Depends(get_db),
    current_user: User = Depends(require_roles(*EDIT_ROLES)),
) -> RecipeItemOut:
    recipe = db.query(Recipe).filter(Recipe.id == recipe_id).first()

    if not recipe:
        raise HTTPException(status_code=404, detail="Recipe not found")

    item = RecipeItem(
        recipe_id=recipe_id,
        **payload.model_dump(),
    )

    db.add(item)
    db.commit()
    db.refresh(item)
    return item
```

Представленные обработчики реализуют получение списка рецептов, создание, изменение и удаление технологических карт, а также добавление ингредиентов. Доступ к операциям изменения данных предоставляется только пользователям с ролями технолога и менеджера.

Авторизация пользователей и разграничение прав доступа:

```python
from collections.abc import Generator

from fastapi import Depends, HTTPException, status
from fastapi.security import OAuth2PasswordBearer
from jose import JWTError, jwt
from sqlalchemy.orm import Session

from irecipe_api.core.config import settings
from irecipe_api.db.session import SessionLocal
from irecipe_api.models.user import User


oauth2_scheme = OAuth2PasswordBearer(
    tokenUrl=f"{settings.api_v1_prefix}/auth/login"
)


def get_db() -> Generator[Session, None, None]:
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()


def get_current_user(
    token: str = Depends(oauth2_scheme),
    db: Session = Depends(get_db),
) -> User:
    credentials_exception = HTTPException(
        status_code=status.HTTP_401_UNAUTHORIZED,
        detail="Не удалось проверить данные пользователя",
    )

    try:
        payload = jwt.decode(
            token,
            settings.secret_key,
            algorithms=["HS256"],
        )
        username: str | None = payload.get("sub")

        if username is None:
            raise credentials_exception
    except JWTError:
        raise credentials_exception

    user = (
        db.query(User)
        .filter(User.username == username)
        .first()
    )

    if not user:
        raise credentials_exception

    return user


def require_roles(*allowed_roles: str):
    allowed_set = set(allowed_roles)
```

```
    def checker(
        current_user: User = Depends(get_current_user),
    ) -> User:
        if current_user.role not in allowed_set:
            raise HTTPException(
                status_code=status.HTTP_403_FORBIDDEN,
                detail="Недостаточно прав для выполнения операции",
            )

        return current_user

    return checker
```

В представленном фрагменте реализовано создание подключения к базе данных и определение текущего пользователя по переданному токену. Функция `require_roles` используется для ограничения доступа к отдельным операциям системы в зависимости от роли сотрудника.

Формирование и проверка токена доступа:

```python
from datetime import datetime, timedelta, timezone

from jose import jwt
from passlib.context import CryptContext

from irecipe_api.core.config import settings


pwd_context = CryptContext(
    schemes=["pbkdf2_sha256"],
    deprecated="auto",
)


def verify_password(
    plain_password: str,
    hashed_password: str,
) -> bool:
    return pwd_context.verify(
        plain_password,
        hashed_password,
    )


def get_password_hash(password: str) -> str:
    return pwd_context.hash(password)


def create_access_token(subject: str) -> str:
    expire = datetime.now(timezone.utc) + timedelta(
        minutes=settings.access_token_expire_minutes
    )

    payload = {
        "sub": subject,
        "exp": expire,
    }
```

```
    return jwt.encode(
        payload,
        settings.secret_key,
        algorithm="HS256",
    )
```

Пароли пользователей хранятся в виде криптографических хешей. После успешной проверки имени пользователя и пароля сервер формирует JWT-токен с ограниченным сроком действия.

Обработка входа пользователя в систему

```python
from fastapi import APIRouter, Depends, HTTPException
from fastapi.security import OAuth2PasswordRequestForm
from sqlalchemy.orm import Session

from irecipe_api.api.deps import get_current_user, get_db
from irecipe_api.core.security import (
    create_access_token,
    verify_password,
)
from irecipe_api.models.user import User
from irecipe_api.schemas.auth import Token, UserOut


router = APIRouter(prefix="/auth", tags=["auth"])


@router.post("/login", response_model=Token)
def login(
    form_data: OAuth2PasswordRequestForm = Depends(),
    db: Session = Depends(get_db),
) -> Token:
    user = (
        db.query(User)
        .filter(User.username == form_data.username)
        .first()
    )

    if not user:
        raise HTTPException(
            status_code=400,
            detail="Неверное имя пользователя или пароль",
        )

    if not verify_password(
        form_data.password,
        user.password_hash,
    ):
        raise HTTPException(
            status_code=400,
            detail="Неверное имя пользователя или пароль",
        )

    token = create_access_token(user.username)
    return Token(access_token=token)


@router.get("/me", response_model=UserOut)
```

```
def get_user_information(
    current_user: User = Depends(get_current_user),
) -> UserOut:
    selectable = (
        int(current_user.enterprise_id or 0) == 0
        and current_user.role in {"technologist", "manager"}
    )

    return UserOut(
        id=current_user.id,
        username=current_user.username,
        role=current_user.role,
        enterprise_id=int(current_user.enterprise_id or 0),
        enterprise_selectable=selectable,
    )
```

Обработчик `/login` выполняет поиск пользователя, проверяет введённый пароль и возвращает токен доступа. Обработчик `/me` предоставляет клиентскому приложению сведения об авторизованном сотруднике и его роли.

Загрузка фотографии блюда:

```python
from pathlib import Path
from uuid import uuid4

from fastapi import Depends, File, HTTPException, UploadFile
from sqlalchemy.orm import Session

from irecipe_api.api.deps import get_db, require_roles
from irecipe_api.models.recipe import Recipe
from irecipe_api.models.user import User
from irecipe_api.schemas.recipe import RecipeOut


MAX_PHOTO_SIZE = 5 * 1024 * 1024

ALLOWED_MIME = {
    "image/jpeg",
    "image/png",
    "image/webp",
    "image/gif",
}

ALLOWED_EXT = {
    ".jpg",
    ".jpeg",
    ".png",
    ".webp",
    ".gif",
}


def get_photo_directory() -> Path:
    path = (
        Path(__file__).resolve().parents[4]
        / "media"
        / "recipe-images"
    ).resolve()
```

```
    path.mkdir(parents=True, exist_ok=True)
    return path


def get_safe_photo_extension(
    filename: str,
    content_type: str | None,
) -> str:
    extension = Path(filename or "").suffix.lower()

    if extension in ALLOWED_EXT:
        if extension == ".jpeg":
            return ".jpg"
        return extension

    mime_type = (content_type or "").lower()

    if mime_type == "image/png":
        return ".png"
    if mime_type == "image/webp":
        return ".webp"
    if mime_type == "image/gif":
        return ".gif"

    return ".jpg"


@router.post(
    "/{recipe_id}/photo",
    response_model=RecipeOut,
)
async def upload_recipe_photo(
    recipe_id: int,
    file: UploadFile = File(...),
    db: Session = Depends(get_db),
    current_user: User = Depends(
        require_roles("technologist", "manager")
    ),
) -> RecipeOut:
    recipe = (
        db.query(Recipe)
        .filter(Recipe.id == recipe_id)
        .first()
    )

    if not recipe:
        raise HTTPException(
            status_code=404,
            detail="Рецепт не найден",
        )

    content_type = (file.content_type or "").lower()

    if content_type not in ALLOWED_MIME:
        raise HTTPException(
            status_code=400,
            detail="Разрешена загрузка только изображений",
        )

    image_data = await file.read(MAX_PHOTO_SIZE + 1)

    if len(image_data) > MAX_PHOTO_SIZE:
        raise HTTPException(
```

```
            status_code=400,
            detail="Размер изображения превышает 5 Мбайт",
        )

    if not image_data:
        raise HTTPException(
            status_code=400,
            detail="Загружен пустой файл",
        )

    extension = get_safe_photo_extension(
        file.filename or "",
        content_type,
    )

    filename = (
        f"recipe_{recipe_id}_{uuid4().hex}{extension}"
    )

    photo_path = get_photo_directory() / filename
    photo_path.write_bytes(image_data)

    recipe.photo_url = (
        f"/media/recipe-images/{filename}"
    )

    db.commit()
    db.refresh(recipe)

    return recipe
```

При загрузке фотографии сервер проверяет формат файла и ограничивает его максимальный размер. Для исключения конфликтов имён каждому изображению присваивается уникальное имя. После сохранения файла ссылка на фотографию записывается в технологическую карту.

Формирование опубликованного меню:

```python
from datetime import date

from fastapi import APIRouter, Depends, Query
from sqlalchemy.orm import Session

from irecipe_api.api.deps import get_db
from irecipe_api.models.menu_stock import (
    MenuDay,
    MenuDayMeta,
    MenuItem,
)
from irecipe_api.schemas.public import PublicMenuDayOut
from irecipe_api.services.recipe_metrics import (
    compute_recipes_metrics_map,
)


router = APIRouter(prefix="/public", tags=["public"])
```

```
def build_menu_day(
    db: Session,
    day: MenuDay,
    meta: MenuDayMeta,
) -> dict:
    menu_items = (
        db.query(MenuItem)
        .filter(MenuItem.menu_id == day.id)
        .order_by(MenuItem.meal_type.asc())
        .all()
    )

    recipe_ids = [
        item.recipe_id
        for item in menu_items
    ]

    metrics = compute_recipes_metrics_map(
        db,
        recipe_ids,
    )

    output_items = []

    for item in menu_items:
        recipe = metrics.get(item.recipe_id)

        if not recipe:
            continue

        output_items.append(
            {
                "menu_item_id": item.id,
                "meal_type": item.meal_type,
                "recipe_name": recipe["recipe_name"],
                "photo_url": recipe["photo_url"],
                "portions": item.portions,
                "portion_grams": recipe["portion_grams"],
                "calories": recipe["calories"],
                "protein": recipe["protein"],
                "fat": recipe["fat"],
                "carbs": recipe["carbs"],
                "cost": recipe["cost"],
            }
        )

    return {
        "menu_day_id": day.id,
        "menu_date": day.menu_date,
        "enterprise_id": day.enterprise_id,
        "canteen_type": meta.canteen_type,
        "items": output_items,
    }


@router.get(
    "/menu/today",
    response_model=list[PublicMenuDayOut],
)
def get_public_menu(
    menu_date: date | None = Query(default=None),
    enterprise_id: int | None = Query(default=None),
    db: Session = Depends(get_db),
```

```
) -> list[dict]:
    target_date = menu_date or date.today()

    query = (
        db.query(MenuDay, MenuDayMeta)
        .join(
            MenuDayMeta,
            MenuDayMeta.menu_day_id == MenuDay.id,
        )
        .filter(MenuDay.menu_date == target_date)
        .filter(MenuDayMeta.is_published.is_(True))
    )

    if enterprise_id is not None:
        query = query.filter(
            MenuDay.enterprise_id == enterprise_id
        )

    menu_days = query.order_by(MenuDay.id.asc()).all()

    return [
        build_menu_day(db, day, meta)
        for day, meta in menu_days
    ]
```

Формирование публичного меню выполняется только для опубликованных записей. Для каждого блюда рассчитываются пищевая ценность, стоимость и размер порции. Полученные сведения используются веб-интерфейсом для отображения актуального меню посетителям столовой.